

17 · Theoretische Informatik

Modul: Theoretische Informatik (Algorithmen, Datenstrukturen, Automatentheorie) **Dozent:** Prof. Dr. Aykan D. Inan — aykan.inan@hnu.de **Studiengang:** DHBW Ravensburg · WDS24 · Data Science und Künstliche Intelligenz **Semester:** WS 2025 (Start 06.10.2025) ⚠️ **Prüfungsform:** PORTFOLIO-PRÜFUNG — KEINE Klausur!

Quellenlage dieser Analyse

Datei	Seiten	Inhalt
Kapitel_00_Organisatorisches.pdf	79	Organisation, Portfolio-Prüfung, 3 Gedankenexperimente
Kapitel_01_Geschichte.pdf	618	Geschichte der Informatik (30.000 v. Chr. – 2025)
Kapitel_02_Grundlagen.pdf	512	Algorithmusbegriff, Datentypen, Zahlendarstellung, Datenstrukturen, Mathe-Grundlagen
Kapitel_03_Laufzeitkomplexität.pdf	176	O-Notation, Komplexitätsklassen, Rekursion, Master-Theorem
Kapitel_04_1...4_Sortieren.pdf	534+289+265+551	13 Sortieralgorithmen + untere Schranke
Kapitel_05_Suchen.pdf	240	6 Suchverfahren + binärer Suchbaum
Kapitel_06_Hashing.pdf	256	Hashfunktionen, Kollisionsbehandlung, Analyse
Kapitel_07_1...3_Graphen.pdf	276+553+363	Graphentheorie, MST, kürzeste Wege, TSP, Heuristiken
Kapitel_08_Sprachen_Automaten.pdf	525	Formale Sprachen, Chomsky-Hierarchie, DEA/NEA, RegEx, Pumping-Lemma
Lösung_Gedankenexperiment_2/3.pdf	2+1	Musterlösungen (12 Kugeln, Stadtrundgang)
02_Gruppen_WS25.pdf	1	Gruppeneinteilung + 13 Projektthemen
Videos/*.mp4	—	Sortier-Visualisierungen, TSP-2-opt, Barack Obama (Sortierfrage)

Zum Dozenten

2009–2012 Duales Studium (ETTI) an der Universität der Bundeswehr München über den **BND** · 2012–2015 **Bundesnachrichtendienst** · Parallelstudium IT-Sicherheit an der Universität Bochum (isits AG) · 2017–2023 Wiss. Mitarbeiter und **Doktorand** an der Hochschule Ravensburg-Weingarten, Fakultät EI, Fachbereich **IT-Sicherheit/Forensik** — **Kryptanalyse von RSA** · 2023–2025 **Professor an der TH Nürnberg** · seit März 2025 **Professor an der HNU**



Hinweis für E-Mails: *„Schreiben Sie in den Betreff zusätzlich die Kennung **,DHBW'**, damit diese nicht unbeabsichtigt im SPAM landet.“*

Literatur (die 11 Quellen des Dozenten)

1. **Cormen, Leiserson, Rivest, Stein:** *Introduction to Algorithms* ← die Hauptquelle
2. **Pomberger, Dobler:** *Algorithmen und Datenstrukturen — Eine systematische Einführung in die Programmierung*
3. **Vossen, Witt:** *Grundkurs Theoretische Informatik*
4. **Homuth:** *Einführung in die Automatentheorie*
5. **Hopcroft, Motwani, Ullman:** *Einführung in die Automatentheorie, Formale Sprachen und Komplexitätstheorie*
6. **Böckenhauer, Hromkovič:** *Formale Sprachen*
7. **Pawelczak:** *Start in die technische Informatik*
8. **Schubert:** *Mathematik für Informatiker*
9. **Tanenbaum, Bos:** *Moderne Betriebssysteme*
10. Vorlesungsfolien **Prof. Eggendorfer** (HS Ravensburg-Weingarten) ← Quelle aller Laufzeitmessungen
11. Vorlesungsfolien **Prof. Dr. Matthias Rarey** (Uni Hamburg)

TEIL 0 — DIE PORTFOLIO-PRÜFUNG

0.1 ★★☆☆ Zusammensetzung und Bestehensgrenze

① Projektbericht	60 Punkte
② Präsentation	40 Punkte
Σ	100 Punkte

BESTANDEN $\iff \Sigma \text{ der Punkte} \geq 50$
 NICHT BESTANDEN $\iff \Sigma \text{ der Punkte} < 50$

 **Achtung:** Arbeiten in **Gruppen von 3 bis max. 5 Personen.**

0.2 Rahmenbedingungen Projektbericht

- Der Umfang ergibt sich **anhand der Anzahl der Gruppenmitglieder: jedes Gruppenmitglied hat 3–6 Seiten zu erstellen**
- Die **Gestaltungsrichtlinien wissenschaftlichen Arbeitens** sind zu beachten und einzuhalten
- Die **Selbstständigkeitserklärung** ist ebenfalls mit abzugeben
- **Abgabeform:** digitale Version als **PDF** über den Moodlekurs oder per Mail an den Dozenten
-  „Es muss **nicht an jeder Stelle gekennzeichnet werden, wer was geschrieben hat**. Wichtig ist, dass die geforderte **Mindestseitenanzahl** erfüllt ist.“

0.3 ★ KI-Regelung (wörtlich!)

? Was ist mit KI erlaubt?

- „Alles ist erlaubt, es sei denn, es wird explizit eingeschränkt.“
- „Sie handeln vollständig eigenverantwortlich.“ → Das, was Sie übernehmen, müssen Sie vollständig verantworten

? Muss ich den Einsatz von KI-Werkzeugen dokumentieren?

- „Ja, Sie sind verpflichtet, ALLE Medien, verwendete Hilfsmittel oder Quellen anzugeben, die Sie bei der Lösung einer Aufgabe verwendet haben.“
- „Machen Sie Ihr Vorgehen transparent — nicht nur den Lösungsweg, sondern auch den ZWECK der Anwendung einer KI.“

0.4 Rahmenbedingungen Präsentation

- **20–30 Minuten pro Gruppe** (abhängig von der tatsächlichen Gruppenanzahl) — **ca. 10 Minuten pro Teilnehmer**
- ⚠ Die Kamera muss, solange man vorträgt, eingeschaltet sein — Prüfungsbedingung!
- ⚠ Bei den Präsentationen gilt ANWESENHEITSPFLICHT!
- ⚠ Jede Gruppe muss mindestens EINE FRAGE pro Präsentation stellen!

0.5 ⚠ ⚠ Deadlines (WS 2025)

Termin	Deadline
Gruppeneinteilung + Thema	13.10.2025
Abgabe Präsentation	10.11.2025
Abgabe Projektbericht	10.11.2025
Präsentationen	11.11.2025

⚠ ⚠ „Die Deadlines sind einzuhalten. Wer dagegen verstößt, ist aus dem ‚Rennen‘ raus. D. h. der Kurs gilt damit als NICHT BESTANDEN!“

0.6 ★ Die 6 möglichen Themenbereiche

1. Berechenbarkeit
2. Sortieralgorithmen
3. Suchalgorithmen
4. Hashing
5. Graphentheorie
6. Automatentheorie

Die tatsächlich vergebenen 13 Projektthemen (WS 2025)

G r.	Thema
1	Visualisierung grundlegender Graphensuchalgorithmen mit GUI — Tool zur Animation von Tiefen- und Breitensuche auf benutzerdefinierbaren Graphen (Pfeile, Farben, Schritte)
2	Eulersche Graphen in der Praxis: Routenplanung für Müllabfuhr und Straßenreinigung
3	Suche in unstrukturierten Datenmengen
4	Grafische Aufbereitung von Sortiervorgängen mittels einfacher GUI oder Web-App
5	Suche in Texten — Implementierung und Analyse des Boyer-Moore-Algorithmus
6	Analyse des Einflusses von Kantengewichten auf Kruskal- und Prim-Algorithmen
7	Automatische Generierung von endlichen Automaten aus regulären Ausdrücken
8	Effizienz und Optimierung von Hashing in der Blockchain-Technologie — Merkle-Bäume, schnelle Validierung/Verifikation von Transaktionen
9	Vergleich von Algorithmen zur Berechnung kürzester Wege — Dijkstra, Bellman-Ford, Floyd-Warshall unter verschiedenen Bedingungen
1 0	Untersuchung der Komplexität von Quantum-Algorithmen
1 1	Graph-Färbung und ihre Anwendung in Zeitplanung und Ressourcenmanagement
1 2	Effizienz von Hashing in Datenbanken
1 3	Implementierung eines Regulärer-Ausdrücke-Generators zur Validierung von Datenformaten

0.7 Themenschwerpunkte und Kapitelübersicht

Themenschwerpunkte der Vorlesung:

- Entwurf von Algorithmen und Datenstrukturen
- Analyse der Platz- und Zeitkomplexität
- Algorithmen für häufig auftretende Probleme

Kap.	Thema
01	Geschichte der Informatik
02	Grundlagen (Basiswissen)
03	Laufzeitkomplexität
04	Sortialgorithmen
05	Suchalgorithmen
06	Hashing
07	Graphen
08	Sprachen

TEIL A — KAPITEL 01: GESCHICHTE DER INFORMATIK (30.000 v. Chr. – 2025)

Der umfangreichste Foliensatz des Kurses (618 Seiten). Hier die **prüfungs- und projektrelevanten Meilensteine** komprimiert.

A.1 ★ Antike und Vorgeschichte

Jahr	Ereignis	Bedeutung
30.000 v. Chr.	Primitive Zahlzeichen (Striche, Kerben, Knoten) zum Abzählen	Erste Datenerfassung
3.500–3.100 v. Chr.	Erste vollständige Zahlensysteme ; Sumerer dokumentieren Warenbestände auf Tontafeln ; Schriftsysteme in Ägypten, China, indianischen Kulturen	⚠ Noch kein abstrakter Zahlbegriff
2700 – 2200 v. Chr.	ABAKUS — vermutlich in Mesopotamien erfunden, Weiterentwicklungen in China, Griechenland, Rom	Frühes mechanisches Rechenhilfsmittel ; Grundlage späterer Recheninstrumente
1184 v. Chr.	Sieg der Griechen über Troja → Feuerzeichenkette von Troja bis Argos (ca. 555 km) ; Lichtsignale über Signaltürme im antiken China	Erste Fernübertragung von Information
70–60 v. Chr.	Mechanismus von Antikythera (gefunden 1900 im Schiffswrack)	★ Das älteste bekannte Beispiel eines ANALOGEN COMPUTERS. Simulierte astronomische Zyklen (Sonne, Mond, Merkur, Venus, Mars, Jupiter, Saturn), kosmischer Kalender, Finsternisberechnung. Feinmechanik-Niveau, das erst im 14./15. Jh. in Europa wieder erreicht wurde. Tony Freeth (UCL): „Wie ein Düsenjet in Tutanchamuns Grab“
200	Die Null in Indien — Konzept der 0 als eigenständige Ziffer; Bakhshali-Manuskript	Fundament des Stellenwertsystems (203 vs. 23) und binärer/moderner Zahlensysteme
500–800	Dezimalsystem in Indien vollendet — 10 Ziffern mit Stellenwert; Aryabhata, Brahmagupta	Grundlage heutiger Zahlschreibweise
900	★ Ibn Musa AL-CHWARIZMI (Haus der Weisheit, Bagdad) — Lehrbuch „ <i>Kitab al-Dschabr wa'l-Muqabala</i> “ (Regeln der Wiedervereinigung und Reduktion)	★ Grundlage der ALGEBRA. Sein Name ist der Ursprung des Begriffs „ALGORITHMUS“. Vermittelte das indisch-arabische Zahlensystem inkl. der Null; im 12. Jh. ins Lateinische übersetzt

A.2 ★ Mechanische Rechenmaschinen (1524–1832)

Ja hr	Person	Leistung	⚠ Einschränkung
15 24	Adam Riese	„Vater des modernen Rechnens“; Rechenbücher in deutscher Sprache statt Latein ; Popularisierung der indisch-arabischen Zahlzeichen	—
16 23	Wilhelm Schickard	★ Erste mechanische Rechenmaschine der Welt — alle 4 Grundrechenarten, Zahnräder + Napiersche Stäbe	Nie funktionstüchtig vollendet ; Teile bei Brand zerstört; erst in den 1950ern wiederentdeckt
16 42	Blaise Pascal (mit 19 Jahren!)	„ Pascaline “ — automatische Addition , spätere Version auch Subtraktion; 6-stellige Zahlen	★ Erste TATSÄCHLICH GEBaute funktionstüchtige Rechenmaschine (ca. 50 Exemplare) ; Zahnradpräzision begrenzte den Nutzen
16 73	Gottfried Wilhelm Leibniz	★ Erste Maschine, die ALLE VIER Grundrechenarten beherrschte ; Staffelwalzenprinzip · 1700: Publikation des BINÄREN Zahlensystems	Mechanisch sehr komplex, nur mit Uhrmachern realisierbar, nicht durchgehend zuverlässig
17 70	Philipp Matthäus Hahn	★ Erste FUNKTIONSFÄHIGE Vier-Spezies-Maschine (Staffelwalzenprinzip, Stellstäbchen)	Erste zuverlässig und praktisch einsetzbare mechanische Rechenmaschine
18 20	Charles Xavier Thomas de Colmar	Arithmometer — erste kommerziell erfolgreiche Rechenmaschine , „Massenproduktion“ von ca. 1500 Stück	Basierte auf Leibniz' Staffelwalze; bis ins 20. Jh. im Büroalltag

A.3 ★★ Die Lochkarte — vom Webstuhl zum Computer

Ja hr	Person	Leistung
17 25	Basile Bouchon	Halbautomatischer Webstuhl mit gelochten Papierstreifen ★ Gilt als ERSTE ANWENDUNG DES DIGITALEN PRINZIPS zur Maschinensteuerung
17 28	Jean-Baptiste Falcon (Assistent Bouchons)	Gelochte BRETTCHENSTREIFEN , miteinander verbunden → Frühform der Lochkartentechnik ; neuer Beruf: Kartenschläger
18 01	Joseph-Marie Jacquard	★ Seriell verbundene LOCHKARTEN (Pappschablonen) — austauschbar und wiederverwendbar ; automatische Herstellung komplexer Stoffmuster. Früher Vorläufer der Programmierung , beeinflusste Babbage
18 32	Semjon Korsakow (Polizeiministerium St. Petersburg)	★ Erster dokumentierter Vorschlag, Lochkarten zur INFORMATIONSVERARBEITUNG zu nutzen — Lochkarten als Datenstruktur für Abgleich/Sortierung großer Datensätze. ⚠ Die Akademie erkannte den praktischen Nutzen nicht an , Idee geriet in Vergessenheit

★ Bouchon (1725) vs. Falcon (1728) — die Vergleichstabelle

Merkmal	Bouchon (1725)	Falcon (1728)
Speichermedium	Einzelner, gelochter Papierstreifen	Reihe gelochter Brettchen
Struktur des Programms	Einteilige Steuerung	Mehrere Brettchen linear verbunden → Musterfolge
Wiederverwendbarkeit	Eingeschränkt	Modular erweiterbar, wiederverwendbar
Stabilität	Papier → recht empfindlich	Holz → robuster und langlebiger
Umfang der Mustersteuerung	Eher begrenzt	Komplexere und längere Muster
Bedeutung	Grundlage des digitalen Prinzips	Frühform der Lochkarte

A.4 ★★ Babbage und Lovelace (1832)

Charles Babbage	Englischer Mathematiker, Philosoph, Ingenieur — einer der Begründer der modernen Informatik und Pionier der Computertechnik. Ziel: Vermeidung menschlicher Rechenfehler in Tabellenwerken (z. B. Logarithmentafeln)
Difference Engine (ab 1822)	Maschine zur Berechnung von Polynomtafeln nach dem Differenzenverfahren
Analytical Engine (ab 1837)	★ Die erste programmierbare, universelle Rechenmaschine der Konzeption nach — mit Lochkartensteuerung
Augusta Ada Byron King (Ada Lovelace)	Menabrea-Abhandlung (1842/43) — ihre Anmerkungen enthalten das, was heute als erstes Computerprogramm gilt

A.5 ★ Nachrichtentechnik und Logik (1780–1906)

Jahr	Person/Sache	Kern
1794	Claude Chappe — optische Telegrafen	Erste Linie Paris–Lille, 270 km, 22 Stationen; 1 Buchstabe $\approx$ 2 Minuten ; nur bei gutem Wetter; 196 Kombinationen (7 Stellungen/Arm $\times$ 7 $\times$ 2 $\times$ 2); über 500 Stationen in Frankreich
1835	Joseph Henry	Elektromagnetisches Relais
1837/ 1844	Samuel Morse — Morsetelegrafie	1844: erste Morse-Telegrafenverbindung
1854	★ George Boole	Boolesche Algebra — mathematische Grundlage aller digitalen Logik
1861	Johann Philipp Reis	Telefon (Vorläufer)
1876	Alexander Graham Bell	Telefon-Patent
1886	Heinrich Rudolf Hertz	Nachweis elektromagnetischer Wellen
1887	★ Herman Hollerith	Lochkarten-Tabelliermaschine für die US-Volkszählung; 1895/1928 Weiterentwicklungen; 1928: Hollerith Typ III Tabulator (IBM 301) → Ursprung von IBM
1897	Guglielmo Marconi	Funktelegrafie
1905	John Ambrose Fleming	Vakuumdiode
1906	Lee De Forest	Triode (De-Forest-Ventil) — Verstärkerröhre, Grundlage der Elektronik

A.6 ★★★★★ Die Gründerzeit der Informatik (1936–1952)

Ja hr	Person/Sache	★ Bedeutung
19 36	★★★ Alan Turing	Turing-Maschine — abstrakte Maschine mit allen Fähigkeiten heutiger Computer; Grenze zwischen Berechenbar und Nicht-Berechenbar (→ Halteproblem)
19 38	★ Konrad Zuse	Z1
19 40	Turing Bombe	Entschlüsselung der Enigma
19 41	★★★ Z3 (Zuse)	Erster funktionsfähiger, programmgesteuerter, frei programmierbarer Rechner in binärer Gleitkommarechnung
19 41	Hedy Lamarr	Frequenzsprungverfahren (Grundlage für WLAN/Bluetooth)
19 43	Colossus	Erster elektronischer Röhren-Spezialrechner (Bletchley Park)
19 43	💬 Thomas J. Watson (CEO IBM)	Das berühmte (zugeschriebene) Zitat: „ <i>I think there is a world market for maybe five computers.</i> “
19 44	Harvard Mark I (Howard H. Aiken)	Elektromechanischer Großrechner
19 45	Z4 (Zuse)	
19 45	★★★ John von Neumann	Von-Neumann-Architektur — Programm und Daten im selben Speicher
19 46	★ ENIAC	Erster elektronischer Universalrechner (ca. 5.000 FLOPS)
19 46	★ ENIAC Girls	Die sechs Programmiererinnen des ENIAC
19 47	★★★ Erster Transistor (Bell Labs)	Ablösung der Röhre
19 47	★ Grace Hopper — der „Bug“	Am Mark II wurde eine echte Motte im Relais gefunden → der Ursprung des Begriffs „ Bug “/„ Debugging “
19 47	Williams-Kilburn-Röhre	Erster elektronischer Speicher
19 48	★ Claude E. Shannon und John W. Tukey	Informationstheorie ; Tukey prägte den Begriff „ BIT “ (binary digit)
19 50	Trommelspeicher, SEAC	
19 51	★ UNIVAC (Universal Automatic Computer), Magnetband , Uniservo	Erster kommerzieller US-Computer
19 52	★ Grace Hopper	Erster Compiler (A-0)
19 52	Alan Turing	(Verurteilung / tragisches Lebensende)

A.7 ★ IBM-Ära, Chips und Theorie (1953–1971)

Jahr	Meilenstein
1953	IBM 701 („Defense Calculator“) — erster kommerzieller IBM-Wissenschaftsrechner
1954	IBM 650 · IBM 704: ca. 12.000 FLOPS (früher kommerzieller Gleitkommarechner)
1956	★ IBM 350 — erste Festplatte (RAMAC)
1956	★★ Noam Chomsky — Chomsky-Hierarchie der formalen Grammatiken (→ Kapitel 08!)
1958	★ Jack Kilby — erster integrierter Schaltkreis (IC)
1959	★ Robert Noyce — planarer IC (Fairchild → später Intel)
1963	★ ASCII (A Standard Code for Information Interchange) · IBM 1311
1966	📷 5 MB Computerdaten — das berühmte Foto der per Gabelstapler verladenen Festplatte
1968	★ Edsger W. Dijkstra (→ Kürzeste-Wege-Algorithmus, Kapitel 07) · ★ Douglas Engelbart — „Mother of All Demos“, die Computermouse · UNIX-Betriebssystem
1969	★★ ARPANET — Vorläufer des Internets
1969	★ Apollo 11 — Margaret Hamilton (Flugsoftware), Katherine Johnson , Dorothy Vaughan
1971	★★ Stephen A. Cook — NP-Vollständigkeit (→ Kapitel 07, TSP!)
1971	★★ Intel 4004 — erster Serien-Mikroprozessor
1971	★ Ray Tomlinson — die erste E-Mail (und das @-Zeichen) · 8" Floppy Disk

A.8 Netze, PCs und das Internet (1972–1998)

Jahr	Meilenstein
1972	FTP · Magnetbandkassette · Magnavox Odyssey (erste Spielkonsole)
1973	★ TCP · Martin Cooper — erstes Mobiltelefonat
1974	Altair 8800
1975	Atari Pong (Heimversion; ursprünglich 1972 als Arcade-Automat)
1976	★ Apple I — erster Personalcomputer · Cray-1 · 5,25" Floppy · ★ Internet Protocol (IP)
1977	💬 Ken Olson (Digital Equipment Company): „There is no reason for any individual to have a computer in his home.“ · Commodore PET 2001 , Apple II , Atari 2600 , UDP
1979	CD-ROM
1980	QDOS (Quick & Dirty Operating System) → später MS-DOS · IBM 3380
1981	Osborne 1 · Spezifizierung verschiedener Netzwerkprotokolle
1982	3½" Floppy · Commodore C64 · GRiD Compass 1100 · ★ IBM 5150 — „der erste Personal Computer“
1983	★ Apple Lisa — 1. Computer mit Maus und GUI · Commodore SX 64 (tragbar) · DynaTAC 8000x · Videospielcrash
1984	★ Erste transatlantische E-Mail · erster „Virus“ · MAC OS 1 · ★ IPv4 · ★ DNS (Domain Name System)
1985	Windows 1.0 · NES · Sega Master System · Usenet
1987	Amiga 500 · Toshiba T1000 · Minix
1989	★★ Tim Berners-Lee — Vorschlag des World Wide Web · Game Boy · Tetris · Windows 2.0
1990	★ Erster Web-Browser (worldwideweb.cern.ch) · ★ Linus Torvalds (→ Linux) · GSM (2G) · Windows 3.0 · NeXTcube · SNES
1993	★ World Wide Web wird freigegeben · Browser Mosaic
1994	Netscape · Netmarket (erster Online-Kauf) · ★ SMS · IBM Simon (erstes Smartphone) · Sony PlayStation · Zip Drive · Suchmaschinen Lycos und Yahoo
1995	Internet Explorer · ★ Amazon · Windows 95 · ★ GPS
1996/ 97	★ Schach: 1996 gewinnt Garri Kasparow 4:2 gegen Deep Blue (100–200 Mio. Stellungen/s) — 1997 gewinnt Deep Blue 2 mit 5:2 (über 200 Mio. Stellungen/s)
1998	★★ Google · Sega Dreamcast

A.9 Das 21. Jahrhundert (1999–2025)

Jahr	Meilenstein
1997–2000	Erste Hochphase des Internets
2001	⚠️ Dotcom-Blase platzt, viele Internetfirmen scheitern
2002	Blu-ray Disc · MMS · Nokia 7650, BlackBerry 5810
2003	Cabir (erster Handy-Wurm) · Call of Duty · facesmash
2004	★ Facebook · Firefox
2005	★ YouTube · Xbox 360 · Motorola RAZR2
2006	PlayStation 3 · Samsung Flash SSD
2007	★★ iPhone (1. Generation) · NFC · Windows Vista
2008	★ Android als offenes Betriebssystem (Open Handset Alliance 2007) · Google Chrome · ★ App Store und Google Play Store · HTC Dream
2009	Windows 7 · ★ WhatsApp
2010	★ LTE (4G) · ★ iPad (1. Generation) · Instagram
2011	Snapchat · iCloud · iPhone 4S · BlackBerry Bold 9900
2012	Samsung Galaxy S3; Samsung ist weltgrößter Mobilfunkhersteller · Tinder · Wii U
2013	PlayStation 4 · Xbox One · Windows 8
2015	Windows 10
2017	Nintendo Switch
2019	★ 5G · Samsung Galaxy Fold
2020	PlayStation 5 · Xbox Series X · Galaxy S20 / iPhone 12
2023	Windows 11 · Linux 6.4.3
2025	MacBook Pro 16 · ★★★ El Capitan (LLNL) und ★ Jupiter (Jülich)

★★ Die beiden Exascale-Supercomputer 2025

	El Capitan	Jupiter
Standort	Lawrence Livermore National Laboratory (USA)	Forschungszentrum Jülich, Deutschland
Status	Schnellster Computer der Welt	Erster europäischer Supercomputer mit Exascale-Leistung
Leistung	1,742 EFLOPS = 1.742.000.000.000.000.000 (1,742 Quintillionen)	1,000 EFLOPS = 1.000.000.000.000.000.000
Kerne	8.730.112	300.000 GPUs + Zehntausende CPU-Kerne
Speicher	4.800.000 GB (4,8 PB)	2.000.000–4.800.000 GB (2–4 PB)

★★★★ DIE FLOPS-VERGLEICHSTABELLE (großartige Projekt-/Präsentationsfolie!)

Epoch e	System	FLOPS	Besonderheiten
ca. 1832	Charles Babbage Analytical Engine	ca. 0,3 FLOPS (Simulationswert)	Rein mechanisch, Konzept einer programmierbaren Maschine
1946	ENIAC	ca. 5.000 FLOPS	Frühester elektronischer Universalrechner
1954	IBM 704	ca. 12.000 FLOPS	Früher kommerzieller Gleitkommarechner
1990	Intel 80486	ca. 1 MFLOPS	Frühe CPU mit integrierter FPU
2020	Intel Core i7	ca. 100 GFLOPS	Moderne Desktop-CPU, Turbo-Boost, mehrere Kerne
2022	Frontier Supercomputer	> 1 EFLOPS	Ex-schnellster Supercomputer der Welt
2025	El Capitan	1,742 EFLOPS	Weltweit leistungsstärkster Supercomputer

🔑 Von 0,3 FLOPS (1832) auf $1,742 \cdot 10^{18}$ FLOPS (2025) — Faktor $\sim 6 \cdot 10^{18}$ in 193 Jahren.

▼ ? Selbsttest Teil A

1. Woher stammt der Begriff „Algorithmus“?
2. Was ist der Mechanismus von Antikythera und warum ist er bedeutend?
3. Wer baute die erste (a) entworfene, (b) tatsächlich gebaute, (c) alle 4 Grundrechenarten beherrschende, (d) zuverlässig einsetzbare mechanische Rechenmaschine?
4. Was ist das Staffelwalzenprinzip und von wem stammt es?
5. Vergleichen Sie Bouchon (1725) und Falcon (1728) anhand von 4 Merkmalen.
6. Was war Korsakows Idee 1832 und warum setzte sie sich nicht durch?
7. Welche zwei Maschinen entwarf Babbage und mit welchem Ziel?
8. Was hat Grace Hopper 1947 und 1952 geleistet?
9. Wer prägte den Begriff „Bit“ und wann?
10. Wer formulierte 1956 die Chomsky-Hierarchie, wer 1971 die NP-Vollständigkeit?
11. Nennen Sie 5 Meilensteine aus dem Jahr 1984.
12. Wie hoch war die Rechenleistung von ENIAC, Intel 80486 und El Capitan?

TEIL B — KAPITEL 02: GRUNDLAGEN

Lernziele: Was sind Algorithmen? Woraus bestehen sie? Welche Eigenschaften haben sie? Was sind Datenobjekte und Datentypen? Unterscheidung elementarer, strukturierter und abstrakter Datentypen. Feld, Verbund, verkettete Liste, Baum, Binärbaum, binärer Suchbaum.

B.1 ★ Was ist Informatik?

Informatik ist ...

- **Information + Mathematik**
- die **Wissenschaft von der automatisierten Verarbeitung von (Daten und) Informationen**
- ... **mehr als nur Programmieren**

$$\text{Information} = \text{Daten} + \text{Kontext}$$

B.2 ★★★★★ Definition: Algorithmus (auswendig!)

Ein Algorithmus ist eine vollständige, präzise und in einer Notation oder Sprache mit exakter Definition abgefasste, **ENDLICHE** Beschreibung eines **SCHRITTWEISEN PROBLEMLÖSUNGSVERFAHRENS** zur Ermittlung gesuchter Datenobjekte (ihrer Werte) aus gegebenen Werten von Datenobjekten, in dem jeder Schritt aus einer Anzahl ausführbarer, **EINDEUTIGER** Aktionen und einer Angabe über den nächsten Schritt besteht.

Ein Algorithmus heißt **TERMINIEREND**, wenn er die Werte der gesuchten Datenobjekte in **ENDLICH VIELEN SCHRITTEN** liefert, andernfalls heißt der Algorithmus **NICHT TERMINIEREND**.

🔑 **Merke:** Ein Algorithmus kann als ein **schrittweises Problemlösungsverfahren** aufgefasst werden. **Die Beschreibung muss dabei nicht konkret formuliert sein.**

★ Algorithmus vs. Programm

	Definition
Algorithmus	Allgemeine Folge von Anweisungen zur Lösung eines Problems
Programm	Umsetzung eines Algorithmus in eine PROGRAMMIERSPRACHE

B.3 ★★★★★ Die 7 EIGENSCHAFTEN eines Algorithmus (auswendig!)

#	Eigenschaft	Bedeutung
1	Korrektheit	Der Algorithmus soll nur das Ergebnis korrekt liefern, welches in der Aufgabenstellung spezifiziert wurde . Nachweis über Schleifeninvariante (Bedingung, die vor, während (nach jeder Iteration) und nach Ausführung einer Schleife gültig ist) und Angabe von Vor- und Nachbedingungen für jede Anweisung. Beweistechniken analog zur Mathematik: vollständige Induktion, Beweis durch Widerspruch
2	Vollständigkeit	Vollständige Beschreibung ; notwendige Vor- und Rahmenbedingungen ; die festgelegten Datenobjekte müssen verfügbar sein und in korrekter Form vorliegen
3	Eindeutigkeit	Jede Aktion muss eindeutig interpretierbar sein; es darf keinen Interpretationsspielraum geben (Prozessor!); die Bedeutung einzelner Elemente und die Abfolge müssen eindeutig definiert sein
4	Ausführbarkeit	Jede Aktion muss tatsächlich ausführbar sein
5	Statische Endlichkeit	Der UMFANG DER BESCHREIBUNG muss endlich sein — unabhängig von Notation oder Sprache. (Endliche Speicherkapazität)
6	Dynamische Endlichkeit	Kommt der Algorithmus nach endlich vielen Schritten zum Halt? Ein Algorithmus, der nach endlich vielen Schritten ein Ergebnis liefert, heißt abbrechend/terminierend ; sonst nicht abbrechend/nicht terminierend (z. B. Ampelanlage). ⚠️ Nicht mit dem Halteproblem verwechseln!
7	Effizienz	Erfüllt seinen Zweck unter bestmöglicher Ausnutzung aller benötigten Ressourcen — Ausführungszeit (Laufzeitverhalten) und Speicherplatz → Kapitel 03

🔑 **Der wichtigste Unterschied: STATISCHE Endlichkeit** = wie lang ist der **TEXT** des Algorithmus ·
DYNAMISCHE Endlichkeit = wie lang **LÄUFT** er.

B.4 ★★☆☆★ Turings HALTEPROBLEM (Berechenbarkeit)

Die Fragestellung

Ziel: Ein Programm **P**, das automatisch auf hängende Software prüft. **Programm (P):** Liest die Datei eines Programms (**R**) und eine Eingabedatei (**d**) für das Programm R ein. **Ausgabe:**

- **1:** wenn **R(d)** hält
- **0:** wenn **R(d)** nicht anhält

? **Gibt es ein Programm, mit dessen Hilfe man prüfen kann, ob ein beliebiges, anderes Programm hält oder nicht hält?**

📝 Die drei einführenden Beispielprogramme

Programm 1:
`a = ['Hello', 'World', '42']`
`for (x in a)`
`print(x, length(x))`

→ hält an: JA
 (keine Endlosschleife)

Programm 2:
`x = -2`
`while (x < 0)`
`x = x - 1`

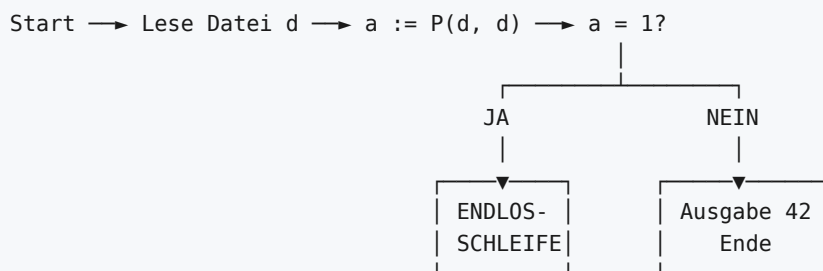
→ hält an: NEIN
 (Endlosschleife!)

Programm 3:
`x = -100`
`if (x < 100)`
`x = x + 1`

→ hält an: JA
 (keine Endlosschleife)

★★★ Der Widerspruchsbeweis (Diagonalisierung)

Konstruiere ein Programm **Q**, das **P** benutzt:



Nun ruft man **Q(Q)** auf:

Annahme	Was P sagt	Was Q tut	Widerspruch
Q(Q) hält	P liefert <code>a=1</code>	Q geht in die Endlosschleife → Q(Q) hält NICHT	↯
Q(Q) hält nicht	P liefert <code>a=0</code>	Q gibt 42 aus und endet → Q(Q) HÄLT	↯

🔑 **In beiden Fällen ein Widerspruch** ⇒ Ein solches Programm **P** kann es **NICHT** geben. Das Halteproblem ist **UNENTSCHEIDBAR**.

B.5 ★ Datenobjekte und Datentypen

Zwei Arten von Datenobjekten

Art	Eigenschaft
Variable (<i>variable</i>)	Kann sich während der Ausführung eines Algorithmus verändern
Konstante (<i>constant</i>)	Verändert sich nicht

★★ Die drei Arten von Datentypen

Art	Definition	Beispiele
① Elementare Datentypen	Können NICHT weiter in einzelne Bestandteile zerlegt werden. Auch atomar, einfach, unstrukturiert genannt	BOOLEAN (1 Bit), BYTE (1 Byte), CHAR , INTEGER (2 Byte), REAL/Float (4 Byte), DOUBLE/LONG/WORD (8 Byte)
② Strukturierte Datentypen	Weisen eine Struktur auf, sind aus kleineren Bestandteilen zusammengesetzt	String (Array of CHAR), Array , Struct , Union
③ Referenzen (Pointer)	Verweise auf andere Daten (Zeiger, Adressen) — Zeiger auf eine Speicherstelle . Kann nur die Adresse eines Datenobjekts oder den speziellen Wert NULL zugewiesen bekommen (NULL = ungültige Adresse , Zeiger ist mit keinem Datenobjekt verbunden)	

Ein Datentyp definiert eine MENGE VON WERTEN und eine MENGE VON OPERATIONEN.

B.6 ★★ Zahlendarstellung im Rechner

Die Einheit Bit

- **Kleinste darstellbare Information** — zwei Zustände: **0 und 1**
- **Bit = binary digit**
- **Anzahl möglicher Zustände bei k Bits:** $n = 2^k$

Bitlänge k	Wertebereich
1	0 ... 1
2	0 ... 3
3	0 ... 7
4	0 ... 15
8	0 ... 255
10	0 ... 1023
16	0 ... 65535
32	0 ... 4.294.967.295
64	0 ... 18.446.744.073.709.551.615

★ Bitfolgen-Namen

Länge	Name
2 Bits	Crumb (engl. „Brösel“)
4 Bits	Nibble bzw. Halbbyte
8 Bits	Oktett bzw. Byte (<i>binary term</i>)
Prozessor-Wortbreite	Word (16-Bit, 32-Bit, 64-Bit) — eindeutige Begriffe für größere Bitfolgen existieren nicht

💡 **Idee:** Die Kombination mehrerer Bits zu einer Bitfolge erlaubt die Speicherung beliebiger Daten, sofern die Bedeutung der Bits festgelegt wird. Diese Festlegung wird als **KODIERUNG** oder **CODE** bezeichnet.

⚠ Größeneinheiten (binär!)

Einheit	Umrechnung
1 KByte	$2^{10} = 1024 \text{ Byte}$
1 MByte	$2^{20} = 1024 \text{ KByte}$
1 GByte	$2^{30} = 1024 \text{ MByte}$
1 TByte	$2^{40} = 1024 \text{ GByte}$

⚠ Die Größeneinheiten im binären System beziehen sich auf die **Basis zwei**. Ein KByte entspricht daher **NICHT 1000 Bytes**.

★ MSB/LSB und Endianness

- **MSB** = *most significant bit* — **höchstwertiges Bit**
- **LSB** = *least significant bit* — **niederwertigstes Bit**

	Big-Endian	Little-Endian
Prinzip	Speicherung des HÖHERwertigen Bytes an erster Stelle	Speicherung des NIEDERwertigen Bytes an erster Stelle

📝 **Beispiel:** Hex 1A 2B 4D 3C (Dez 26, 43, 77, 60 / Binär 00011010 00101011 01001101 00111100) an den Adressen 1000–1003.

★★ Die Stellenwertformel (allgemein)

$$z = \sum_{v=0}^{n-1} a_v \cdot B^v = a_0 + a_1 B + a_2 B^2 + \dots + a_{n-1} B^{n-1}$$

Symbol	Bedeutung
B	Basis
a	Ziffer
z	Zahl
n	Stellen

📝 **Dezimal** ($B = 10$): $247_{10} = 2 \cdot 10^2 + 4 \cdot 10 + 7$ 📝 **Dual** ($B = 2$): $247_{10} = 11110111_2 = 1 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 = 128 + 64 + 32 + 16 + 4 + 2 + 1$

★ Umwandlung Dezimal → Dual: die zwei Methoden

Methode	Prinzip
① Subtraktionsmethode	Verwendet B^n — größte Potenz abziehen, Rest weiterverarbeiten
② Divisionsmethode	Teilt durch B — Reste von unten nach oben lesen

🔑 Prinzipiell lassen sich **beide Methoden zur Umwandlung in jedes andere Zahlensystem mit einer Basis B weiterführen.**

Weitere Zahlensysteme

Hexadezimal (0–9, A–F = 10–15) · Oktal · BCD-Code · Gray-Code

B.7 ★★ Darstellung NEGATIVER ganzer Zahlen

Verfahren	Prinzip	✅ Vorteile	❌ Nachteile
Vorzeichen-Betrag-Methode	MSB wird als Vorzeichen-Bit eingesetzt	Vorteilhaft für Multiplikation und Division	⚠️ Null wird DOPPELT abgebildet; erhöhter Rechenaufwand bei Addition und Subtraktion; Reduzierung des nutzbaren Wertebereichs
Einserkomplement	Bitweise Negierung aller Bits		⚠️ Vorteile sehr gering, daher äußerst selten zur Kodierung von Integerzahlen verwendet — bildet aber die Vorstufe für die tatsächlich verwendete Darstellung
★ Zweierkomplement	Drehen der negativen Seite des Zahlenkreises um eins — Bildung des Einserkomplements + 1; -1 schließt direkt an die Null an	Wertebereich stetig um die Null; negativer Zahlenbereich erhöht sich um eins; ★ Einfache Addition und Subtraktion — bitweise und unabhängig vom Vorzeichen!	Mehr Rechenaufwand bei Multiplikation und Division

🔑 Das Zweierkomplement ist die in der Rechner-technik verwendete Integerdarstellung.

B.8 ★ Rationale Zahlen und Nachkommastellen

Zulassen negativer Exponenten für k :

$$\sum_{k=1}^{\infty} 2^{-k} = 1 \quad \text{und} \quad \lim_{n \rightarrow -\infty} 2^k = 0$$

Potenz	Wert
2^{-1}	$1/2 = 0,5$
2^{-2}	0,25
2^{-3}	0,125
2^{-4}	$1/16 = 0,0625$
2^{-8}	0,00390625 = $1/256 \rightarrow$ „ungefähr auf zwei Nachkommastellen genau rechnen!“

Umwandlung einer Nachkommazahl: $0,6875 = ?$

$0,6875 \cdot 2 = 1,375$	$\geq 1 \rightarrow 1$		Rest $0,375$
$0,375 \cdot 2 = 0,75$	$< 1 \rightarrow 0$		Rest $0,75$
$0,75 \cdot 2 = 1,5$	$\geq 1 \rightarrow 1$		Rest $0,5$
$0,5 \cdot 2 = 1,0$	$\geq 1 \rightarrow 1$		Rest 0

⇒ Ergebnis: $0,1011$

Merke: Nachkommateil wiederholt mit 2 multiplizieren, den Vorkommaanteil (0 oder 1) notieren und abziehen — VON OBEN NACH UNTEN lesen.

⚠ Die **Auflösung** gebrochen rationaler Zahlen ist durch die **Bitlänge begrenzt**.

B.9 ★★★★★ GLEITKOMMAZAHLEN (IEEE 754)

Grundidee

Erweiterung eines Wertebereichs durch Einführung eines EXPONENTEN. Gleitkommazahl = Fließkommazahl = *floating point number* — die Position des Kommas kann bei der Speicherung „gleiten“.

$$1237000000 = 1,237 \cdot 10^9 = \underbrace{0,123700}_{\text{Mantisse}} \cdot 10^{\overbrace{10}^{\text{Exponent}}}$$

Mantisse + Exponent = Gleitkommazahl

Binäres Beispiel:

$$11,625_{10} = 1011,101_2 \cdot 2^0 \quad \Rightarrow \quad 11,625_{10} = \underbrace{1,011101_2}_{\text{Mantisse}} \cdot 2^{\overbrace{3}^{\text{Exponent}}}$$

★★★ Das HIDDEN BIT

Jede Binärzahl, mit Ausnahme der Null, lässt sich mit einer **NORMALISIERTEN** Mantisse darstellen (Verschiebung des Kommas nach rechts oder links). Bei dieser Normierung ist die erste Stelle **VOR dem Komma IMMER EINE EINS**. Da diese konstant ist, muss sie bei der Speicherung **NICHT** berücksichtigt werden. ★ Diese zusätzliche Information wird als **HIDDEN BIT** bezeichnet und erhöht gleichzeitig die Stellengenauigkeit um **EIN BIT**.

★★★ Die IEEE-754-Formate

	float (single)	double
Gesamt	32 Bit	64 Bit
Vorzeichen (V)	1 Bit (Bit 31) — 1 = negativ	1 Bit (Bit 63)
Exponent (E)	8 Bit	11 Bit
Mantisse (M)	23 Bit (Folie nennt „32“; gespeichert werden 23 Bit + 1 Hidden Bit)	52 Bit
Bias (Offset)	127	1023

Besonderheiten:

- Der Exponent wird **NICHT im Zweierkomplement oder in Vorzeichen-Betrag-Darstellung** gespeichert → **Speicherung mit einem OFFSET (Bias)**; der Offset gibt die Null an
- Der **Wert Null lässt sich nicht (normal) speichern** → **Sonderfall durch Speicherung einer Null im Exponenten**
- **Exponent $E = 255$ und Mantisse $M = 0 \rightarrow \pm\infty$**
- **Exponent $E = 255$ und Mantisse $M \neq 0 \rightarrow \text{NaN (Not a Number)}$** — Wert als reelle Zahl nicht definiert

★★★★ DAS VERFAHREN (4 Schritte, auswendig!)

Falls $M = 0$: Fülle alle Felder mit 0 und terminiere.

I. NORMALISIERUNG DER MANTISSE

Solange die Binärzahl $M \geq 2$:

Verschiebe das Komma nach LINKS und INKREMENTIERE den Exponenten

Solange die Binärzahl $M < 1$:

Verschiebe das Komma nach RECHTS und DEKREMENTIERE den Exponenten

II. ENTFERNEN DES HIDDEN BITS UND ANPASSUNG DER MANTISSE

Streiche die Vorkommastelle von M

Solange Anzahl Stellen von $M < 23$: Füge Nullen hinzu

Solange Anzahl Stellen von $M > 23$: Runde die 23. Stelle auf,
streiche die weiteren Stellen

III. ADDIERE DEN BIAS-WERT AUF DEN EXPONENTEN

$E = E + 01111111_2$ (= 127)

IV. EINTRAGEN DER WERTE

Trage die Binärzahlen V , E und M in die entsprechenden Felder ein

📝★★★★ DAS VOLLSTÄNDIG DURCHGERECHNETE BEISPIEL: $-247,09375_{10}$

Ausgangswert: $M = 11110111,00011_2$, $E = 00000000_2$, $V = 1$ (negativ)

Schritt I — Normalisierung (Komma nach links wandern lassen):

M	E
11110111.00011	00000000
1111011.100011	00000001
111101.1100011	00000010
11110.11100011	00000011
1111.011100011	00000100
111.1011100011	00000101
11.11011100011	00000110
1.111011100011	00000111 ← fertig (führende 1)

Schritt II — Hidden Bit entfernen + auf 23 Stellen auffüllen:

$M = 11101110001100000000000$

Schritt III — Bias addieren:

$E = 7 + 127 = 134 = 10000110_2$

Schritt IV — Eintragen: $V = 1$, $E = 10000110$, $M = 11101110001100000000000$

Ergebnis in anderen Systemen:

System	Wert
Dual	1100 0011 0111 0111 0001 1000 0000 0000
Hex	C3771800
Oktal	30335614000

⚠ Die Umwandlung in das **double**-Format kann nicht nach dem gleichen Schema erfolgen — hier sind die **erweiterten Längen** von Mantisse und Exponent sowie der **Bias-Wert 1023** zu beachten.

⚠⚠ GEFAHREN von Gleitkommazahlen

Rundungsfehler:

- Jede Zahl wird auf signifikante Stellen gerundet
- Der Exponent zur Basis 10 muss zur Basis 2 umgerechnet werden
- ★ Durch die begrenzte Anzahl an Bits lässt sich nur eine WINZIGE TEILMENGE aus $\mathbb{R}$ darstellen

Zur darstellbaren Teilmenge aus $\mathbb{R}$ gehören:

1. Ganze Zahlen im Wertebereich der Mantisse
2. Rationale Zahlen, die sich aus Zweierpotenzen mit positiven und negativen Exponenten im Rahmen der Mantissenlänge darstellen lassen
3. Alle Produkte dieser rationalen Zahlen mit Zweierpotenzen, die sich im Rahmen der Exponentengröße darstellen lassen

⚠ Aufpassen bei Zuweisungen von Integerzahlen!

- Die Zuweisung einer **32-Bit-Integerzahl** an ein **float** kann zu Rundungsfehlern führen:
 - 16777217 wird zu 16777216
 - 16777219 wird zu 16777218
- **32-Bit-Integerzahlen** sollten dem **DOUBLE**-Format zugewiesen werden
- Bei **64-Bit-Integerzahlen** reicht auch **double** nicht mehr → **BIGNUM**

⚠⚠ Das Ariane-5-Beispiel:

Eine falsche Umwandlung kann zu **BEREICHSÜBERSCHREITUNGEN** führen — mit fatalen Folgen.
Beispiel Ariane-5-Rakete: Zuweisung einer **double-Gleitkommazahl** auf eine **(16-Bit !!!)** Integerzahl.

B.10 ★ Darstellung von Textzeichen

Kodierung	Eigenschaften
ASCII (<i>American Standard Code for Information Interchange</i>)	7-Bit-Kodierung ; das 8. Bit ursprünglich als PRÜFBIT (Fehlerdetektion bei der Übertragung); die ersten 32 Symbole sind nicht druckbare Steuerzeichen (Tabulator, Zeilenende etc.)
Unicode (ISO/IEC 10646, UCS-2 = Universal Character Set with 2 Bytes)	16-Bit-Kodierung → 65536 Zeichen (chinesisch, koreanisch, japanisch etc.); die ersten 256 Zeichen entsprechen dem ASCII-Code
UTF-8 (<i>Universal Character Set Transformation Format</i>)	Mehrbyte-Kodierung , wenn MSB gesetzt ist ; die Länge der Kodierung ist im ersten Byte festgelegt

★ Die UTF-8-Bitmuster

Länge	Muster	Bereich
1 Byte	0xxxxxxx	ASCII-Code (0–127)
2 Byte	110xxxxx 10xxxxxx	11-Bit UCS-2-Code (0–2047)
3 Byte	1110xxxx 10xxxxxx 10xxxxxx	UCS-2-Code (0–65535)

🔑 Die Daten befinden sich in den verbleibenden Bits des ersten Bytes und den unteren 6 Bits jedes nachfolgenden Bytes.

B.11 ★★★ Speicherung: sequentiell vs. verkettet

	Sequentielle Speicherung	Verkettete Speicherung
Layout	Hauptspeicher: ... 1 2 3 4 5 6 ... (zusammenhängend)	Hauptspeicher: ... 1 5 2 3 4 6 ... (verstreut, über Zeiger verbunden)
✓	Schneller Zugriff	Schnelles Einfügen/Löschen
✗	Langsames Einfügen/Löschen	Langsamer Zugriff, höherer Speicherbedarf

B.12 ★★★ FELDER (Arrays) und VERBUNDE

Definition Feld

Ein Feld (Array) ist ein Datenobjekt, das eine Sammlung einer **FIXEN ANZAHL** von Elementen repräsentiert, die alle **DENSELBE** DATENTYP haben und auf die über einen **INDEX** zugegriffen werden kann.

★★★ Die SPEICHERABBILDUNGSFUNKTION (1-dimensional)

$$\text{AddrOf}(a[i]) = \text{AddrOf}(a) + (i - \text{first}) \cdot \text{elementSize}$$

Symbol	Bedeutung
<code>AddrOf(a)</code>	Startadresse
<code>first</code>	Untergrenze des Indexbereichs
<code>i</code>	Index
<code>elementSize</code>	Speicherbedarf eines Elements

🔑 ★ Effizienter Zugriff auf jedes Element in KONSTANTER ZEIT — unabhängig vom Wert des Index!

★ Zweidimensionale Felder

$$\text{AddrOf}(m[i, j]) = \text{AddrOf}(m) + (i - \text{firstZeile}) \cdot \text{rowSize} + (j - \text{firstSpalte}) \cdot \text{elementSize}$$

$$\text{mit } \text{rowSize} = (\text{lastSpalte} - \text{firstSpalte} + 1) \cdot \text{elementSize}$$

Pseudocode Array-Operationen

```

ARRAY-SEARCH(x, L)                // L: Array 0..MAX, Speicherung ab Position 1
    L[0] = x                       // Wächter (Sentinel)!
    pos = L.size
    while (L[pos] != x)
        pos = pos - 1
    return pos

ARRAY-INSERT(x, p, L)
    if (L.size == MAX) error "Overflow"
    else
        if (p > L.size + 1 or p < 1) error "Invalid position"
        for (pos = L.size downto p)
            L[pos + 1] = L[pos]      // ⚠ alle Elemente verschieben!
        L[p] = x
        L.size = L.size + 1

ARRAY-DELETE(p, L)
    if (L.size == 0) error "Empty List"
    if (p > L.size or p < 1) error "Invalid position"
    L.size = L.size - 1
    for (pos = p to L.size)
        L[pos] = L[pos + 1]         // ⚠ alle Elemente verschieben!

```

★ Definition Verbund

Ein **Verbund (Compound)** ist ein Datenobjekt, das eine Sammlung einer **FIXEN ANZAHL** von Komponenten repräsentiert, deren Datentyp **BELIEBIG** ist und auf die über entsprechende **BEZEICHNER** auf die einzelnen Komponenten und das Datenobjekt als Ganzes zugegriffen werden kann.

⚠ Es existiert **KEINE** Speicherabbildungsfunktion zur effizienten Berechnung der Adresse der i -ten Komponente — **unterschiedliche Datentypen beanspruchen unterschiedlichen Speicherplatz**.

★ Assoziatives Datenfeld (Dictionary)

Datenstruktur, welche die **Schlüssel (Werte) NICHT FORTLAUFEND speichert** — die Abspeicherung erfolgt **in keiner festgelegten Reihenfolge**. Die Speicherung von **key** und **value** ist so implementiert, dass man **sehr effizient den Wert zu einem gegebenen Schlüssel bestimmen** kann (**Wertezuordnung über eine mathematisch definierte Funktion** → Kapitel 6, Hashing).

Funktionen	Mengen-Operationen OHNE:
Einfügen (<code>insert</code>), Löschen (<code>delete</code>), Ist leer? (<code>isempty</code>), Enthält/Ist Mitglied (<code>member</code>)	Durchschnitt, Vereinigung, Differenz

🔑 Im Gegensatz zu Feldern spricht man bei Dictionaries von einem **UNGEORDNETEN CONTAINER**. Aus diesem Grund spricht man i. d. R. auch nicht von einem ersten, zweiten, dritten Element.

B.13 ★★ Dynamische Datenstrukturen: VERKETTETE LISTEN

Datenobjekte sind in der Praxis häufig gekennzeichnet durch die Veränderung ihres WERTES und ihrer STRUKTUR. Solche Datenobjekte können zur Laufzeit **wachsen/schrumpfen (Größe)**, den **Inhalt** und/oder die **Beziehung (Struktur)** zwischen ihren Komponenten ändern.

Definition

Eine verkettete Liste ist entweder **LEER** oder sie repräsentiert eine **SEQUENZIELLE ANORDNUNG VON KNOTEN (nodes) gleichen Datentyps**, deren Reihenfolge **EXPLIZIT** durch **VERKETTUNG (linkage)** festgelegt ist. Die Verkettung wird durch **in den Knoten enthaltene ZEIGER** realisiert:

- bei **einfach-verketteten Listen** auf den **NACHFOLGER (successor)**
- bei **doppelt-verketteten Listen** auch auf den **VORGÄNGER (predecessor)**

★ Attribute

Attribut	Bedeutung
x	Listenelement
x.key	Schlüssel des Elements x
x.next	Zeiger auf das Nachfolger-Element von x
x.prev	Zeiger auf das Vorgänger-Element von x (nur doppelt verkettet)
L.head	Zeiger auf das ERSTE Listenelement
L.rear	Zeiger auf das LETZTE Listenelement
NIL	Nicht benötigter Zeiger

Pseudocode einfach-verkettete Liste

```

LIST-SEARCH(x, L)
  p = L.head
  while (p != NIL)
    <Element = x (gesuchtes Element)>
    p = p.next

// Einfügen von x HINTER dem Element p
  n.key = x
  n.next = p.next
  p.next = n

// Entfernen des Elements HINTER p
  n = p.next
  p.next = p.next.next
  delete(n)

```

!★ Das Problem doppelt-verketteter Listen — und die WÄCHTER-STRATEGIE

! Für das Einfügen und Löschen am LISTENANFANG und -ENDE sind besondere Regeln notwendig. Aus diesem Grund verwendet man die WÄCHTER-STRATEGIE!

Idee:

- **Vergrößerung der Liste um ein zusätzliches Element** → die Liste ist **NIE LEER**, sie **enthält mindestens immer ein Element**
- ★ **Sonderfälle am Listenanfang und -ende treten NICHT AUF**
- D. h. man **fügt nie ein Element ganz am Anfang ein** und **löscht nie das letzte Element**

Attribut	Bedeutung
<code>L.nil</code>	Spezielles Listenelement, welches Listenanfang UND -ende repräsentiert
<code>L.nil.next</code>	Listenanfang (= <code>L.head</code>)
<code>L.nil.prev</code>	Listenende (= <code>L.rear</code>)
<code>L.nil.key</code>	NIL (speichert keinen Wert)

Einfügen MIT Wächter — nur noch 4 Zeilen statt einer langen Fallunterscheidung:

```
x.prev.next = x
x.next.prev = x
```

★★★ DIE GEGENÜBERSTELLUNG Feld ↔ verkettete Liste (auswendig!)

Merkmal	Eindimensionales Feld	Verkettete Liste
Größe und Struktur	FIX	VERÄNDERLICH
Zugriff auf das <i>i</i>-te Element	✓ Effizient , in KONSTANTER Zeit mit Indizierungsoperation	✗ Ineffizient , da z. B. in einer Schleife der <i>i</i> -te Knoten gesucht werden muss
Löschen und Einfügen	✗ Ineffizient , da die restlichen Datenobjekte elementweise verschoben werden müssen	✓ Effizient , da die Struktur (Verkettung) nur LOKAL verändert werden muss

B.14 ★★ STACK und QUEUE

Kellerspeicher / Stapel (Stack) — LIFO

last-in first-out (LIFO) — Speicherung mit **Zugriffsmöglichkeit nur auf das ZULETZT gespeicherte Objekt**. Implementierung: **sequentielle oder verkettete Speicherung**

Operation	Bedeutung
<code>HEAD(S)</code>	Wert des OBERSTEN Elements
<code>PUSH(S, x)</code>	Lege x oben auf den Stapel
<code>POP(S)</code>	Entferne das oberste Element vom Stapel

```

EMPTY-STACK(S)          PUSH(S, x)
  if (S.top == 0) return TRUE    if (S.top == MAX) error "Überlauf"
  else return FALSE             else S.top = S.top + 1
                                S[S.top] = x

POP(S)                   HEAD(S)
  if (EMPTY-STACK(S)) error "Unterlauf"  if (EMPTY-STACK(S)) error "Stapel leer"
  else S.top = S.top - 1                  else return S[S.top]
      return S[S.top + 1]
```

★ **Anwendungen: Mikroprozessoren** (Registerinhalte bei Interrupt- und Funktionsaufrufen) · **Programmiersprachen** (Übergabe von Funktionsargumenten und lokalen Variablen) · **Compiler** (Übersetzung des Quellcodes einer formalen Sprache) · ★ **TIEFENSUCHE (Kapitel 7)**

Warteschlange (Queue) — FIFO

first-in first-out (FIFO) — Speicherung mit **Zugriffsmöglichkeit nur auf das ZUERST gespeicherte Objekt. Zyklischer Speicher (Ringpuffer)**

Operation	Bedeutung
HEAD(Q)	Wert des VORDERSTEN Elements
ENQUEUE(Q, x)	Füge x am ENDE der Schlange an
DEQUEUE(Q)	Entferne das VORDERSTE Element

```

INIT(Q)                                EMPTY-QUEUE(Q)
    Q.head = Q.rear = 0                if (Q.head == Q.rear) return TRUE
                                        else return FALSE

ENQUEUE(Q, x)
    if ((Q.rear + 1) mod MAX) == Q.head error "Überlauf"
    else Q[Q.rear] = x
        Q.rear = (Q.rear + 1) mod MAX    // ★ mod MAX = Ringpuffer!

DEQUEUE(Q, x)
    if (EMPTY-QUEUE(Q)) error "Unterlauf" // Q.head == Q.rear
    else x = Q[Q.head]
        Q.head = (Q.head + 1) mod MAX
        return x

```

★ **Anwendungen: Druckerschlange** · **Tastatureingabe** · **Multitask-Systeme** · ★ **BREITENSUCHE (Kapitel 7)**

B.15 ★ Variablentausch: tmp vs. XOR

SWAP mittels tmp	SWAP mittels XOR
<pre>void swap(int x, int y){ int tmp; tmp = x; x = y; y = tmp; }</pre>	<pre>void swap(int a, int b){ a ^= b; b ^= a; a ^= b; }</pre>

★ Warum funktioniert XOR-Swap? (Die Beweistabelle)

Aktion	Variable 1	Variable 2
Startwerte	A	B
$V_1 = V_1 \oplus V_2$	$A \oplus B$	B
$V_2 = V_1 \oplus V_2$	$A \oplus B$	$(A \oplus B) \oplus B = A \oplus (B \oplus B) = A$
$V_1 = V_1 \oplus V_2$	$(A \oplus B) \oplus A = (A \oplus A) \oplus B = B$	A

🔑 **Genutzt werden: Assoziativität, Kommutativität und $X \oplus X = 0$, $X \oplus 0 = X$.**

B.16 ★★★ BÄUME

🔑 **Bäume sind wie Listen, jedoch mit einer ENDLICHEN ANZAHL AN NACHFOLGERN.**

Definition Baum

Ein (allgemeiner) Baum ist entweder LEER, oder er repräsentiert eine Menge von Knoten gleichen Datentyps, in der es GENAU EINEN AUSGEZEICHNETEN KNOTEN (die Wurzel) und eine ENDLICHE MENGE DISJUNKTER BÄUME (Teilbäume) gibt, die mit der Wurzel verbunden sind. Jeder allgemeine Baum wird auch als MEHRWEGBAUM bezeichnet. → Hierarchisch strukturierte Objekte

★★ Die Fachbegriffe

Begriff	Definition
Wurzel (root)	Knoten OHNE Vorgänger
Blätter (leaves)	Knoten OHNE Nachfolger
Innere Knoten (inner nodes)	Knoten MIT Nachfolger
Ebene (level)	Die Wurzel hat immer die Ebene 0
Höhe (height)	Länge des Weges = Anzahl der Verbindungen von der Wurzel bis zum Blatt auf der höchsten Ebene
Ordnung eines Knotens	Anzahl der DIREKTEN Nachfolger eines Knotens
Pfad ($V_1 \dots V_n$)	Folge von Knoten mit $V_i = \text{parent}(V_{i+1})$

★★ Definition Binärbaum

Ein Binärbaum (binary tree) ist ein spezieller Baum, in dem jeder Knoten MAXIMAL ZWEI NACHFOLGER hat. In der Regel: linker und rechter Nachfolger bzw. linker und rechter Teilbaum.  Jeder Mehrwegbaum lässt sich, unabhängig davon, wie viele Nachfolger seine Knoten haben, in einen ÄQUIVALENTEN BINÄRBAUM umwandeln.

★★★★ Die BINÄRBAUM-FORMELN (auswendig!)

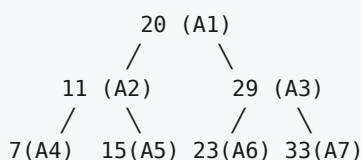
Größe	Formel
Höhe	h
Anzahl der Knoten auf Ebene i	2^i
Anzahl der Blätter (perfekter Baum)	2^h
Anzahl innerer Knoten	$2^h - 1$
Anzahl der Knoten gesamt	$ K = 2^{h+1} - 1$
Höhe bei $ K $ Knoten	$h = \log(K + 1) - 1 = \Theta(\log K)$

★★ Die 4 Arten von Binärbäumen

Art	Definition	Konsequenz
Vollständiger Binärbaum	Jeder Knoten hat ENTWEDER KEINEN ODER GENAU ZWEI Nachfolger	(In der Literatur auch erweiterter Binärbaum genannt)
Perfekter Binärbaum	Ist ein vollständiger Binärbaum, bei dem ALLE BLÄTTER AUF EIN UND DERSELBEN EBENE liegen	maximale Ausgeglichenheit
⚠ Degenerierter (entarteter) Baum	Jeder Knoten hat NUR EINEN linken bzw. rechten Nachfolger	⚠ Der Effizienzvorteil, den ein Binärbaum hat, GEHT VERLOREN! (wird zur Liste)
★ Ausgewogener (balancierter) Baum = AVL-Baum	Die Höhen der Teilbäume eines Knotens unterscheiden sich HÖCHSTENS UM DEN WERT 1 — der Höhenunterschied zwischen kürzestem und längstem Pfad ist maximal 1	Garantierte $O(\log n)$

★★ Implementierung von Bäumen mit ARRAYS (die Index-Formeln!)

Array: A1 A2 A3 A4 A5 A6 A7
20 11 29 7 15 23 33



Linkes Kind von $n = 2n$

Rechtes Kind von $n = 2n + 1$

Elter von $n = \lfloor n/2 \rfloor$

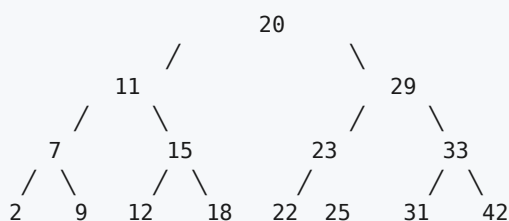
(Weitere Implementierungen: **Binärbäume mit Listen** · Bäume mit unbeschränktem Grad über **left-child/right-sibling**.)

B.17 ★★★ DIE 4 BAUMDURCHLÄUFE (tree walks)

Attribute: `x.key` (Schlüssel), `x.left` (linkes Kind), `x.right` (rechtes Kind), `x.parent` (Eltern-Knoten), `T.root` (Wurzel)

Verfahren	★ Reihenfolge	Pseudocode-Kern
Pre-Order	WURZEL → LINKS → RECHTS	<code>print x.key; PRE(x.left); PRE(x.right)</code>
In-Order	LINKS → WURZEL → RECHTS	<code>IN(x.left); print x.key; IN(x.right)</code>
Post-Order	LINKS → RECHTS → WURZEL	<code>POST(x.left); POST(x.right); print x.key</code>
Level-Order	EBENENWEISE von links nach rechts, beginnend bei Ebene 0	rekursiv über <code>level</code> oder mit einer Queue

✎ ★★★ DER DURCHGERECHNETE BEISPIELBAUM



Durchlauf	Ergebnis
Pre-Order	20 - 11 - 7 - 2 - 9 - 15 - 12 - 18 - 29 - 23 - 22 - 25 - 33 - 31 - 42
In-Order	2 - 7 - 9 - 11 - 12 - 15 - 18 - 20 - 22 - 23 - 25 - 29 - 31 - 33 - 42
Post-Order	2 - 9 - 7 - 12 - 18 - 15 - 11 - 22 - 25 - 23 - 31 - 42 - 33 - 29 - 20
Level-Order	20 - 11 - 29 - 7 - 15 - 23 - 33 - 2 - 9 - 12 - 18 - 22 - 25 - 31 - 42

🔑 **DIE ZENTRALE ERKENNTNIS: Der IN-ORDER-Durchlauf eines binären SUCHbaums liefert die Elemente in AUFSTEIGENDER Reihenfolge — er sortiert!** 🔑 Pre-Order = Tiefensuche · Level-Order = Breitensuche (siehe Kapitel 07!)

```

LEVEL-ORDER-TREE-WALK(x, level)    // Aufruf mit (T.root, 1..h)
  if (x == NIL) return NIL
  if (level == 1) print x.key
  elseif (level > 1)
    LEVEL-ORDER-TREE-WALK(x.left, level - 1)
    LEVEL-ORDER-TREE-WALK(x.right, level - 1)

```

B.18 ★ Mengenlehre (Mathematische Grundlagen)

Definition Menge

Eine Menge ist eine Zusammenfassung von bestimmten, wohl unterschiedenen Objekten zu einem Ganzen. Die zusammengefassten Objekte heißen **ELEMENTE** der Menge.

Begriff	Definition
Teilmenge	$A \subseteq B \Leftrightarrow \forall x (x \in A \rightarrow x \in B)$
Echte Teilmenge	$A \subset B \Leftrightarrow A \neq B \wedge \forall x (x \in A \rightarrow x \in B)$ — es gibt Elemente in B , die nicht zu A gehören
Durchschnitt (Schnittmenge)	$x \in A \cap B \Leftrightarrow x \in A \wedge x \in B$
Vereinigung	$x \in A \cup B \Leftrightarrow x \in A \vee x \in B$
Differenz	$x \in A \setminus B \Leftrightarrow x \in A \wedge x \notin B$
Leere Menge	$\{\} = \emptyset \subseteq M$ — enthält überhaupt kein Element
Disjunkte Mengen	$A \cap B = \emptyset$ — kein gemeinsames Element
★ Potenzmenge	$\mathcal{P}(A) = \{X \mid X \subseteq A\}$ — die Menge ALLER Teilmengen von A

📝 **Beispiel Potenzmenge:** $A = \{1, 2, 3\} \Rightarrow \mathcal{P}(A) = \{\{\}, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$ —
 $|\mathcal{P}(A)| = 2^{|A|} = 8$

★ Tupel vs. Menge

	Tupel $(x_1, \dots, x_n)$	Menge $\{x_1, \dots, x_n\}$
Bezeichnung der Objekte	Komponenten	Elemente
Reihenfolge der Objekte	★ Spielt eine Rolle	★ Spielt KEINE Rolle

Datentypen für Mengen: Bitvektor · einfach verkettete Liste · Dictionaries (Hashing) **Operationen auf Mengen:** Initialisieren, Ist-Leer, Ist-Element, Einfügen, Löschen, Ausgeben, Durchschnitt, Vereinigung, Differenz

B.19 ★★ Logarithmus- und Modulo-Rechenregeln

Logarithmus

$$a^n = z \iff \log_a(z) = n$$

#	Regel
(1)	$\log_a(x) + \log_a(y) = \log_a(x \cdot y)$
(2)	$\log_a(x) - \log_a(y) = \log_a\left(\frac{x}{y}\right)$
(3)	$\log_a(x^y) = y \cdot \log_a(x)$
(4)	$a^{\log_a(x)} = x$
(5)	★ Basiswechsel: $\log_a(x) = \frac{\log_b(x)}{\log_b(a)}$
(6)	$\log_a(1) = 0$ (da $a^0 = 1$)
(7)	$\log_a(a) = 1$ (da $a^1 = a$)
(8)	⚠ $\log_a(0) = \mathbf{n. d.}$ (nicht definiert, da $a^? = 0$)
(9)	$\log_{10}(x) = \lg(x)$
(10)	$\log_e(x) = \ln(x)$
(11)	★ $\log_2(x) = \mathbf{ld}(x)$

Modulo

$$a \equiv b \pmod{q} \Leftrightarrow b = a \bmod q \qquad a \bmod q = a - q \cdot \left\lfloor \frac{a}{q} \right\rfloor$$

#	Regel
(1)	$(a + b) \bmod n = (a \bmod n + b \bmod n) \bmod n$
(2)	$(a - b) \bmod n = (a \bmod n - b \bmod n) \bmod n$
(3)	$(a \cdot b) \bmod n = (a \bmod n \cdot b \bmod n) \bmod n$

▼ ? **Selbsttest Teil B**

1. Geben Sie die Definition eines Algorithmus wieder.
2. Nennen Sie die 7 Eigenschaften eines Algorithmus.
3. Was ist der Unterschied zwischen statischer und dynamischer Endlichkeit?
4. Was ist eine Schleifeninvariante?
5. Beschreiben Sie das Halteproblem und den Widerspruchsbeweis.
6. Nennen Sie die 3 Arten von Datentypen mit je 3 Beispielen.
7. Wie viele Zustände hat man bei 10 Bit? Bei 16?
8. Wie heißen Bitfolgen der Länge 2, 4 und 8?
9. Erklären Sie Big-Endian und Little-Endian.
10. Wandeln Sie 0,8125 ins Dualsystem um. (0,1101)
11. Nennen Sie die 3 Verfahren zur Darstellung negativer Zahlen mit je einem Vor- und Nachteil.
12. Was ist das Hidden Bit und was bringt es?
13. Nennen Sie die 4 Schritte der IEEE-754-Umwandlung.
14. Wie groß sind Vorzeichen/Exponent/Mantisse/Bias bei float und double?
15. Was passierte bei der Ariane 5?
16. Wie viele Bit hat ASCII, wie viele UCS-2? Wie funktioniert UTF-8?
17. Feld vs. verkettete Liste: 3 Kriterien.
18. Wozu dient die Wächter-Strategie?
19. Stack vs. Queue: Prinzip, 3 Operationen, je 2 Anwendungen.
20. Zeigen Sie, dass der XOR-Swap funktioniert.
21. Nennen Sie die Binärbaum-Formeln für Knoten je Ebene, Blätter, Gesamtknoten, Höhe.
22. Was ist ein degenerierter Baum? Was ein AVL-Baum?
23. Wie berechnet man linkes/rechtes Kind bei Array-Implementierung?
24. Geben Sie für den Beispielbaum alle 4 Durchläufe an.
25. Welcher Durchlauf sortiert einen binären Suchbaum?

TEIL C — KAPITEL 03: LAUFZEITKOMPLEXITÄT



Lernziele: Laufzeitverhalten als Qualitätsmerkmal · Komplexität im Sinne der Informatik · Effektivität vs. Effizienz · Methoden zur Bestimmung des groben Laufzeitverhaltens · asymptotische Laufzeitkomplexität · wichtige Komplexitätsklassen.

C.1 ★★ Begriffe

$$\boxed{\text{Komplexität} = \text{Aufwand} = \text{Speicherbedarf} + \text{Rechenzeit}}$$

$$\text{Rechenzeit} = \text{Laufzeit} = \text{running time}$$

→ Man unterscheidet **Laufzeitkomplexität** und **Speicherkomplexität**.

★ Funktional vs. Nicht-funktional

	Frage	Kurz
Funktional	Welche Funktionalität wird durch den Algorithmus gestellt?	→ WAS
Nicht-funktional	Unter welchen Bedingungen erbringt der Algorithmus seine Leistung? Mit welcher Laufzeit?	→ WIE

★★★★ EFFEKTIVITÄT vs. EFFIZIENZ (auswendig!)

	Definition	Leitfrage
Effektivität	= WIRKSAMKEIT	Wie GUT wird die Aufgabenstellung gelöst? — unabhängig davon, wie viel Zeit dafür notwendig ist
Effizienz	= WIRTSCHAFTLICHKEIT	Wie viel SPEICHERPLATZ und LAUFZEIT spielen eine Rolle? — eine Aufgabenstellung soll mit möglichst wenig Aufwand gelöst werden

🔑 MERKE: EFFIZIENZ SETZT EFFEKTIVITÄT VORAUS.

C.2 ★★ Warum die Laufzeitkomplexität ermitteln? (4 Gründe)

1. Um **Vorhersagen über die zu erwartende Laufzeit** machen zu können
2. Um die **Laufzeitverhalten miteinander vergleichen** zu können
3. Um das **Potential von Optimierungen** beurteilen zu können
4. Um die **benötigten Ressourcen** feststellen zu können

Definition

Die Laufzeitkomplexität ist das LAUFZEITVERHALTEN IN ABHÄNGIGKEIT VON EINER BESTIMMTEN GRÖSSE, die als n bezeichnet wird.

$$t = f(n)$$

- $f(n)$ ist die **eigentliche Laufzeit** des Algorithmus in Abhängigkeit von n , z. B. $t = f(n) = n^2 + 2n$
- n ist „quasi“ **das Problem, welches das Wachsen der Funktion verursacht bzw. beeinflusst** — vergleichbar mit $f(x)$ aus der Schulmathematik

★ Was ist n konkret? (Die Problemgröße)

- **Anzahl der zu bearbeitenden Elemente eines Feldes oder einer Matrix** (→ verkettete Liste)
- **Anzahl der Knoten einer Datenstruktur** (→ Binärbaum)
- **Länge einer Zeichenkette**
- **Grad eines Polynoms** (→ Anzahl der zu berechnenden Koeffizienten)

Annahme: Größe in Form einer **natürlichen Zahl**; mehrere Einzelgrößen lassen sich zu einer **Gesamtgröße zusammenfassen**.

★ Was beeinflusst die Laufzeit in der PRAXIS?

Ebene	Faktoren
Hardware	Prozessorart, Taktfrequenz, Cache, Grafikspeicher
Betriebssystem	Multi-Processing
Programmiersprache	—
Compiler	verwendete Bibliotheken, Optimierung bei der Übersetzung

C.3 ★★ Fein- vs. Grobanalyse

	FEINANALYSE	GROBANALYSE
Prinzip	Jedes Detail, das die Laufzeit beeinflusst, wird erfasst — Häufigkeit und Zeitaufwand der Aktionen/Operationen	Identifizierung und Analyse der Teile, die das Laufzeitverhalten SIGNIFIKANT beeinflussen
Aufwand	⚠ Aus Aufwandsgründen nicht praktikabel	✅ Analyse in vertretbarem Aufwand
Einsatz	Bei sehr harten Laufzeitanforderungen — kritische Teile des Systems, Echtzeitsysteme	Der Normalfall — Vorhersage möglich, auch wenn nicht alle Details des Algorithmus ausformuliert sind
Werkzeug	—	Fallunterscheidung

★★★ Die 3 Fälle der Grobanalyse

Fall	Englisch	Definition
① Günstigster Fall	<i>best case</i>	MINIMALE Anzahl von Arbeitsschritten
② Ungünstigster Fall	<i>worst case</i>	MAXIMALE Anzahl von Arbeitsschritten
③ Durchschnittlicher Fall	<i>average case</i>	DURCHSCHNITTliche Anzahl von Arbeitsschritten — über viele Anwendungen betrachtet und gemittelt; basiert auf Annahmen und Wahrscheinlichkeitsrechnung

C.4 ★★ DIE O-NOTATION

Geschichte

- 1892 von Paul BACHMANN eingeführt (Zahlentheoretiker) — Wachstumsgeschwindigkeit von $1^k + 2^k + \dots + n^k = n^{k+1}$
- Edmund LANDAU machte die O-Notation im deutschen Sprachraum bekannt → LANDAU-NOTATION

★★ Die formale Definition

Wenn f und g Funktionen sind, die natürliche Zahlen auf positive reelle Zahlen abbilden ($g : \mathbb{N} \rightarrow \mathbb{R}^+$), so ist f VON DER ORDNUNG g , wenn f ein Element folgender Menge von Funktionen ist:

$$O(g(n)) = \{f(n) \mid \exists \text{ positive Konstanten } n_0, c, \text{ so dass } \forall n \geq n_0 : 0 \leq f(n) \leq c \cdot g(n)\}$$

Schreibweisen: $f(n) \in O(g(n))$ oder — weniger exakt — $f(n) = O(g(n))$

!!! WICHTIG (wörtlich!)

- Die O-Notation liefert KEINE Information über die TATSÄCHLICHE Laufzeit eines Algorithmus
- Die O-Notation drückt NUR aus, WIE SICH DIE LAUFZEIT IM VERHÄLTNIS ZUR PROBLEMGRÖSSE ENTWICKELT
- Die tatsächliche Laufzeit liegt DARUNTER

★★★★ DIE 5 SCHRANKEN (auswendig!)

Symbol	Bedeutung	Formale Bedingung	Interpretation
O	Obere Schranke	$\exists c > 0, \exists n_0, \forall n > n_0 : f(n) \leq c \cdot g(n) $	★ f wächst HÖCHSTENS SO SCHNELL wie g
o (klein-o)	strikte obere Schranke	$\dots f(n) < c \cdot g(n) $	f wächst WENIGER SCHNELL (langsamer) als g
Ω	Untere Schranke	$\exists c > 0, \exists n_0, \forall n > n_0 : f(n) \geq c \cdot g(n) $	★ f wächst MINDESTENS SO SCHNELL wie g
ω (klein-omega)	strikte untere Schranke	$\dots f(n) > c \cdot g(n) $	f wächst SCHNELLER als g
Θ	Enge (harte) Schranke	$\exists c_0 > 0, \exists c_1 > 0, \exists n_0, \forall n > n_0 : c_0 g(n) \leq f(n) \leq c_1 g(n) $	★ f wächst GENAU SO SCHNELL wie g

🔑 Von PRAKTISCHER RELEVANZ ist die O-Notation. ⚠️ Hinweis: Die einzelnen Funktionen müssen sich nicht berühren oder schneiden.

Grenzwertkriterien

$$f \in O(g) : \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \leq c \quad f \in \Omega(g) : \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} \leq c \quad f \in \Theta(g) : c_0 \leq \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \leq c_1$$

C.5 ★★★★★ DIE KOMPLEXITÄTSKLASSEN (die zentrale Tabelle!)

$$1 < \log(n) < \sqrt{n} < n < n \cdot \log(n) < n^2 < n^3 < 2^n < n!$$

$f(n)$	Bezeichnung	$n = 10$	$n = 1.000$	$n = 1.000.000$
1	konstant	1	1	1
$\log_2(n)$	logarithmisch	10 (Folie: „10 20“)	10	20
$\log^2(n)$	log-quadrat			
$\sqrt{n}$	wurzel	3	30	1.000
n	linear	10	1.000	1.000.000
$n \cdot \log(n)$		30	10.000	20.000.000
n^2	quadratisch	100	1.000.000	10^{12}
n^3	kubisch	1.000	10^9	10^{18}
2^n	exponentiell	1.024	10^{300}	$10^{300.000}$

Zur Berechnung der Beispielzahlen wurde $\log_2() = \text{ld}()$ verwendet.

! Die „optische Täuschung“

Bei kleinen n (bis ca. $n = 10$) kann $O(2^n)$ **noch unter** $O(n^2)$ oder $O(n^3)$ liegen. Das asymptotische Verhalten zeigt sich **erst bei großen n !**

C.6 ★★ RECHENREGELN für O

#	Regel	
1	$f = O(f)$	Reflexivität
2	$f, g = O(F)$	$\Rightarrow f + g = O(F)$
3	$f = O(F)$, c konstant	$\Rightarrow c \cdot f = O(F)$ ★ Konstante Faktoren fallen weg!
4	$f = O(F)$ und $g = O(f)$	$\Rightarrow g = O(F)$ Transitivität
5	$f = O(F)$ und $g = O(G)$	$\Rightarrow f \cdot g = O(F \cdot G)$
6	$f = O(F \cdot G)$	$\Rightarrow f = F \cdot O(G)$
7	$f = O(F)$ und $ F \leq G $	$\Rightarrow f = O(G)$

📝 ★ Die Beispiele des Dozenten

$$f(n) = 2n^2 + 7\sqrt{n} = O(n^2)$$

$$f(n) = 5^{100} \cdot n + 4 \cdot n \cdot \log(n) = O(n \cdot \log(n))$$

$$f(n) = 2^{2n} = (2^2)^n = 4^n = O(4^n)$$

$$f(n) = \log_{10}(n) = O(\log(n))$$

★ **Warum die Basis egal ist:** Die Basis muss in der O -Notation nicht berücksichtigt werden, da die Änderung der Basis zu einem **KONSTANTEN FAKTOR** führt:

$$\log_{10}(n) = \frac{\log_2(n)}{\log_2(10)} = \text{const} \cdot \log_2(n) = O(\log(n))$$

C.7 ★★ Modellannahmen

Laufzeit:

- Mathematische Operationen benötigen, unabhängig von der Größe der Operanden, die gleiche Zeiteinheit
- Zugriff auf Daten benötigt eine konstante Zeiteinheit
- Algorithmen werden sequentiell ausgeführt
- Statt einer genauen Laufzeit wird eine **SCHRANKE** angegeben
- Konstante Faktoren werden vernachlässigt

Speicherplatz:

- Die Speicherung eines elementaren Datenobjekts benötigt, unabhängig vom Wert, einen konstanten Speicherplatz

★★ Die Kostenzuordnung (auswendig!)

Konstrukt	Kosten
Mathematische Operationen	$\Rightarrow O(1)$
Bedingte Ausführungen	$\Rightarrow O(1)$
Funktionsaufrufe	$\Rightarrow O(1)$
Schleifen	$\Rightarrow O(\text{\#Schleifendurchläufe})$
Rekursion	$\Rightarrow T(n)$

⚠ **ACHTUNG:** $\sum_{i=1}^n O(i) \neq O(1) + O(2) + \dots + O(n)$

C.8 ★★★ REKURSIVE ALGORITHMEN

Die allgemeine Rekursionsgleichung

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

Symbol	Bedeutung
a	Anzahl der Teilprobleme, in die ein Problem zerlegt wird (= Anzahl der rekursiven Aufrufe einer Funktion)
b	Jedes Teilproblem hat die Größe $\frac{n}{b}$ des Originalproblems (Faktor, um welchen das Problem verringert wird)
$f(n)$	Alle NICHT-rekursiven Aufrufe der Funktion (Zeit zum Aufteilen und Zusammenfügen des Problems)

★ Die 3 Verfahren

Master-Theorem $\Rightarrow$ Substitutionsmethode $\Rightarrow$ Rekursionsbaum

🔑 Das ist die empfohlene Reihenfolge: zuerst das Master-Theorem probieren, dann die Substitution, zuletzt den Rekursionsbaum.

★★ Verfahren 1: SUBSTITUTIONSMETHODE

Vorgehen:

1. Gleichung $T(n)$ so oft in sich selbst einsetzen und vereinfachen, bis sich eine **VERMUTUNG ableiten lässt** $\rightarrow$ Aufstellen einer allgemeinen Form
2. **Berechnen, wie viele Substitutionsschritte notwendig sind** — wie oft muss rekursiv eingesetzt werden, um auf einen **Rekursionsabbruch** zu treffen? (Teilprobleme so klein, dass sie direkt gelöst werden können)
3. **Beweis der Vermutung durch VOLLSTÄNDIGE INDUKTION**

📝 **Beispiel:** $T(n) = \begin{cases} 2T\left(\frac{n}{2}\right) + \frac{n}{2}, & n > 2 \\ 1, & n \leq 2 \end{cases}$

Schritt 1 — einsetzen:

$$T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{2} = 4T\left(\frac{n}{4}\right) + 2 \cdot \frac{n}{2} = 8T\left(\frac{n}{8}\right) + 3 \cdot \frac{n}{2} = \dots = 2^i T\left(\frac{n}{2^i}\right) + i \cdot \frac{n}{2}$$

Schritt 2 — Abbruchbedingung: $\frac{n}{2^i} \leq 2 \Leftrightarrow n \leq 2^{i+1} \Leftrightarrow \log_2(n-1) \leq i$

$$T(n) = 2^{\log_2(n-1)} T(\cdot) + \log_2(n-1) \cdot \frac{n}{2} \approx n + \log_2(n) \cdot n \in O(n \cdot \log(n))$$

Schritt 3: Die Vermutung müsste noch durch Induktion bewiesen werden.

☆☆ Verfahren 2: REKURSIONSBAUM

? „Was machen wir, wenn uns die Intuition fehlt?“

Vorgehen:

1. Rekursionsbaum zeichnen
2. Abschätzen der HÖHE, der KNOTEN PRO EBENE und der KOSTEN PRO KNOTEN
 - Jeder Knoten repräsentiert den **Aufwand für ein Teilproblem** innerhalb der Rekursionshierarchie
 - **Aufsummieren innerhalb jeder Ebene** liefert den **Aufwand pro Ebene**
 - **Der Laufzeitbedarf für jede Ebene ist immer gleich**
 - **Summieren über alle Ebenen** liefert den **Gesamtaufwand**
3. Allgemeine Formel für die letzte Ebene aufstellen
4. Beweis der Vermutung durch vollständige Induktion

 **Beispiel:** $T(n) = 3T\left(\frac{n}{4}\right) + cn^2$

Größe	Wert
Teilproblemgröße auf Ebene i	$\frac{n}{4^i}$ — wird 1 für $i = \log_4(n)$
Anzahl Knoten pro Ebene	3^i
Aufwand pro Knoten auf Ebene i	$c\left(\frac{n}{4^i}\right)^2$
Aufwand pro Ebene	$\left(\frac{3}{16}\right)^i cn^2$
Anzahl Knoten auf unterster Ebene	$3^{\log_4(n)} = n^{\log_4(3)}$

$$T(n) = cn^2 \sum_{i=0}^{\log_4(n)-1} \left(\frac{3}{16}\right)^i + \Theta(n^{\log_4 3}) \leq cn^2 \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i + \Theta(n^{\log_4 3}) = cn^2 \cdot \frac{1}{1-\frac{3}{16}} + \dots \approx \boxed{O(n^2)}$$

★ **Verwendete Formel — unendliche geometrische Reihe:** $\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$ für $|x| < 1$

☆☆☆☆ Verfahren 3: DAS MASTER-THEOREM (auswendig!)

Für $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ mit $a \geq 1$, $b > 1$ und $f(n) > 0$:

$$E = \log_b a = \frac{\log a}{\log b}$$

In jedem der 3 Fälle wird die Funktion $f(n)$ mit n^E verglichen:

Fall	Wenn ...	Dann ...	Typ
1	$f(n) \in O(n^{E-\varepsilon})$ für ein $\varepsilon > 0$	$T(n) \in \Theta(n^E)$	Obere Abschätzung
2	$f(n) \in \Theta(n^E)$	$T(n) \in \Theta(n^E \cdot \log n)$	Exakte Abschätzung
3	$f(n) \in \Omega(n^{E+\varepsilon})$ für ein $\varepsilon > 0$ UND $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$ für $c < 1$	$T(n) \in \Theta(f(n))$	Untere Abschätzung

🔑 „Das Master-Theorem bietet eine SCHNELLE Lösung für die Frage, in welcher Laufzeitklasse eine gegebene rekursiv definierte Funktion liegt.“ ⚠️ „Lässt sich KEINER der drei möglichen Fälle auf die Funktion $T(n)$ anwenden, so muss man die Komplexitätsklasse ANDERWEITIG berechnen.“

★★★★ DIE 3 DURCHGERECHNETEN MASTER-THEOREM-BEISPIELE

Beispiel 1: $T(n) = 3 \cdot T\left(\frac{n}{3}\right) + \sqrt{n}$

- $a = 3, b = 3, f(n) = \sqrt{n}$
- $E = \log_b a = \log_3 3 = \frac{\log 3}{\log 3} = 1$
- **Fall 1 prüfen:** $\sqrt{n} = n^{0.5} \in O(n^{1-\varepsilon})? \rightarrow 0.5 = 1 - \varepsilon \Rightarrow \varepsilon = 0.5 > 0$ ✔
- $\Rightarrow T(n) \in \Theta(n)$

Beispiel 2: $T(n) = 4 \cdot T\left(\frac{n}{4}\right) + 4n$

- $a = 4, b = 4, f(n) = 4n, E = \log_4 4 = 1$
- **Fall 1 prüfen:** $4n \in O(n^{1-\varepsilon})? \rightarrow 4n^1 \in O(n^{1-\varepsilon}) \rightarrow 1 = 1 - \varepsilon$, da $\varepsilon > 0$ **WIDERSPRUCH** — es existiert kein $\varepsilon > 0$
- **Fall 2 prüfen:** $4n \in O(n^E) = O(n^1)? \rightarrow 1 = 1$ ✔ **wahre Aussage**
- $\Rightarrow T(n) \in \Theta(n \log n)$

Beispiel 3: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + n^2$

- $a = 2, b = 2, f(n) = n^2, E = \log_2 2 = 1$
- **Fall 1:** $n^2 \in O(n^{1-\varepsilon})? \rightarrow 2 = 1 - \varepsilon, \varepsilon > 0$ **Widerspruch**
- **Fall 2:** $n^2 \in O(n^1)? \rightarrow 2 = 1$ **Widerspruch**
- **Fall 3, 1. Bedingung:** $n^2 \in O(n^{1+\varepsilon})? \rightarrow 2 = 1 + \varepsilon \Rightarrow \varepsilon = 1 > 0$ ✔ $\rightarrow$ Fall 3 könnte gelten
- **Fall 3, 2. Bedingung:** $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$

$$2 \cdot f\left(\frac{n}{2}\right) = 2 \cdot \left(\frac{n}{2}\right)^2 = 2 \cdot \frac{n^2}{4} = \frac{1}{2}n^2 \leq c \cdot n^2$$

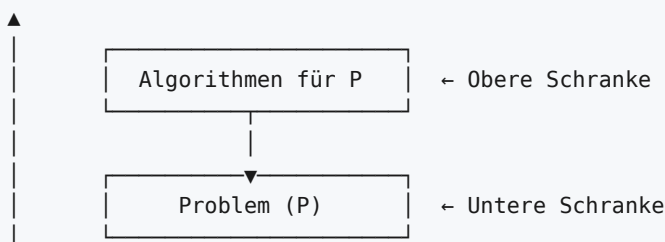
Dies gilt für alle $\frac{1}{2} \leq c < 1$ ✔

- $\Rightarrow T(n) \in \Theta(n^2)$

C.9 ★ Analyse von Algorithmen — die Kernfragen

- Welche Probleme sind lösbar?
- Welche Ressourcen (Zeit, Speicherplatz) werden MINDESTENS benötigt? $\rightarrow$ Untere Schranke des PROBLEMS
- Welche Ressourcen benötigt ein GEGEBENER Algorithmus? $\rightarrow$ Obere Schranke des ALGORITHMUS

Zeitbedarf



▼ ? Selbsttest Teil C

1. Wie ist Komplexität definiert (Formel)?
2. Effektivität vs. Effizienz — Definitionen und der Merksatz?
3. Nennen Sie 4 Gründe, warum man die Laufzeitkomplexität ermittelt.
4. Nennen Sie 4 Beispiele, was n konkret sein kann.
5. Fein- vs. Grobanalyse: Prinzip und Einsatzbereich?
6. Nennen Sie die 3 Fälle der Grobanalyse.
7. Wer führte die O-Notation ein, wer machte sie im deutschen Sprachraum bekannt?
8. Schreiben Sie die formale Definition von $O(g(n))$ auf.
9. Was sagt die O-Notation NICHT aus?
10. Nennen Sie alle 5 Schranken mit ihrer Interpretation.
11. Ordnen Sie: n^3 , $\log n$, 2^n , $n \log n$, 1 , $\sqrt{n}$, n , $n!$, n^2
12. Wie viele Operationen bei $n = 1.000.000$ für n^2 ? Für 2^n ?
13. Vereinfachen Sie: $3n^2 + 100n \log n + 5^{200}$
14. Warum ist die Basis des Logarithmus in O egal?
15. Welche Kosten haben Schleifen, Rekursion, Funktionsaufrufe?
16. Was bedeuten a , b und $f(n)$ in der Rekursionsgleichung?
17. Nennen Sie die 3 Schritte der Substitutionsmethode.
18. Nennen Sie die 4 Schritte des Rekursionsbaum-Verfahrens.
19. Schreiben Sie das Master-Theorem mit allen 3 Fällen auf.
20. Lösen Sie $T(n) = 2T(n/2) + n$ mit dem Master-Theorem. ($E=1$, Fall 2 $\rightarrow \Theta(n \log n)$)
21. Lösen Sie $T(n) = 8T(n/2) + n^2$. ($E=3$, Fall 1 $\rightarrow \Theta(n^3)$)
22. Was tut man, wenn kein Fall des Master-Theorems greift?

TEIL D — KAPITEL 04: SORTIERALGORITHMEN



Die 13 behandelten Verfahren: Selectionsort · Insertionsort · Bubblesort · Shakersort · Quicksort · Mergesort · Heapsort · Shellsort · Combsort · Bucketsort · Radixsort · Countingsort (+ *Bogosort als Anti-Beispiel*)

D.1 ★ Ziel und Grundbegriffe

Ziel des Sortierens: Datenobjekte sollen so **umgeordnet** werden, dass sie eine **Folge** bilden. Die Elemente der Folge sind **bzgl. ihrer SCHLÜSSELWERTE** sortiert — **aufsteigend (ascending)** oder **absteigend (descending)**.

Eingabe	Array A mit <code>array[1..N]</code>
Ausgabe	A mit vertauschten Einträgen, so dass $A[i] \leq A[i+1]$ für $1 \leq i < N$

★ Die 3 typischen Sortier-Operationen

Symbol	Operation
c_Z	Zugriffsoperation
c_V	Vergleichsoperation
c_D	Datenaustauschoperation — ⚠ ggf. $c_D \gg c_V$

🔑 **Merke:** Alle drei Operationen haben im O -Kalkül eine **konstante** Laufzeit. **In der Praxis können die Laufzeiten jedoch teils sehr stark variieren und die „Qualitätsordnung“ der Sortierverfahren „verschieben“.**

D.2 ★★★★★ Die 2 KLASSIFIKATIONS-EIGENSCHAFTEN

① STABILITÄT

	Definition
STABIL	Die Reihenfolge von GLEICHEN Daten/Elementen bleibt nach der Sortierung GLEICH bzw. unverändert
INSTABIL	Die Reihenfolge bleibt NICHT erhalten

📝 **Merkbeispiel des Dozenten:** Die Buchstabenpaare IL / ST, A / B bei gleichen Zahlenschlüsseln — bei stabiler Sortierung bleibt IL vor ST, bei instabiler nicht.

② PLATZKOMPLEXITÄT

	Latein	Deutsch	Definition
In-Place	<i>in situ</i> („am Ort“)	internes Sortieren	Elemente können DIREKT in der Liste bearbeitet und modifiziert (z. B. vertauscht) werden
Out-of-Place	<i>ex situ</i> („außerhalb des Ortes“)	externes Sortieren	Elemente können NICHT direkt in der Liste bearbeitet und modifiziert werden

D.3 ★★ SELECTIONSORT (Sortieren durch Auswahl)

Platzkomplexität	In-Place
Stabilität	⚠ INSTABIL

Idee: Es wird das **kleinste Element** aus dem zu sortierenden Feld ermittelt und mit dem **ersten Element, welches noch nicht zum sortierten Bereich gehört, vertauscht** → schreibe das kleinste Element aus $A[i \dots n]$ an die Position $A[i]$. Analog mit allen weiteren Elementen, bis alle einsortiert sind.

```

Selectionsort(A)
  // Suche das kleinste Element aus A[i,...,n]
  for (i = 1 to A.length-1)
    min = i
    for (j = i+1 to A.length)
      if (A[j] < A[min])
        min = j
    // Vertausche A[i] und A[min]
    SWAP(A[min], A[i])

```

Farbcode des Dozenten: ● aktuell analysiert · ● aktuelle Zielposition · ● aktuell kleinste Zahl · ● bereits sortiert

Laufzeit: Die Anzahl der Schritte ist **UNABHÄNGIG von den WERTEN** der Elemente — **keine**

Fallunterscheidung. Äußere Schleife n -mal, Array wird immer komplett durchlaufen, n **Vertauschungen**.

$$O(n) = \sum_{i=1}^n (n-i) = (n-1) + (n-2) + \dots + 2 + 1 = \frac{n(n-1)}{2} = \frac{n^2 - n}{2} \approx \frac{n^2}{2} = \boxed{O(n^2)}$$

★ „Selectionsort ist mit $O(n)$ **OPTIMAL** bzgl. der Anzahl der DATENAUSTAUSCHOPERATIONEN.“

D.4 ★★ INSERTIONSORT (Sortieren durch Einfügen)

Platzkomplexität	In-Place
Stabilität	✓ STABIL

Idee: Das erste Element gilt als bereits sortierter Bereich. Das nächste Element wird — je nach Wertigkeit — **vor oder hinter** dem ersten einsortiert → **füge das j -te Element in die sortierte Folge $A[1 \dots j-1]$ ein.**

```
Insertionsort(A)
  for (j = 2 to A.length)
    key = A[j]
    i = j - 1
    // setze A[j] sortiert in A[1,...,j-1] ein
    while (i > 0 and A[i] > key)
      A[i+1] = A[i]
      i = i - 1
    A[i+1] = key
```

Struktogramm-Variante:

```
FOR (i=2; i<=n; i++)
  tmp = A[i]; j = i;
  WHILE (j > 1) AND (A[j-1] > tmp)
    A[j] = A[j-1]; j--;
  A[j] = tmp;
```

★★ Fallunterscheidung

Fall	Verhalten	Komplexität
Best Case	Die While-Schleife wird NIE durchlaufen (Feld ist bereits sortiert) — alle Elemente von Index 1 bis $n-1$ durchlaufen, Anzahl der Vergleiche = $n-1$	$O(n) = (n-1) \cdot 1 \approx \boxed{O(n)}$
Worst Case	Die While-Schleife wird jedes Mal KOMPLETT durchlaufen	$\boxed{O(n^2)}$
Average Case	$O(n) = \frac{1}{2} \sum_{i=1}^n (n-i) = \frac{1}{2} \cdot \frac{n(n-1)}{2}$	$\boxed{O(n^2)}$

⚠ ★ **Hinweis des Dozenten:** *„Das Vorkommen einer WHILE-SCHLEIFE verursacht i. d. R. eine **FALLUNTERSCHIEDUNG!**“* Die Anzahl der Schritte ist **nicht nur von der Anzahl (n), sondern auch von deren VERTEILUNG abhängig.**

Detaillierte Worst-Case-Analyse (Feinanalyse):

$$(n-1) \cdot \underbrace{(4Z + V + I)}_{=:A} + \underbrace{(Z + 2V + I)}_{=:B} \cdot \sum_{i=1}^{n-1} i \quad \text{mit} \quad \sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} = \frac{n^2}{2} - \frac{n}{2}$$

$$\Rightarrow \frac{n^2}{2} \cdot B + n \cdot \left(A - \frac{B}{2}\right) + \underbrace{\dots}_{\text{irrelevant}} \Rightarrow \text{Größenordnung} = \boxed{n^2}$$

D.5 ★★ BUBBLESORT (Sortieren durch Aufsteigen / Austauschsortieren)

Platzkomplexität	In-Place · Stabilität: stabil
------------------	-------------------------------

Idee: Benachbarte Elemente werden solange verglichen und ggf. vertauscht, bis das Feld sortiert ist. Das Feld wird von vorne nach hinten durchlaufen, bis sich das **größte Element am Ende** befindet; analog für das um ein Element verkürzte Feld.

```
Bubblesort(A)
  for (i = 1 to A.length)
    for (j = A.length downto i+1)
      if (A[j] < A[j-1])
        SWAP(A[j], A[j-1])
```

★ Die Verbesserung

```
Bubblesort(A)
  for (i = 1 to A.length)
    noSwap = TRUE
    for (j = A.length downto i+1)
      if (A[j] < A[j-1])
        SWAP(A[j], A[j-1])
        noSwap = FALSE
    if (noSwap) break           // ★ Frühabbruch!
```

Fall	Komplexität
Best Case (mit Verbesserung)	$\boxed{O(n)}$
Average Case	$O(n^2)$
Worst Case	$\boxed{O(n^2)}$

D.6 ★ SHAKERSORT (Bidirektionales Austauschsortieren)

Auch bekannt als **Cocktailsort**, **Ripplesort**, **Shearsort** oder **BiDiBubbleSort**.

Idee: Das Feld wird **abwechselnd von vorne nach hinten und von hinten nach vorne** durchlaufen, bis sich das **größte Element am oberen Ende UND das kleinste Element am unteren Ende** befindet.

🔑 „Durch diese **Bidirektionalität** kommt es zu einem **schnelleren Absetzen von großen bzw. kleinen Elementen**.“

D.7 ★★★★★ QUICKSORT

Erfinder	Charles Antony Richard HOARE
Platzkomplexität	In-Place
Stabilität	⚠️ NICHT STABIL
Strategie	★ Teile und Herrsche (Divide and Conquer)

★★ Definition Divide-and-Conquer

Ein zu lösendes Problem wird auf ein IDENTISCHES Problem mit KLEINERER Eingabemenge zurückgeführt — d. h. ein Problem wird **so lange REKURSIV** in kleinere und einfachere Teilprobleme zerlegt, bis dieses (beherrschbar) ist und gelöst werden kann. Die Lösung wird anschließend aus den TEILLÖSUNGEN zusammengesetzt.

🔑 **Merke:** Ausnutzen der Eigenschaft, dass bei vielen Problemen der **Lösungsaufwand SINKT**, wenn das Problem in kleinere Teilprobleme zerlegt wird. (Rekursive Programmierung)

Der Quicksort-Ansatz

1. **Teile die Folge in zwei Teilfolgen** $A[1 \dots q-1]$ und $A[q+1 \dots n]$ mit $A[i] \leq A[q] \ \forall i < q$
2. **Sortiere die Teilfolgen** rekursiv

★ **Hinweis:** Die **KOMBINATION der Teilfolgen ist aufgrund von Schritt 1 NICHT MEHR NOTWENDIG.**

Teilen: Man wählt ein beliebiges Element aus — das **PIVOTELEMENT** (*Pivot = Dreh-, Angelpunkt*) mit Wertigkeit x . Es bildet **die Grenze zwischen den Teilfeldern**: links nur Elemente $\leq x$. **Herrschen:** Das Verfahren des Aufteilens wird **rekursiv auf beide Teilfelder** angewandt, bis keine weitere Teilung mehr möglich ist.

```
Quicksort(A, left, right)
  if (left < right)
    q = PARTITION(A, left, right)  // Bestimmt q und stellt Bedingung 1 her
    Quicksort(A, left, q-1)
    Quicksort(A, q+1, right)
// Initialer Aufruf: QUICKSORT(A, 1, A.length)
```

★★ PARTITION nach HOARE

```
PARTITION_Hoare(A, left, right)          // A[right] = Pivotelement
  i = left - 1
  j = right
  while i < j
    repeat i = i + 1 until A[i] >= A[right]
    repeat j = j - 1 until A[j] <= A[right]
    if (i < j) SWAP(A[i], A[j])
  // Schiebt das Pivotelement in die Mitte zwischen die Teilfolgen
  SWAP(A[i], A[right])
  return i                                // Position des Pivotelements
```

★★ PARTITION mit nur EINER Schleife (Cormen)

```

PARTITION(A, left, right)
    i = left - 1
    for (j = left to right-1)
        if (A[j] <= A[right])
            i = i + 1
            SWAP(A[i], A[j])
    SWAP(A[i+1], A[right])
    return i+1

```

PARTITION zerlegt das Array in 4 Bereiche:

Bereich	Bedingung
$\text{left} \leq k \leq i$	$A[k] \leq A[\text{right}]$
j	$A[k] > A[\text{right}]$
$j \leq k < \text{right}$	$A[k] = ?$ (noch unbekannt)
$k = \text{right}$	das Pivotelement

Fall	Aktion
Fall 1: $A[j] > A[r]$	Verlängere Bereich 2 um ein Feld
Fall 2: $A[j] \leq A[r]$	Vertausche $A[i]$ und $A[j]$; vergrößere Bereich 1; verlängere Bereich 2

★★★★ Laufzeitverhalten

BEST CASE ($k = \frac{n}{2}$, kontinuierliche Halbierung):

$$\begin{aligned}
 T(n) &= 2T\left(\frac{n}{2}\right) + cn = 4T\left(\frac{n}{4}\right) + 2cn = \dots = 2^k T\left(\frac{n}{2^k}\right) + k \cdot cn \quad \text{mit } k = \lg(n) \\
 &= n + \lg(n) \cdot cn \approx \boxed{O(n \cdot \log(n))}
 \end{aligned}$$

($\lg_2(n) = \lg(n)$ Durchläufe, ungefähr n Vergleiche pro Durchlauf.)

AVERAGE CASE: Im Schnitt $c \cdot n \cdot \log(n)$ Vergleiche, wobei $1,4 \leq c \leq 2$ (mittels Analysen ermittelt) $\rightarrow \boxed{O(n \log n)}$

WORST CASE ⚠ Das Pivotelement ist immer das **kleinste oder größte** Element — d. h. das **Feld ist bereits auf- oder absteigend sortiert**:

$$T(n) = T(1) + T(n-1) + cn \Rightarrow \dots \approx c \cdot n + c \cdot n^2 \approx \boxed{O(n^2)}$$

⚠ ★ Dieses Problem wird als **DEGENERATION** bezeichnet. ★ Zwei Verfahren, für die das Problem der Degeneration **NICHT** besteht, die aber im Average Case etwas schlechter sind: **MERGESORT** und **HEAPSORT**.

★★★★ DAS PIVOTELEMENT — das Auswahlproblem

? Wie findet man das optimale Pivotelement? **Auswahlproblem:** Suche des i -t größten Elements in einem unsortierten Feld aus paarweise verschiedenen Zahlen mit Suchindex $1 \leq i \leq n$. 🔑 **Der MEDIAN wird gebraucht, um in einem Divide-and-Conquer-Verfahren eine GUTE TEILUNGSSTRATEGIE zu haben!**

Problem	i	Komplexität
Minimum-Problem	$i = 1$	$O(n)$
Maximum-Problem	$i = n$	$O(n)$
★ Median-Problem	$i = \frac{n}{2}$	? — „Die Bestimmung des Medians ist recht aufwändig, da man in einem unsortierten Feld zuerst jedes Element einmal betrachten müsste. Das Problem wird immer schwieriger, je mehr man sich $\frac{n}{2}$ nähert.“

★★ Die 6 Möglichkeiten zur Pivotwahl

#	Verfahren	Beschreibung	Bewertung
1	Wahl des mittleren Elements	Pivot = $\frac{\text{letztes} - \text{erstes Element}}{2} \rightarrow$ möglichst symmetrische Aufteilung	✗ nicht wirklich gut für den praktischen Einsatz
2	Iteratives Löschen der Minima	Sukzessives Streichen des Minimums, $T(n, i) = \Theta(i \cdot n)$ — Minimum/Maximum/Median jeweils $O(n^2)$	✗
3	Sortieren des Feldes	z. B. HEAPSORT(A), dann output(a, i) — Laufzeit $O(n \log n)$	✗ viel zu großer Aufwand!
4 ★	RAND-SELECT (partielle Sortierung mit Quicksort) — Tony Hoare	Zufällige Auswahl des Pivotelements über einen Zufallszahlengenerator; partielle Sortierung, Aufruf nur des linken ODER rechten Teils. Verringert die Wahrscheinlichkeit einer ungünstigen Teilung. Best-Case $O(n)$, Worst-Case $O(n^2)$	✓ praxisrelevant — „Wählt man zufällig ein Pivotelement, dann kann man mathematisch beweisen, dass im Mittel eine Laufzeit von $O(n \log n)$ erreicht wird — unabhängig davon, wie die Daten im Feld verteilt sind.“
5 ★	Median-of-three — Richard C. Singleton	Man wählt drei Elemente (per Zufall) — z. B. das erste, mittlere und letzte — sortiert sie separat; das mittlere Element ist das Pivotelement. Erhöhung der Wahrscheinlichkeit durch Median-of-five, -seven usw.	✓ praxisrelevant — erhöht die Wahrscheinlichkeit, ein Element näher an der Mitte zu finden
6 ★★ ★★	SELECT-Algorithmus (Median der Fünfer-Mediane)	① Teile die Menge in Fünfer-Gruppen · ② Berechne die Fünfer-Mediane (mit Insertionsort) · ③ Berechne rekursiv den Median der Fünfer-Mediane	✓ DETERMINISTISCH und löst das Auswahlproblem auch im WORST-CASE in LINEARER Zeit

```

FIND-PIVOT-POSITION(A, left, right)
  n = right - left + 1                // Anzahl der Elemente
  // Berechne die Mediane von je 5 benachbarten Elementen
  for (i = 1 to n/5)
    INSERTION-SORT( A[left+5(i-1)] , ... , min(left+5i, right) )
    M[i] = A[ min(left + 5i - 3, right) ] // Median des 5er-Blocks
  // Berechne rekursiv den Median der Mediane-von-5
  x = SELECT(M, 1, n/5, (n/5)/2)
  return (min(left + 5x - 3, right))  // Position im ursprünglichen Array

```

★ **Praxisrelevanz (wörtlich):** „Der **SELECT-Algorithmus** löst das Auswahlproblem im Worst-Case in asymptotisch linearer Zeit. Aufgrund seiner (mathematischen) Eigenschaft ist er **nur von Interesse, falls die Eingaben sehr groß sind und eine Laufzeitgarantie gefordert ist**. Der **RAND-SELECT-Algorithmus** wird in der **PRAXIS EHER EINGESETZT** und kann das Auswahlproblem in **asymptotisch linear ERWARTETER** Laufzeit lösen.“

★★ Warum bewirkt eine Halbierung ein LOGARITHMISCHES Verhalten?

Bei jeder Iteration halbiert sich die Zahl der noch zu bearbeitenden Elemente:

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \dots \rightarrow \frac{n}{2^x} \approx 1 \implies n = 2^x \implies \boxed{x = \log_2(n) = ld(n)}$$

(x = Anzahl der maximal erforderlichen Iterationen.) Der **Gesamtaufwand** ist die geometrische Reihe:

$$\sum_{k=0}^{\infty} \left(\frac{1}{2}\right)^k = 1 + \frac{1}{2} + \frac{1}{4} + \dots = \frac{1}{1-\frac{1}{2}} = 2$$

D.8 ★★★★★ MERGESORT (Mischsortieren)

Platzkomplexität	⚠ OUT-OF-PLACE
Stabilität	✅ STABIL
Strategie	Divide and Conquer

Idee: Zerlegung des Feldes in zwei Teilfelder $L = A[1 \dots \lfloor \frac{n}{2} \rfloor]$ und $R = A[\lfloor \frac{n}{2} \rfloor + 1 \dots n]$.

⚠ ★ **DER ENTSCHEIDENDE UNTERSCHIED ZU QUICKSORT:** „Im Gegensatz zu Quicksort findet **WÄHREND DER TEILUNG KEINE SORTIERUNG** statt.“ — Das Aufteilen erfolgt **rekursiv, bis ein einziges Element übrig bleibt**; die eigentliche Arbeit passiert beim **Verschmelzen (Merge)**.

```
Mergesort(A, left, right)
  if (left < right)
    m = floor((left + right) / 2)
    Mergesort(A, left, m)
    Mergesort(A, m+1, right)
    Merge(A, left, m, right)
```

★★ Laufzeitverhalten

Die Anzahl der Schritte ist **unabhängig von den Werten** — **keine Fallunterscheidung**.

- **Teilen** geht in **konstanter Zeit** $O(1)$ (nur Positionsbestimmung) $\rightarrow 2 \cdot T(\frac{n}{2})$
- **Mergeoperation** ist **proportional zur Anzahl der Elemente** in L und R zusammen $\rightarrow O(n)$

$$T(n) = \underbrace{T(\lfloor \frac{n}{2} \rfloor)}_{\text{linke Hälfte}} + \underbrace{T(\lceil \frac{n}{2} \rceil)}_{\text{rechte Hälfte}} + \underbrace{n}_{\text{Zusammenfügen}} = 2T(\frac{n}{2}) + cn = \boxed{O(n \cdot \log(n))}$$

⚠ ★★ Die spannende Frage: Warum nicht in k Teile teilen?

? „Eine Halbierung beschleunigte das Verfahren von n^2 auf $\log(n)$! Dann sollte das Verfahren nochmal schneller werden, wenn die Aufteilung in 3, 4 oder mehr Teile erfolgt!?!“

Teile und verschmelze in k Teilfolgen:

$$T(n) = k \cdot T\left(\frac{n}{k}\right) + c \cdot k \cdot n = \boxed{O(n \cdot \log(n))} \quad \text{— KEIN Gewinn!}$$

★ **Begründung:** „In jeder Iteration muss das **MINIMUM AUS k LISTENKÖPFEN** bestimmt werden. Die Bestimmung des Minimums von k Listen erfolgt **nicht in konstanter Zeit, sondern k Schritte**.“

Extremfall $k = n$:

$$T(n) = n \cdot T(1) + c \cdot n^2 = \boxed{O(n^2)} \quad \text{— SCHLECHTER!}$$

! Speicherbedarf

Der Speicherbedarf umfasst das zu sortierende Array ($c \cdot n$) **UND ein zweites Array zur Zwischenspeicherung** ($c \cdot n$) — d. h. es ist der **DOPPELTE Speicherbedarf** notwendig. Bei der Implementierung mit Listen ist der Speicher-Overhead der Listenzeiger zu verwalten.

📝 Merge mit Arrays (Pseudocode-Kern)

```
i = l; j = m + 1; k = 1           // Indizes von L.Head, R.Head, A.Rear
while (i <= m) and (j <= r) do    // Beide Listen nicht leer
    if (A[i] <= A[j]) B[k] = A[i]; i = i + 1
    else B[k] = A[j]; j = j + 1
    k = k + 1
// Anfügen des Rests der anderen Liste
if (i > m) for (h=j to r) B[k+h-j] = A[h] // Liste L ist leer
else for (h=i to m) ... // Liste R ist leer
// Speichere sortierte Folge von B zurück nach A
for (h=l to r) A[h] = B[h]
```

★ Divide-and-Conquer: Quicksort vs. Mergesort (Recap-Tabelle!)

	Quicksort	Mergesort
Teilen	★ KERNIDEE (Partition sortiert bereits!)	einfach (nur Mitte bestimmen)
Rekombination	einfach (entfällt!)	★ KERNIDEE (Merge sortiert!)

D.9 ★★★★★ HEAPSORT (Haldensortierung)

Motivation: Selectionsort arbeitete damit, dass es das kleinste/größte Element auswählte und an die sortierte Liste anhängte. **Das SUCHEN dieses Elements war jedoch sehr zeitaufwändig. ?** Wie gestaltet man diesen Auswahlsschritt effizient? → 💡 **Mit Hilfe eines HEAPS.**

★★ Was ist ein Heap?

- **Spezielle Datenstruktur**
- **Basiert auf der Definition eines (fast) vollständigen BINÄREN BAUMS**
- **! Heap (Sortieren) ≠ Heap (dynamischer Speicher)!**
- **Bildhaft:** Objekte liegen auf einer **Halde**; ein Objekt ist immer **mindestens so groß wie die darunter liegenden Objekte**.

★★★★ Die HEAP-EIGENSCHAFT

	Definition	Formal
MAX-Heap	Die Wertigkeit der Wurzel ist GRÖßER als alle ihre Nachfolger	$\forall \text{ Knoten } i \text{ außer der Wurzel: } A[\text{PARENT}(i)] \geq A[i]$
MIN-Heap	Die Wertigkeit der Wurzel ist KLEINER als alle ihre Nachfolger	$\forall \text{ Knoten } i \text{ außer der Wurzel: } A[\text{PARENT}(i)] \leq A[i]$

🔑 **MERKE:** Gilt $A[\text{PARENT}(i)] \geq A[\text{LEFT}(i)]$ **UND** $A[\text{PARENT}(i)] \geq A[\text{RIGHT}(i)]$ **für ALLE Teilbäume**, dann ist der Baum ein (MAX-)Heap. 🔑 Spricht man nur von einem **Heap**, so ist damit i. d. R. ein **MAX-Heap** gemeint.

★★★ Array-Darstellung des Heaps

Feld $h[1 : n]$, $h[1]$ = **Wurzel**. Für jedes Element an der Stelle i gilt:

$$\text{Linker Nachfolger: } 2i \quad \text{Rechter Nachfolger: } 2i + 1 \quad \text{Vorgänger: } \left\lfloor \frac{i}{2} \right\rfloor$$

★★★ Die Eigenschaften eines Heaps

#	Eigenschaft
i)	Ein Heap der Größe n hat maximal die Höhe $\lfloor \log(n) \rfloor$
ii)	Auf der Ebene i liegen maximal 2^i Knoten
iii)	★ Auf das Element mit dem größten/kleinsten Wert kann in KONSTANTER ZEIT $O(1)$ zugegriffen werden — unabhängig von n !
iv)	Entfernen des größten Elements in $O(\log n)$ (übrig bleibt ein Heap mit $n - 1$ Elementen)
v)	Hinzufügen eines neuen Elements in $O(\log n)$ (Heap mit $n + 1$ Elementen)
vi)	★ Der Baum muss nicht vollständig, aber LINKSVOLLSTÄNDIG (linkseindeutig) sein

⚠️⚠️ **HEAP-BAUM $\neq$ SORTIERTER BINÄRBAUM!** Der Heap basiert aber auf einer binären Baumstruktur. Er ist **PARTIELL SORTIERT**: jeder Teilbaum ist partiell sortiert und damit auch der gesamte Baum. Ein **Blatt als trivialer Teilbaum ist immer sortiert**.

★★★★ Die 3 HEAP-OPERATIONEN

Operation	Aufgabe
BUILD-MAX-HEAP	Baue eine Halde aus einer Menge von Elementen
HEAP-EXTRACT-MAX	Nehme das größte Element von der Halde
MAX-HEAPIFY	Stelle die Heap-Bedingung nach der Entnahme des Elements wieder her
(gilt analog für MIN)	

```

HEAPSORT(A)
    BUILD-MAX-HEAP(A)                // A[1] enthält das Maximum
    A.heap_size = A.length
    for (i = A.length downto 2)
        SWAP(A[i], A[1])              // ≈ "HEAP-EXTRACT-MAX"
        A.heap_size = A.heap_size - 1
        MAX-HEAPIFY(A, 1)

MAX-HEAPIFY(A, i)
    if (LEFT(i) <= A.heap_size and A[LEFT(i)] > A[i])    max = LEFT(i)
    else                                                    max = i
    if (RIGHT(i) <= A.heap_size and A[RIGHT(i)] > A[max]) max = RIGHT(i)
    if (max != i)                                           // Heap-Eigenschaft war verletzt
        SWAP(A[i], A[max])
        MAX-HEAPIFY(A, max)                                // rekursiv weiter nach unten

BUILD-MAX-HEAP(A)
    A.heap_size = A.length
    for (i = floor(A.heap_size/2) downto 1)    // ★ nur bis zur Hälfte!
        MAX-HEAPIFY(A, i)

HEAP-EXTRACT-MAX(A)
    m = A[1]
    A[1] = A[A.heap_size]                // letztes Element nach oben
    A.heap_size = A.heap_size - 1
    MAX-HEAPIFY(A, 1)
    return m

```

★★ Wo fängt MAX-HEAPIFY an? (Die Indexformel!)

$$\text{WurzelIndex} = \left\lfloor \frac{N}{2} \right\rfloor$$

🔑 **Begründung:** Die Elemente $\lfloor A.\text{heap_size}/2 \rfloor + 1$ bis $A.\text{heap_size}$ sind die **BLÄTTER** — und **ein-elementige Heaps sind bereits sortiert!** BUILD-MAX-HEAP verwendet MAX-HEAPIFY, um den Heap **von unten nach oben** aufzubauen.

★ Das Heapsort-Vorgehen (6 Schritte)

- ① Aufbauen des Heaps – alle Elemente werden entsprechend ihrer Position im Feld der Reihe nach eingetragen
- ② Bestimmung des Index, bei dem die Prüfung der Heap-Eigenschaft beginnt
- ③ Transformation des Heaps, damit die Heap-Eigenschaft erfüllt wird
- ④ Entfernen der Wurzel aus dem Heap
- ⑤ Das letzte Element wird an die Position der Wurzel verschoben
- ⑥ Wiederhole bei Schritt ②, bis der Heap leer ist

Zwei Phasen: AUFBAUPHASE (alle Elemente in einen Heap eintragen) · **ABBAUPHASE** (Elemente schrittweise entfernen und **von hinten** in das Feld eintragen)

★★ Laufzeitverhalten

Bestandteil	Komplexität	Begründung
BUILD-MAX-HEAP	$O(n)$	$\frac{n}{2}$ Durchläufe; genauer: $T(n) = c \sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil h = O\left(n \sum_{h=0}^{\log n} \frac{h}{2^h}\right) = O(n)$ mit der Ableitung der geometrischen Reihe $\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1-1/2)^2} = 2$
MAX-HEAPIFY	$O(\log n)$	$T_{\text{HEAPIFY}}(n) = T_{\text{HEAPIFY}}\left(\frac{2n}{3}\right) + \Theta(1) = O(\log n)$
HEAP-EXTRACT-MAX	$O(1) + \text{Heapify}$	for-Schleife $(n-1)$ -mal, pro Durchlauf $\log_2(n) = ld(n)$
★ GESAMT	$O(n \cdot \log(n))$	★ Keine Fallunterscheidung — auch im WORST CASE!

Herleitung der $\frac{2n}{3}$: Für den linksvollständigen Baum gilt mit $N = 2^h - 1$ (Knoten des vollständigen Baums gleicher Höhe): Knoten je Teilbaum $\frac{N-1}{2}$, Blätter $\frac{N+1}{2}$; im Worst Case $n_L = \frac{3N-1}{4}$, woraus $N = \frac{4n+1}{3}$ und damit $n_L \leq \frac{2n}{3}$ folgt.

D.10 ★ SHELLSORT

Platzkomplexität	In-Place · Stabilität: ⚠ nicht stabil
-------------------------	--

Vorüberlegung: Bei Insertionsort ist die **Anzahl der erforderlichen Zuweisungen relativ hoch** — jedes Element, das von seiner Zielposition weit entfernt liegt, muss über eine **weite Strecke verschoben** werden, und **eine große Anzahl von Elementen muss jeweils um eine Position nach hinten verschoben** werden. **Idee:** ★ Shellsort verfolgt den Ansatz, **die DISTANZ, um die ein Element verschoben werden muss, zu VERRINGERN**. Es werden **nur Elemente miteinander verglichen, die einen bestimmten ABSTAND (Distanz h) zueinander haben**. Der Abstand wird in jedem weiteren Durchlauf verkleinert. **Im letzten Schritt ($h = 1$) wird wieder mit Insertionsort sortiert.**

```

ShellSort(A)
  for (gap = n/2; gap > 0; gap /= 2)
    for (i = gap; i < n; i += 1) {
      int temp = A[i];
      for (j = i; j >= gap && A[j - gap] > temp; j -= gap)
        A[j] = A[j - gap];
      A[j] = temp;
    }

```

Distanzbestimmung: $h = \left\lfloor \frac{N}{2} \right\rfloor$ (und dann jeweils halbieren)

🔑 Bei $h = 1$ (letzte Runde) ist Shellsort wie Insertionsort.

★★ Die SCHRITTFOLGEN (h-Folgen) und ihre Komplexität

Folge	Werte	Komplexität
Ursprünglich von Shell	$2^n, \dots, 4, 2, 1$	$O(n^2)$ ⚠️ aus praktischer Sicht nicht gut geeignet — „Es werden nur GERADE Stellen in der Vorsortierung berücksichtigt; die UNGERADEN Positionen werden erst im letzten Schritt berücksichtigt“
Folge von HIBBARD	$1, 3, 7, 15, 31, 63, \dots, 2^n - 1$	$O(n^{1.5})$
Folge von PRATT	$1, 2, 3, 4, 6, 8, 9, 12, \dots, 2^{p3^q}$	$O(n \cdot \log(n)^2)$

⚠️ ★ **Die Existenz einer Folge mit der Komplexität $O(n \cdot \log(n))$ wurde bereits AUSGESCHLOSSEN (Bjorn Poonen, Plaxton, Suel).** Beweis, dass für hinreichend großes n immer eine Folge mit einer Komplexität von $O(n^{1+e})$ gefunden werden kann.

Warum ist das so?

- **Zu GROSSE Abstände** zwischen den Folgegliedern ergeben **zu große Verschiebungen**
- **Zu ENGE Abstände** bewirken **zu viele Durchläufe** bis zur letztendlichen Sortierung

⚠️ * „Die Analyse der Laufzeitkomplexität des Shellsorts ist **sehr anspruchsvoll und noch nicht im vollen Umfang gelungen. Bis heute wurde keine Funktion veröffentlicht, die das Laufzeitverhalten vollständig beschreibt.**“ * ✓ **Vorteil:** „Shellsort kann **bereits vorhandene Sortierungen ausnutzen.** Dabei spielt es nur eine geringe Rolle, ob das Array sortiert **oder invers sortiert** vorliegt.“ ⚠️ „Bei nur neun Elementen wäre sogar **Insertionsort effizienter als Shellsort.**“

D.11 ★ COMBSORT

Combsort = Bubblesort (verbessert) + Shellsort

Vorüberlegung: Bubblesort ist für den praktischen Einsatz **völlig ungeeignet** — es **erkennt während der Sortierung nicht, ob ein Feld bereits sortiert ist.** Die 1. Verbesserung war der „Einbau“ der boolschen Variablen `noSwap`; diese verbesserte Variante bildet die **Basis für Combsort.**

```
Combsort(A)
    int gap = n; bool swapped = true;
    while (gap != 1 || swapped == true)
        // Finde die nächste Distanz und verkleinere gap um Faktor 1.3
        if (gap < 1) gap = 1;
        else gap = (gap*10)/13;
        swapped = false;
        for (int i = 0; i < n-gap; i++)
            if (a[i] > a[i+gap])
                swap(a[i], a[i+gap]); swapped = true;
```

$$\text{gap} = \left\lfloor \frac{n}{1.3} \right\rfloor \quad (\text{der „Shrink-Faktor“ } 1.3)$$

🔑 **Bei gap = 1 (letzte Runde) ist Combsort wie Bubblesort. Laufzeit:** „Die Laufzeitanalyse gestaltet sich genauso schwierig wie bei Shellsort ... das Laufzeitverhalten liegt, je nach Vorsortierung, **zwischen $O(n^2)$ und $O(n \cdot \log(n))$.**“

D.12 😊 BOGOSORT (Sortieren durch Jonglieren)

Auch bekannt als **Monkeysort**, **Stupidsort**, **Dumbsort** oder als **Las-Vegas-Algorithmus**. Stabilität: **instabil**. **Idee**: „Die Elemente werden so lange **ZUFÄLLIG GEMISCHT**, bis sie sortiert sind.“

```
Bogosort(A)
  while not Sorted(list) do
    shuffle(list)
  done
```

Fall	Komplexität
Best Case	$O(n) = (n - 1) \cdot 1 \approx n$
Worst Case	$O(n) = n \cdot [n \cdot (n - 1) \cdot (n - 2) \cdots 1] = n \cdot n!$
Wahrscheinlichkeit bei $n = 9$	$\frac{1}{9!} = \frac{1}{362.880} = 0,0000027557319224$

D.13 ★★★★★ IST DAS MAXIMUM ERREICHT? — Die untere Schranke

Die Zwischenbilanz-Tabelle

Verfahren	Worst Case	Bemerkung
Selectionsort	$O(n^2)$	
Insertionsort	$O(n^2)$	★ Vorteilhaft, wenn die Eingabe nahezu sortiert ist
Bubblesort	$O(n^2)$	⚠ Generell NICHT empfehlenswert
Quicksort	$O(n^2)$	★ $O(n \log n)$, wenn der Median der Eingabefolge in $O(n)$ Zeit bestimmt werden kann
Mergesort	$O(n \log n)$	✅ Sehr gutes Laufzeitverhalten in der Praxis · eignet sich gut für große Datenbestände · ★ lässt sich gut PARALLELISIEREN · ⚠ ist NICHT in-place
Heapsort	$O(n \log n)$	✅ in-place UND $O(n \log n)$ im Worst-Case
Shellsort	$O(n^2)$	Worst-Case kann besser sein; nicht eindeutig bestimmbar : $O(n \log n) < O \leq O(n^2)$
Combsort	$O(n^2)$	Worst-Case kann besser sein

★★★★★ Das ENTSCHEIDUNGSBAUMMODELL und die untere Schranke

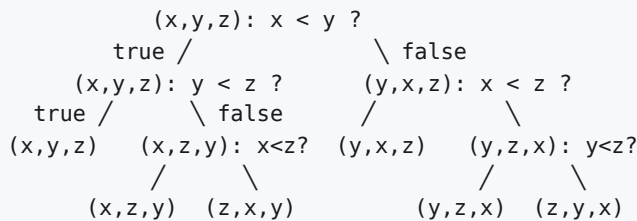
❓ „Wie schnell kann man **prinzipiell** sortieren? Sind $n \log n$ -Sortierverfahren **laufzeitoptimal**? Ist die optimale Laufzeitschranke schon erreicht?“

Die Laufzeiteffizienz wird bestimmt durch: die **Anzahl der Schlüsselvergleiche** und die **Anzahl der Daten, die bewegt werden müssen** (Verwaltungsarbeit).

Unter den Annahmen:

1. Alle zu sortierenden Elemente sind **PAARWEISE VERSCHIEDEN**
2. Die Sortierv Verfahren basieren **AUSSCHLIESSLICH** auf **PAARVERGLEICHEN** zwischen den Elementen
→  **Darstellung als ENTSCHEIDUNGSBAUMMODELL** — ein **vollständiger binärer Baum**, dessen **BLÄTTER** die **PERMUTATIONEN** der Elemente sind

 **Beispiel** $n = 3$ (Elemente x, y, z): Die $n! = 6$ Permutationen als Blattknoten:



Für das Beispiel gilt: es sind **mindestens zwei und maximal drei Vergleiche** notwendig; **im Durchschnitt** $\frac{16}{6} = 2,67$ **Vergleiche**.  **Anzahl der Vergleiche $\approx$ HÖHE DES BAUMS**

★★★★ **DAS THEOREM (auswendig!)**

THEOREM: Jedes **VERGLEICHENDE** Sortierv Verfahren benötigt bei n zu sortierenden Datenelementen im **WORST-CASE** $\Omega(n \cdot \log(n))$ **VERGLEICHOPERATIONEN**.

Begründung:

- Jedes Sortierv Verfahren führt eine Serie von Vergleichsoperationen aus — entspricht **einem Pfad** von der Wurzel zum Blatt
- Jedes Sortierv Verfahren muss **JEDE MÖGLICHE PERMUTATION** erzeugen können
- Ein Entscheidungsbaum hat $n!$ Blätter
- Die **HÖHE** des Entscheidungsbaums ist eine **UNTERE SCHRANKE** für die Anzahl der Vergleiche:

$$h \geq \log(n!) \approx n \cdot \log(n) - n \approx \Omega(n \cdot \log(n))$$

★★★★ **Das FAZIT (wörtlich!)**

- Für n Elemente gibt es $n!$ **Permutationen** und **genau eine davon wendet der Sortieralgorithmus an**.
- Die Entscheidung, welche anzuwenden ist, braucht im schlimmsten Fall **mindestens** $\log(n!) = \Theta(n \log n)$ **Vergleiche**.
-  Best-Case-Zeiten von z. B. $O(n)$ sind **Sonderfälle**, die in dieser Betrachtung **nicht berücksichtigt** werden!
-  Gute vergleichsbasierte Sortierv Verfahren haben eine asymptotische Laufzeitkomplexität von $O(n \log n)$ und **liegen somit nur einen Faktor von der unteren Schranke entfernt**.
-  **Es kann also KEIN BESSERES Sortierv Verfahren geben, das auf dem VERGLEICH VON SCHLÜSSELWERTEN basiert.**

D.14 ★★★★★ SORTIEREN IN LINEARER ZEIT

? „Was wäre, wenn das Sortierverfahren **gar keine Vergleiche** benötigt? Gibt es Sortierverfahren, die nicht vergleichend arbeiten?“ Im Allgemeinen **NICHT**, da man für die zu sortierenden Objekte lediglich die **Ordnungsrelation** (Kleiner-Gleich-Beziehung) kennt. ★ **Bessere Sortierverfahren mit $O(n)$ sind möglich — ALLERDINGS NUR, WENN SPEZIELLE RAHMENBEDINGUNGEN GEGEBEN SIND.**

★★ Die 4 Annahmen und die zugehörigen Verfahren

#	Annahme	Verfahren
1	Die zu sortierenden Schlüssel sind ganzzahlig, PAARWEISE VERSCHIEDEN und aus $\{1, \dots, n\}$	☞ Weirdsort
2	Die Elemente $A[i]$ sind in einem Intervall (fester Wertebereich) GLEICHVERTEILT	☞ Bucketsort (funktioniert auch mit reellen Zahlen!)
3	Die Sortierung erfolgt nach einem m-adischen Schlüssel $A[i] = (k_1, k_2, \dots, k_d)$, $\forall 1 \leq j \leq d, k_j$ ganzzahlig	☞ Radixsort
4	Die Schlüssel sind ganzzahlig und NICHT paarweise verschieden aus $\{0, \dots, k\}$ mit $k = O(n)$	☞ Countingsort

(Weirdsort, Radixsort und Countingsort funktionieren mit **diskreten Schlüsseln** (natürliche Zahlen).)

WEIRD SORT

```
WEIRD SORT(A)
  for (i = 1 to n)
    B[A[i]] = A[i]           // Schreibe die 5 an Position 5,
  return B                  // die 7 an Position 7 usw.
```

Schlüssel = Index des Elements $\rightarrow O(n)$. ⚠ „Kein echtes Sortierproblem.“

★★ BUCKETSORT (Eimersortierung)

Platzkomplexität	OUT-OF-PLACE
Stabilität	abhängig vom untergeordneten Sortierverfahren

Idee: Vorsortieren in **Buckets**, dann Sortieren kleinerer Mengen **innerhalb** der Buckets (mit einem beliebigen Sortieralgorithmus). Ist dem Prinzip des **Divide and Conquer** sehr ähnlich.

- **Teilen:** Elemente aus dem Feld herauslösen und **einem Wertigkeitsbereich zuordnen**
- **Herrschen:** **jeden einzelnen Wertigkeitsbereich separat analysieren**

Vorgehen: ① Elemente entsprechend ihrer Wertigkeit auf Buckets verteilen · ② auf jeden Bucket ein Sortierverfahren anwenden · ③ die sortierten Bucketinhalte **konkatenieren**

```

Bucketsort(A)
  for (i = 1 to A.length)
    List_Insert( B[ floor(A.length * A[i]) ], A[i] )    // Vorsortierung O(n)
  i = 1
  for (j = 0 to A.length-1)
    Insertionsort(B[j])
    while not EMPTY(B[j])
      A[i] = List-Head(B[j]); B[j] = List-Tail(B[j]); i = i + 1

```

Fall	Komplexität	Bedingung
Vorsortierung	$O(n)$	
Best Case	$O(n)$	einer pro Bucket
Worst Case	$O(n^2)$	⚠ alles in einem Bucket (mit Insertionsort) — wenn keine Gleichverteilung vorliegt

⚠ **Probleme:** Wie viele Buckets? Welche Wertebereiche? — Bei Zahlen schwierig (z. B. 2^{64}), bei Buchstaben vorhersagbar.

Zahlenreihe	# Buckets	# Durchläufe
0 bis 99	100	1
0 bis 999	1000	1

⚠ *, „Bucketsort hat den Nachteil, dass **mehrere Durchläufe notwendig sind, wenn nicht genügend Buckets** zur Verfügung stehen. **Die Laufzeit ist nicht exakt abschätzbar, da sie von der Datenverteilung abhängt.**“*

★ Erwartungswertanalyse (bei Gleichverteilung):

$$E[T_{\text{Bucket}}(n)] = cn + \sum_{i=0}^{n-1} cE[n_i^2] \quad \text{mit} \quad E[n_i^2] = 2 - \frac{1}{n}$$

$$\Rightarrow E[T] = cn + \sum_{i=0}^{n-1} c \left(2 - \frac{1}{n}\right) = cn \left(3 - \frac{1}{n}\right) = O(n)$$

(Basierend auf: $P(X_{ij}) = \frac{1}{n}$ bei Gleichverteilung; $E[X_{ij}^2] = 1 \cdot \frac{1}{n} + 0 \cdot \left(1 - \frac{1}{n}\right) = \frac{1}{n}$.)

★★ **RADIXSORT (Wurzel- oder Basissortierung)**

Platzkomplexität	In-Place mit Listen · Out-of-Place mit Arrays
Stabilität	✅ STABIL

Idee: Zahlen werden **schrittweise** sortiert. **Für jede STELLE in der Zahl wird ein Durchlauf von Bucketsort durchgeführt** — die Elemente werden in **lexikografischer Ordnung** betrachtet.

Zwei Phasen (je Stelle):

Phase	Beschreibung
① Partitionierungsphase	Die Elemente werden in vorhandene Buckets aufgeteilt — abhängig davon, welche Stelle in der Ziffer betrachtet wird
② Sammelphase	Einsammeln der Elemente: Elemente aus dem ersten Bucket zuerst , dann der nächste usw.

★ **Richtung:** Es wird mit der **NIEDERWERTIGSTEN Stelle (erste Stelle von RECHTS)** begonnen. In der letzten Iteration wird die **erste Stelle von links** zum Aufteilen verwendet.

Vorgehen:

1. Sofern eine Ziffer nicht genügend Stellen besitzt, wird sie **mit Nullen aufgefüllt**
2. Ziffern entsprechend der Wertigkeit der zu analysierenden Stelle in ein Bucket einsortieren
3. Buckets der Reihe nach wieder leeren
4. Wenn noch nicht alle Stellen bearbeitet: weiter mit Schritt 2

Regeln: *Befindet sich bereits eine Ziffer in einem Bucket, wird die einzusortierende Ziffer **UNTERHALB** der bereits enthaltenen geschrieben. Beim Herausholen werden die Elemente **von OBEN nach UNTEN** entnommen.*
 ← **das erzeugt die Stabilität!**

 **Beispiel „328“:** Durchlauf 1 → Bucket 8 · Durchlauf 2 → Bucket 2 · Durchlauf 3 → Bucket 3

 **Durchgerechnetes Beispiel:** 5, 122, 333, 51, 845, 15, 0, 266, 444, 87 → Anpassen: 005 122 333 051 845 015 000 266 444 087 → 1. Stelle von rechts, dann 2., dann 3. → **sortiert**

Laufzeit:

Pro Runde: n Schritte zum Partitionieren + n Schritte zum Sammeln = $2n \approx n$
 m Runden: $2n + 2n + \dots + 2n = m \cdot 2n \approx m \cdot n$

$$O(n \cdot m) \quad m = \text{Anzahl der Stellen}$$

★★ COUNTINGSORT (Sortieren durch Zählen)

Platzkomplexität	OUT-OF-PLACE
Stabilität	★ STABIL — aber nur unter einer bestimmten Voraussetzung!

Idee:

1. **Für jedes vorkommende Element wird dessen ANZAHL (Vorkommen) separat in einem ZÄHLER-FELD gespeichert** — das Zähler-Feld enthält dann die **Anzahl der Elemente, deren Schlüssel gleich i sind**
2. **Anschließend werden die benachbarten Werte im Zähler-Feld AKKUMULIERT**, sodass das Zähler-Feld die **Anzahl der Elemente enthält, deren Schlüssel $\leq i$ sind**
3. ★ **Aus dem Zähler-Feld lässt sich DIREKT die ENDPOSITION des Elements bestimmen**

```
Countingsort(A, B, k)           // B: sortierte Ausgabe, C: Zähler-Feld [0..k]
  for (i = 0 to k)               C[i] = 0                               // 0(k)
  for (j = 1 to A.length)       C[A[j]] = C[A[j]] + 1                 // 0(n)  ZÄHLEN
  for (i = 1 to k)               C[i] = C[i] + C[i-1]                 // 0(k)  AKKUMULIEREN
  for (j = A.length downto 1)   // 0(n)  ★ RÜCKWÄRTS!
    B[ C[A[j]] ] = A[j]
    C[A[j]] = C[A[j]] - 1
  return B
```

$$T(n, k) = O(n + k) \quad \text{— für } k = O(n) \text{ ist die Sortierung in LINEARZEIT}$$

 „Die Laufzeit ist **nicht nur abhängig von der Menge der Elemente, sondern auch davon, wie viele VERSCHIEDENE SCHLÜSSELWERTE vorliegen.**“

!★ Das Stabilitätsproblem von Countingsort

? Wann läuft der Algorithmus stabil? ★ „Countingsort läuft NUR STABIL, wenn die Schleife RÜCKWÄRTS durchlaufen wird. Ansonsten wird die Ordnung der Elemente UMGEDREHT.“

Weitere Sortiervverfahren (nur genannt): Binary Tree Sort, Gnomesort, Smoothsort

D.15 ★★ Die Laufzeitmessungen von Prof. Eggendorfer (echte Sekunden!)

Elemente	Insertionsort	Bubblesort	Quicksort	Mergesort	Heapsort
100	0,000017	0,000036	0,000012	0,000019	0,000014
1.000	0,001037	0,003419	0,000128	0,002019	0,000202
10.000	0,106248	0,326836	0,001679	0,018642	0,002302
100.000	10,10	33,13	0,016473	—	0,028058
1.000.000	1.020,88	3.167,04	★ 0,2456	0,2100	0,3196

🔑 Der Praxisbeweis: Bei 1 Million Elementen braucht Bubblesort ~53 MINUTEN, Quicksort ~0,25 SEKUNDEN — Faktor ~12.900!

D.16 ★★★★★ DIE GROSSE SORTIER-VERGLEICHSTABELLE

Verfahren	Best	Average	Worst	Platz	Stabil?	Merkmal
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	in-place	✗	optimal bzgl. Datenaustausch ($O(n)$ Swaps)
Insertion sort	★ $O(n)$	$O(n^2)$	$O(n^2)$	in-place	✓	gut bei nahezu sortierter Eingabe
Bubblesort	★ $O(n)^*$	$O(n^2)$	$O(n^2)$	in-place	✓	⚠️ *nicht empfehlenswert · *nur mit noSwap
Shakersort	$O(n)$	$O(n^2)$	$O(n^2)$	in-place	✓	bidirektional
Quicksort	$O(n \log n)$	$O(n \log n)$	⚠️ $O(n^2)$	in-place	✗	schnellster in der Praxis; Degeneration bei sortierter Eingabe
Mergesort	$O(n \log n)$	$O(n \log n)$	★ $O(n \log n)$	⚠️ out-of-place	✓	parallelisierbar, gut für große Datenbestände
Heapsort	$O(n \log n)$	$O(n \log n)$	★ $O(n \log n)$	★ in-place	✗	in-place UND $O(n \log n)$ garantiert
Shellsort	—	—	$O(n^2) / O(n^{1.5}) / O(n \log^2 n)$	in-place	✗	abhängig von der h -Folge
Combsort	—	—	$O(n^2) \dots O(n \log n)$	in-place	✗	Bubblesort + Shellsort
Bucket sort	★ $O(n)$	$O(n)$	⚠️ $O(n^2)$	out-of-place	(abhängig)	Gleichverteilung nötig; auch reelle Zahlen
Radixsort	$O(n \cdot m)$	$O(n \cdot m)$	$O(n \cdot m)$	in-place (Listen)	✓	m = Anzahl Stellen
Counting sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	out-of-place	✓*	*nur bei rückwärts durchlaufener Schleife
Bogosort 😬	$O(n)$	—	$O(n \cdot n!)$	in-place	✗	Las-Vegas-Algorithmus

▼ ? Selbsttest Teil D

1. Nennen Sie die 3 typischen Sortier-Operationen.
2. Was bedeutet Stabilität? In-Place vs. Out-of-Place?
3. Beschreiben Sie die Idee von Selectionsort. Warum ist es bzgl. Datenaustausch optimal?
4. Warum hat Insertionsort eine Fallunterscheidung, Selectionsort aber nicht?
5. Welche Regel gilt für While-Schleifen und Fallunterscheidungen?
6. Wie verbessert man Bubblesort? Was ändert sich am Best Case?
7. Definieren Sie Divide-and-Conquer.
8. Was ist der Unterschied zwischen Quicksort und Mergesort bzgl. Teilen und Rekombination?
9. Was ist Degeneration bei Quicksort? Wann tritt sie auf?
10. Nennen Sie die 6 Möglichkeiten zur Pivotwahl mit Bewertung.
11. Wie funktioniert der SELECT-Algorithmus (Median der Fünfer-Mediane)?
12. Warum wird Mergesort bei Aufteilung in k Teile nicht schneller?
13. Was ist der Speicherbedarf von Mergesort?
14. Definieren Sie MAX-Heap und MIN-Heap.
15. Wie berechnet man linkes/rechtes Kind und Elter im Heap-Array?
16. Bei welchem Index startet BUILD-MAX-HEAP und warum?
17. Nennen Sie die 3 Heap-Operationen mit ihren Komplexitäten.
18. Nennen Sie die 3 Shellsort-h-Folgen mit Komplexität.
19. Welche Komplexität wurde für Shellsort ausgeschlossen?
20. Was ist der Shrink-Faktor bei Combsort?
21. Schreiben Sie das Theorem zur unteren Schranke auf und begründen Sie es.
22. Nennen Sie die 4 Annahmen für lineares Sortieren mit den zugehörigen Verfahren.
23. Warum ist Bucketsort im Worst Case $O(n^2)$?
24. Erklären Sie Radixsort in 2 Phasen. Warum von rechts nach links?
25. Beschreiben Sie Countingsort in 3 Schritten. Wann ist es stabil?
26. Sortieren Sie `[5, 2, 8, 1, 9]` mit Selectionsort, Insertionsort, Bubblesort (Schritt für Schritt).

TEIL E — KAPITEL 05: SUCHALGORITHMEN

Die 7 Verfahren: Lineare Suche · Bisektionssuche (binäre Suche) · Exponentielle Suche · Interpolationssuche · Quadratische Binärsuche · Binärer Suchbaum · Hashing-basierte Suche (→ Kapitel 06)

E.1 ★ Warum überhaupt suchen?

1. **Dokumentenanalyse** — Stichwortverzeichnis; Suche nach allen im Text vorkommenden Bezeichnern oder Namen
2. **Compilerbau** — Identifikationsschlüssel

Wonach sucht man? Merkmale · Attribute · Schlüssel (Key)

Die 3 elementaren Suchverfahren: Sequentielle Suche · Binäre Suche · Hashing-basierte Suche

E.2 ★ LINEARE SUCHE (Sequentielle Suche)

Vorgehen: Elemente als **lineare Anordnung** betrachtet; **Durchsuchung erfolgt elementweise von vorne nach hinten.**

```

LINEAR-SEARCH(A, q)
  for (i = 1 to A.length)
    if (A[i] == q) return i
  return -1

```

✓ Vorteile

Einfachstes Verfahren · auf JEDE Art von Datenstruktur anwendbar

✗ Nachteile

Ineffizient

Fall	Komplexität	Begründung
Best Case	$O(n) = 1$	Erstes Element = gesuchtes Element (oder nur ein Element)
Worst Case	$O(k)$ falls Element vorkommt, sonst $O(n) = n$	Letztes Element = gesuchtes Element oder Element nicht enthalten
Average Case	$O(n) = \frac{1}{n} \sum_{i=1}^n i = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2} \approx \left\lceil \frac{n}{2} \right\rceil$	Alle Elemente haben unterschiedliche Schlüssel; nach jedem Element wird genau einmal und mit gleicher Wahrscheinlichkeit gesucht

E.3 ★★ BINÄRE SUCHE (Bisektionssuche)

Vorgehen: Durchsuchung erfolgt durch **ständige HALBIERUNG** des zu betrachtenden Feldes. 🔑 **Prinzip: Divide & Conquer**

```

BINARY-SEARCH(A, key)
  left = 1; right = n
  while (key == NIL)
    if (right < left) return 0 // key existiert nicht
    middle = left + (right - left) / 2
    if (A[middle] < key) left = middle + 1
    if (A[middle] > key) right = middle - 1
    if (A[middle] == key) return A[middle]

```

★★ Die 2 VORAUSSETZUNGEN (auswendig!)

1. ★ Die Elemente müssen bereits „SORTIERT“ angeordnet sein (sortierte Liste als Basis)
2. ★ WAHLFREIER ZUGRIFF auf das „mittlere“ Element muss möglich sein

Vorteile: Effizientes Suchen eines bestimmten Knotens · **schnelles Löschen/Einfügen**

$$T(n) = T\left(\frac{n}{2}\right) + c = O(\log(n))$$

⚠️ ★ Das (kleine) Problem

⚠️ „Die binäre Suche ist NICHT IMMER SCHNELLER als die lineare Suche.“ Wann? Wenn das Element weiter VORNE im Array steht — die Position von q wäre kleiner als $\log(n) \rightarrow O(k)$ wäre besser als $O(\log(n))$.

E.4 ★ EXPONENTIELLE SUCHE

Idee: Binäre Suche **nicht auf dem GANZEN Array** — zuerst **Suche des Bereichs/Intervalls**, auf dem die binäre Suche ausgeführt werden soll.

Vorgehen:

1. Verdopple den rechten Rand, bis $A[\text{right}] > q$ gilt
2. Führe dann die binäre Suche im Intervall $[\frac{r}{2}, \dots, r]$ durch

```
EXPONENTIAL-SEARCH(A, q)
    right = 1
    while (A[right] < q and right < n)           // Suche das Intervall
        right = min(2*right, n)
    return BINARY-SEARCH(A, right/2 + 1, right, q) // Suche binär im Intervall
```

Fall	Komplexität
1. Fall: $\exists k : A[k] = q$ (q kommt vor)	while-Schleife: $O(\log(k)) \cdot \text{binäre Suche im Intervall } \frac{r}{2} : O(\log(r-1)) \approx O(\log k) \rightarrow \text{Gesamt } O(\log(k))$
2. Fall: $\forall k : A[k] \neq q$ (q kommt nicht vor)	Worst-Case $O(\log(n))$

E.5 ★★ INTERPOLATIONSSUCHE

Idee: Verbesserung der binären Suche — „**ERRATEN**“ der **Position**, wo sich der gesuchte Schlüssel im betrachteten Intervall befinden könnte. Bei der binären Suche wurde das mittlere Element bestimmt: $m = \frac{\text{left} + \text{right}}{2}$. **Der Faktor 0,5 wird nun durch eine SCHÄTZUNG ersetzt.**

★★ Die Formel (auswendig!)

$$m = \text{left} + \frac{\text{key} - A[\text{left}]}{A[\text{right}] - A[\text{left}]} \cdot (\text{right} - \text{left})$$

Vorgehen: ① Vergleichsposition berechnen · ② $A[m] = \text{key}$? **JA** → gefunden; **NEIN** → $\text{left}_{\text{neu}} = m + 1$ (bzw. $\text{right}_{\text{neu}} = m - 1$), zurück zu ①

 **Das Beispiel des Dozenten**

Array: 12 · 14 · 24 · 32 · 44 · 57 · 73 · 81 · 92, gesucht: **key = 32**

Iteration 1: left = 1, right = 9, $A[\text{left}] = 12$, $A[\text{right}] = 92$

$$m = 1 + \frac{32 - 12}{92 - 12} \cdot (9 - 1) = 1 + \frac{20}{80} \cdot 8 = 1 + 2 = 3$$

$A[3] = 24 \neq 32 \rightarrow \text{left}_{\text{neu}} = m + 1 = 4$

Iteration 2: left = 4, $A[\text{left}] = 32$

$$m = 4 + \frac{32 - 32}{92 - 32} \cdot (9 - 4) = 4 + 0 = 4$$

$A[4] = 32 = 32$ ✓ **gefunden!**

Fall	Komplexität
Best Case	$O(\log(\log(n)))$ (ohne Beweis)
Worst Case	⚠ $O(n)$ — die Position von m befindet sich immer am Rand

✓ Vorteile	✗ Nachteile
I. d. R. weniger Vergleiche nötig als bei der binären Suche	⚠ Setzt GLEICHMÄSSIG VERTEILTE Schlüsselwerte voraus · Berechnung von m ist komplexer als bei der binären Suche

E.6 ★ QUADRATISCHE BINÄRSUCHE

Idee: Kombination der Vorteile der „normalen“ binären Suche mit den Vorteilen der Interpolationssuche — **Reduzierung des Intervalls in jedem Rekursionsschritt auf ein Intervall der Länge $\sqrt{n}$.**

Anders ausgedrückt: Nach jedem Vergleich wird geprüft, ob der Wert größer oder kleiner als `key` ist. Dementsprechend wird **nach links oder rechts um $\sqrt{n}$ Felder gesprungen**, bis ein kleinerer/größerer Wert gefunden wird. **In dem eingegrenzten Intervall wird anschließend nach `key` gesucht.**

Fall	Aktion
Fall 1: $A[\text{key}] = \text{key}$	gefunden
Fall 2: $A[\text{key}] > \text{key}$	Überprüfe $A[\text{key} - \sqrt{n}]$, $A[\text{key} - 2\sqrt{n}]$, ...
Fall 3: $A[\text{key}] < \text{key}$	Überprüfe $A[\text{key} + \sqrt{n}]$, $A[\text{key} + 2\sqrt{n}]$, ...

mit $n = \text{right} - \text{left} + 1$

Worst Case: $O(\sqrt{n})$

E.7 ★★★★★ BINÄRER SUCHBAUM

★★ Die zwei Definitionen

Definition I: Ein **binärer Suchbaum (binary search tree)** ist ein **Binärbaum**, dessen Knoten so angeordnet sind, dass die Werte **aller LINKEN Nachfolger KLEINER** und die **aller RECHTEN Nachfolger GRÖßER ODER GLEICH** dem Wert des betrachteten Knotens sind.

Definition II: Die **Wurzel** enthält einen Wert, der **größer ist als alle Werte im linken Teilbaum**, und der Wert in der Wurzel ist **kleiner oder gleich allen Werten im rechten Teilbaum**. **D. h. der Baum muss GEORDNET sein!**

$$\text{Linkes Kind} < \text{Eltern-Knoten} \leq \text{Rechtes Kind}$$

Für jeden Knoten x gilt:

- Sei y ein Knoten im **linken** Teilbaum von $x \implies y.\text{key} < x.\text{key}$
- Sei y ein Knoten im **rechten** Teilbaum von $x \implies y.\text{key} \geq x.\text{key}$

★ „Trifft diese Eigenschaft **nicht nur für die Wurzel, sondern auf die Wurzeln ALLER TEILBÄUME** zu, dann ist der Baum ein binärer Suchbaum!“

★★ Die CATALANSCHES ZAHLE (Eugène C. Catalan, belgischer Mathematiker)

Anzahl der verschiedenen Kombinationsmöglichkeiten (Ausprägungen) eines binären Suchbaums für n Elemente:

$$C(n) = \frac{1}{n+1} \binom{2n}{n}$$

 **Beispiel:** Binärer Suchbaum für die **ersten vier Primzahlen**:

$$C(4) = \frac{1}{5} \binom{8}{4} = \frac{1}{5} \cdot 70 = 14$$

★ Die 5 Funktionen eines binären Suchbaums

Finden eines Knotens · Finden des Minimums · Finden des Nachfolgers eines Knotens · Einfügen von Knoten · Löschen von Knoten

```

TREE-SEARCH(x, k)
    if (x == NIL or k == x.key)
        return x
    if (k < x.key)
        return TREE-SEARCH(x.left, k)
    else
        return TREE-SEARCH(x.right, k)
// Initialer Aufruf: TREE-SEARCH(root[T], 42)

TREE-MINIMUM(x)
    while (x.left != NIL)
        x = x.left
    return x

ITERATIVE-TREE-SEARCH(x, k)
    while (x != NIL and k != x.key)
        if (k < x.key) x = x.left
        else x = x.right
    return x

TREE-INSERT(T, z)
    y = NIL; x = T.root
    while (x != NIL)
        y = x
        if (z.key < x.key) x = x.left
        else x = x.right
    z.parent = y
    if (y == NIL) T.root = z
    elseif (z.key < y.key) y.left = z
    else y.right = z

```

★★ TREE-SUCCESSOR — das nächstgrößere Element

```

TREE-SUCCESSOR(x) // Finde y mit y.key > x.key und y.key minimal
    if (x.right != NIL) // Fall 1: rechter Nachfolger existiert
        return TREE-MINIMUM(x.right)
    else // Fall 2: rechter Nachfolger existiert nicht
        y = x.parent
        while (y != NIL and x == y.right)
            x = y; y = y.parent
        return y

```

Für jeden Knoten x durchläuft der Algorithmus entweder den **Pfad von x bis höchstens zu einem Blatt** oder **zur Wurzel**.

★★★★ LÖSCHEN von Knoten — die 3 Fälle (auswendig!)

Fall	Situation	Aktion
Fall 1	z ist ein BLATT	Lösche z
Fall 2	z hat NUR EINEN Nachfolger x	x wird Nachfolger von $z.parent$
Fall 3	z hat ZWEI Nachfolger	★ Ersetze z durch <code>TREE-SUCCESSOR(z)</code>

★★ Laufzeitverhalten

Fall	Komplexität	Begründung
Best Case	$O(n) = 1$	Erstes Element = gesuchtes Element (Wurzelknoten)
Worst Case	$O(\log n)$ bei ausgeglichenem Baum	$n = 2^{h+1} - 1 \Rightarrow \lg(n+1) = h+1 \Rightarrow h = \lg(n) - 1; O(n) = \lceil \lg(n) \rceil$
⚠ Worst Case bei ENTARTUNG	$O(n)$	„Haben wir etwas vergessen bzw. übersehen?“ — der degenerierte Baum!

★★★★ BALANCIEREN / ROTATION

★ „Um für binäre Suchbäume auch im schlechtesten Fall eine schnelle Laufzeit zu erhalten, muss die **HÖHE** des Suchbaums **MÖGLICHST KLEIN** gehalten werden. Man muss dafür sorgen, dass ein Baum bei einer ungünstigen Einfügereihenfolge **NICHT ENTARTET**.“ 💡 **Idee:** Man muss **bereits bei jeder Einfüge- oder Löschoperation** versuchen, den Baum auszugleichen. Das **Ausgleichen (Balancieren)** erfolgt durch die **Vertauschung von Knoten** — diese Vertauschungen nennt man **ROTATIONEN**.

★★★★ Die 4 Fälle und ihre Rotationstypen (auswendig!)

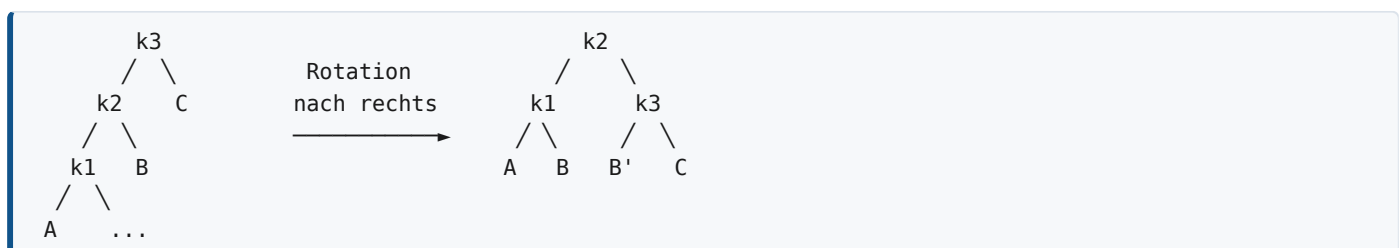
Wie identifiziert man den Rotationstyp?

1. Identifiziere den Knoten, der das **UNGLEICHGEWICHT** verursacht
2. Finde das zugehörige **KIND** dieses Knotens
3. Finde den zu diesem Kind gehörigen darunterliegenden Baum (Subtree)

Fall	Einfügen ...	★ Rotationstyp
1	im LINKEN Teilbaum des LINKEN Kindes	Einfache Rotation (RECHTS)
2	im RECHTEN Teilbaum des LINKEN Kindes	DOPPELROTATION (links-rechts)
3	im RECHTEN Teilbaum des RECHTEN Kindes	Einfache Rotation (LINKS)
4	im LINKEN Teilbaum des RECHTEN Kindes	DOPPELROTATION (rechts-links)

🔑 „Die Fälle 1 und 3 bzw. 2 und 4 sind **SYMMETRISCH** — die Durchführung erfolgt nur spiegelverkehrt. $\Rightarrow$ Beschränkung auf die Fälle 1 und 2.“

📝 **Fall 1 — Einfache Rotation:** Rotation von k_3 über k_2



Beispiel des Dozenten: Rotation von **83** über **62**

📝 **Fall 2 — Doppelrotation:** ① Rotation von k_1 über k_2 (nach links) · ② Rotation von k_3 über k_1 (nach rechts)

Beispiel des Dozenten: ① Rotation von **71** über **62** nach links · ② Rotation von **83** über **71**

Aufbau eines geordneten Baumes (das große Beispiel)

Einfügereihenfolge: 19 – 7 – 23 – 9 – 25 – 11 – 22 – 12 – 42 – 31 – 18 – 29 – 2 – 15 – 20

19 = Wurzel · 7 < 19 → links · 23 > 19 → rechts · 9: 7 < 9 < 19 → links und rechts von 7
 25: 19 < 23 < 25 → rechts und rechts von 23 · 11: 9 < 11 → links und rechts von 9
 22: 19 < 22 < 23 → rechts und links von 23 · 12: 11 < 12 → links und rechts von 11
 42: 25 < 42 → rechts und rechts von 25 · 31: 25 < 31 < 42 → ...
 18: 12 < 18 → ... · 29: 29 < 31 → links und links von 31 · 2: 2 < 7 → links und links von 7
 15: 12 < 15 → links und rechts von 12 · 20: 20 < 22 → rechts und links von 22

In-Order-Durchlauf des Ergebnisses: 2 – 7 – 9 – 11 – 12 – 15 – 18 – 20 – 22 – 23 – 25 – 29 – 31 – 33 – 42

★★ Der binäre Suchbaum zum SORTIEREN

★★ „Erfüllt ein Baum die binäre Suchbaumeigenschaft, dann werden die Objekte bei einem IN-ORDER-DURCHLAUF in AUFSTEIGENDER Reihenfolge ausgegeben.“

Beispiel: Suche 17

20 > 17 → nach links
 11 < 17 → nach rechts
 15 < 17 → nach rechts
 18 > 17 → 17 NICHT ENTHALTEN

E.8 ★★★ DIE SUCH-VERGLEICHSTABELLE

Verfahren	Best	Average	Worst	Voraussetzung
Lineare Suche	$O(1)$	$O(n/2)$	$O(n)$	keine — auf jede Datenstruktur anwendbar
Binäre Suche	$O(1)$	$O(\log n)$	$O(\log n)$	★ SORTIERT + wahlfreier Zugriff
Exponentielle Suche	$O(1)$	$O(\log k)$	$O(\log n)$	sortiert; gut, wenn das Element weit vorne liegt
Interpolationssuche	$O(\log \log n)$	$O(\log \log n)$	⚠ $O(n)$	sortiert UND GLEICHVERTEILT
Quadratische Binärsuche	—	—	$O(\sqrt{n})$	sortiert
Binärer Suchbaum	$O(1)$	$O(\log n)$	⚠ $O(n)$ (entartet) / $O(\log n)$ (AVL)	geordneter Baum
Hashing (Kap. 06)	$O(1)$	★ $O(1)$	$O(n)$	gute Hashfunktion

? **Der Überleitungssatz zu Kapitel 06:** „Gibt es ein noch effizienteres Verfahren, das **unabhängig von der Anzahl der Elemente** ist und **annähernd mit einer Komplexität von $O(1)$** für Einfügen, Löschen und Suchen arbeitet?“ — **Mittels hashing-basierter Suche.**

▼ ? Selbsttest Teil E

1. Nennen Sie die 3 elementaren Suchverfahren.
2. Best/Average/Worst Case der linearen Suche mit Begründung?
3. Nennen Sie die 2 Voraussetzungen der binären Suche.
4. Wann ist die lineare Suche schneller als die binäre?
5. Wie funktioniert die exponentielle Suche in 2 Schritten?
6. Schreiben Sie die Interpolationsformel auf und erklären Sie sie.
7. Warum ist die Interpolationssuche im Worst Case $O(n)$?
8. Wie funktioniert die quadratische Binärsuche? Welche Komplexität?
9. Definieren Sie den binären Suchbaum. Wie lautet die Merkformel?
10. Wie viele verschiedene Suchbäume gibt es für 4 Elemente? Für 5? (14 bzw. 42)
11. Nennen Sie die 3 Fälle beim Löschen eines Knotens.
12. Was macht TREE-SUCCESSOR in den beiden Fällen?
13. Warum muss man Suchbäume balancieren?
14. Nennen Sie die 4 Rotationsfälle mit dem jeweiligen Rotationstyp.
15. Welche Fälle sind symmetrisch?
16. Welcher Durchlauf sortiert einen binären Suchbaum?
17. Bauen Sie einen Suchbaum aus 50, 30, 70, 20, 40, 60, 80 und geben Sie den In-Order-Durchlauf an.

TEIL F — KAPITEL 06: HASHING ★★

Inhalte: Grundprinzip · Transformation · Hashfunktionen · Arten von Hashing · Kollisionsbehandlung · Beispiel aus der Praxis

F.1 ★ Allgemein

⚠ Hashing ≠ Kryptographisches Hashing! 🔑 **Merke:** Hashing wird auch als **STREUSPEICHERUNG** (*scatter storage*) bezeichnet.

★ **DAS ZIEL:** Die Speicherung von Elementen mit einem **BEGRENZTEN PLATZBEDARF** $\Theta(|K|)$ unter Erhalt einer **NAHEZU KONSTANTEN LAUFZEIT** mit den Operationen **INSERT, DELETE und SEARCH** in $O(1)$.

Die Idee

- **Behälter als ein FELD (Array) von Datenobjekten** konzipieren
- **SUCHSCHLÜSSEL als FELDINDEX** verwenden (zur Speicherung **und** Suche)
- Die Elemente sind **nach den Werten ihrer Schlüsselwerte** im Feld angeordnet
- Auf das gesuchte Element kann **mit dem Schlüssel in einer Adresstabelle** zugegriffen werden

★ Die HASHTABELLE

Das Feld, in dem die Werte gespeichert werden, wird als **HASHTABELLE** (*hash table*) bezeichnet. Es wird **ZU BEGINN FESTGELEGT**, wie groß das Feld sein soll.

⚠️ Warum die naive Idee NICHT funktioniert

Schlüsselwerte als Indizes für ein Feld zu verwenden ist nicht ohne weiteres möglich, weil ...

1. die Schlüsselwerte als Datentyp **UNGEEIGNET** sind
2. der Wertebereich **ZU GROSS** ist, um ein Feld mit dieser Elementanzahl anzulegen ($|U| \gg |K|$)

★ Schlüsselwerte müssen daher in eine verwendbare Form **TRANSFORMIERT** werden, da eine direkte Speicheradressierung ineffizient ist!

📝 Das kleine Beispiel des Dozenten:

26 Buchstaben des Alphabets, Wörter mit einer Länge von bis zu **zehn** Buchstaben $\rightarrow 26^{10}$
Kombinationsmöglichkeiten. Annahme: 1 Byte pro Element $\Rightarrow$ **128 TB!**

Die konkreten Annahmen

1. Schlüssel sind aus der Menge $U = \{0, \dots, m-1\}$ (**Universalmenge**)
2. Die Schlüssel lassen sich auf natürliche Zahlen abbilden
3. Alle Schlüssel sind voneinander verschieden
4. $m = O(|K|)$, wobei K die Menge der **verwendeten** Schlüssel ist
5. Schlüsselwerte k sind nicht negativ und ganze Zahlen ($k \in \mathbb{N}$)
6. Der Zugriff erfolgt in konstanter Zeit

★ Das Grundprinzip (die Größenverhältnisse!)

- Die Anzahl der Objekte, die wir speichern wollen, ist **KLEINER** als der Indexbereich
- **ABER** die Anzahl der **MÖGLICHEN** Schlüssel ist viel **GRÖßER** als der Indexbereich selbst
- ★ D. h. der Indexraum ist viel größer als die zu speichernden Daten, was heißt, dass es **WESENTLICH WENIGER SPEICHER** als mögliche Schlüssel gibt.

$$\text{Universalmenge } U \gg \text{Indexbereich } I > \text{tatsächliche Schlüssel } K$$

F.2 ★★ Die 1. TRANSFORMATION: String $\rightarrow$ natürliche Zahl

Transformation von Schlüsselwerten vom Datentyp `string` in natürliche Zahlen. Auf die einzelnen Zeichen kann mittels Index zugegriffen werden; von jedem Zeichen kann mittels `Int` der **Zeichencode** ermittelt werden.

Symbol	Bedeutung
K	Zeichenkette
$KLen$	Länge der Zeichenkette
$K[n]$	Zugriff auf die einzelnen Zeichen
k	Schlüsselwert und eine natürliche Zahl

★ Die 3 Funktionen

① Funktion für LANGE Zeichenketten — Summe aller Zeichencodes:

$$k = \text{Int}(K[1]) + \text{Int}(K[2]) + \dots + \text{Int}(K[KLen])$$

② Funktion für KÜRZERE Zeichenketten — Summe aus dem einzigen bzw. den ersten zwei Zeichen sowie der Länge, mit den Faktoren 7 und 17:

$$\text{Für } KLen = 1 : \quad k = [\text{Int}(K[1]) \cdot 7 + 1] \cdot 17$$

$$\text{Für } KLen \geq 2 : \quad k = [\text{Int}(K[1]) \cdot 7 + \text{Int}(K[2]) + \text{Int}(KLen)] \cdot 17$$

③ hashCode der Java-Klasse `java.lang.String` — Zeichencodes werden der Reihe nach mit fallenden Potenzen von 31 multipliziert (→ breitere Streuung, vor allem bei kurzen Zeichenketten):

$$k = \text{Int}(K[1]) \cdot 31^{KLen-1} + \text{Int}(K[2]) \cdot 31^{KLen-2} + \dots + \text{Int}(K[KLen])$$

F.3 ★★ Die 2. TRANSFORMATION: Zahl → Index

Transformation des numerischen Wertes k in einen Wert hc des Indexbereiches $[0 \dots m - 1]$:

$$h : B \rightarrow \{0 \dots m - 1\}$$

$$h(k) = hc = \text{Hashwert/Indexwert des Schlüssels } k$$

Die Daten werden in einer Hashtabelle $T[0 \dots m - 1]$ gespeichert.

★ Definition Hashfunktion

Eine Hashfunktion (*hash function*) ermöglicht es, jedes Element der Schlüsselmenge K (Definitionsbereich) in eine NATÜRLICHE ZAHL abzubilden: $h : K \rightarrow \mathbb{N}$. Man erhält dadurch eine TEILMENGE von $\mathbb{N}$ (Bildbereich).

★ Das ZIEL einer Hashfunktion

1. **Berechne** aus dem Schlüssel einen Index $h(\text{key})$ und **speichere** die Daten an die Position $h(\text{key})$
2. **MINIMIERUNG** des Auftretens von Kollisionen — durch die Wahl einer geeigneten Hashfunktion
3. **Handhabung** von Kollisionen

⚠ „Die Wahl von Hashfunktionen ist nicht so einfach.“ (Diskrete Mathematik)

★★★★ Die 3 EIGENSCHAFTEN einer guten Hashfunktion (auswendig!)

#	Eigenschaft	Details
1	GLEICHMÄSSIGE VERTEILUNG der Schlüsselwerte über den gesamten Indexbereich $[0, \dots, m-1]$	surjektiv — auf jeden Index etwa $\frac{n}{m}$ Schlüssel
2	Abbildung soll möglichst ZUFÄLLIG sein	Ähnliche Schlüsselwerte sollen wertmäßig auf STARK UNTERSCHIEDLICHE Indizes abgebildet werden; keine Häufungen für erwartete Muster
3	EFFIZIENT berechenbar	Der Berechnungsaufwand darf nur vom SCHLÜSSEL und NICHT von n abhängen; Verwendung von elementaren Grundoperationen

🔑 **UNIFORMES HASHING:** „Die Hashfunktion sorgt dafür, dass die Schlüssel auf den Adressraum gleichverteilt und **UNABHÄNGIG** von anderen bereits festgelegten Hashwerten abgebildet werden.“

F.4 ★★★★★ DIE 4 TYPEN VON HASHFUNKTIONEN

① Divisions- oder Modulo-Methode

$$hc = k \bmod m$$

⚠️ **Problem:** m gerade + k gerade $\Rightarrow hc$ gerade $\rightarrow$ schlechte Streuung!

★ Gute Ergebnisse im Sinne der Gleichverteilung ergeben sich nur dann, wenn m ...

- eine **PRIMZAHL** ist
- **NICHT ZU NAHE** an einer **ZWEIERPOTENZ** liegt
- allgemein: **möglichst wenige Teiler** hat

Sonderfall $m = 2^p$: Wesentlich schneller zu berechnen als die Modulo-Operation (**Bitshift**). Ist die Schlüsselmenge K in U gleichverteilt, dann ergibt auch 2^p keine Häufung.

✓ Vorteile	✗ Nachteile
Einfaches Verfahren	⚠️ Ungünstig bzgl. STREUUNG — „ähnliche Schlüssel werden durch die Restbildung meist auf BENACHBARTE Indizes abgebildet“ · Berechnung ist stark von m abhängig

② Multiplikations-Methode

$$hc = \lfloor (k \cdot A \bmod 1) \cdot m \rfloor \quad \text{mit } 0$$

★ Die Wahl von m ist **UNKRITISCH** und **UNABHÄNGIG** von A — z. B. $m = 2^p$

★ Die optimale Wahl von A (**Fibonacci-Hashing**):

$$A = \frac{\sqrt{5} - 1}{2} = 0,618033980 \dots \quad \text{— der GOLDENE SCHNITT (Donald E. Knuth)}$$

Effiziente Implementierung: $A = \frac{s}{2^w}, \quad w = \text{Wortlänge des Computers}, \quad 0$

- **Optimale Ausnutzung der Binärarithmetik des Prozessors**
- **A , und damit auch die Hashfunktion, ist UNABHÄNGIG von der Tabellengröße**
- Das Produkt $k \cdot s$ ist $2w$ breit: $k \cdot s = r_1 \cdot 2^w + r_0$ mit $r_0, r_1 < 2^w$
- Es gilt: $r_1 = \lfloor k \cdot A \rfloor$ und $r_0 = (k \cdot A - \lfloor k \cdot A \rfloor) \cdot 2^w$
- ★ $h(k)$ besteht aus den $p = \log_2(m)$ **HÖCHSTWERTIGEN Bits von r_0**

 **Das durchgerechnete Beispiel:** $k = 123456$, $p = 14$, $m = 2^{14} = 16384$, $w = 32$

$$s = A \cdot 2^{32} = 2654435769$$

$$k \cdot s = 123456 \cdot 2654435769 = 327706022297664 = \underbrace{76300}_{r_1} \cdot 2^{32} + \underbrace{17612864}_{r_0}$$

$$\Rightarrow h(k) = 67 \quad (\text{die 14 höchstwertigen Bits von } r_0)$$

✓	×
Gute Streuung, Wahl von m unkritisch	Etwas „komplizierter“

③ Mittelquadrat-Methode

$$hc = (\text{mittlere Ziffern von}) k^2$$

 **Beispiel:**

k	k^2	hc
117	13.689	68
118	13.924	92
119	14.161	16

✓	×
Einfache Implementierung · aufeinanderfolgende Werte werden besser gestreut	⚠ Hoher Rechenaufwand · größere Kollisionshäufigkeit als andere Verfahren

⚠ „Für die Verwendung als Hash-Funktion eher **UNGEEIGNET**.“

④ ★★ Universelles Hashing

? „Wie kann man im Mittel eine gute Performanz erhalten? **Hashing mit einer FESTGELEGTEN Hashfunktion kann sich negativ auswirken.**“ ★ **Idee: ZUFÄLLIGE Auswahl einer Hashfunktion aus einer MENGE von Hashfunktionen (wie RAND-SELECT bei Quicksort!)**

Definition „universell“:

$$\forall k, l \in U : |\{h \in H \mid h(k) = h(l)\}| \leq \frac{|H|}{m}$$

Bei zufälliger Wahl einer Hashfunktion ist die Kollisionswahrscheinlichkeit:

$$P\{h(k) = h(l)\} = \frac{1}{m}$$

★ **Wie findet man so eine Menge?** Sei p eine Primzahl mit $p > m$ und $p > k \forall k \in U$, und $\mathbb{Z}_x = \{0, \dots, x-1\}$:

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m \quad H_{p,m} = \{h_{a,b} : a \in \mathbb{Z}_p \setminus \{0\}, b \in \mathbb{Z}_p\}$$

⚠ **Zur Qualität:** „Eine direkte Aussage ist nicht möglich, da man zur sinnvollen Auswahl einer Hashfunktion etwas über die **VERTEILUNG von K** wissen muss.“

F.5 ★★★★★ KOLLISIONEN

Definition

Es gibt (mindestens) zwei Schlüssel $k_1 \neq k_2$, für die gilt: $h(k_1) = h(k_2)$ — mehrere Schlüssel werden auf den GLEICHEN INDEX abgebildet. ⚠⚠ „In der Regel übersteigt die Schlüsselmenge K jene des Indexbereichs I bei weitem ($|K| \gg |I|$). Da $|U| > m$ ist, sind KOLLISIONEN UNVERMEIDBAR!“

★★★ Die Kollisionswahrscheinlichkeit (Geburtstagsparadoxon!)

Bei einer idealen Hashfunktion (Gleichverteilung von n Schlüsseln auf m Buckets, $\$n$

📝 **Geburtstagsparadoxon** ($m = 365$):

n	$P_{\text{Kollision}}$
22	47,5 %
23	★ 50,7 %
50	97 %

★★★ REHASHING

„Stellt man bei dem Versuch, ein Datenobjekt in der Hashtabelle zu speichern, fest, dass diese Position bereits belegt ist (KOLLISION), so wird versucht, das Objekt an einer ANDEREN FREIEN POSITION zu speichern. Den Vorgang, eine neue Einfügeposition zu finden, nennt man REHASHING.“

★★★★ Die 2 Strategien zur Kollisionsbehandlung

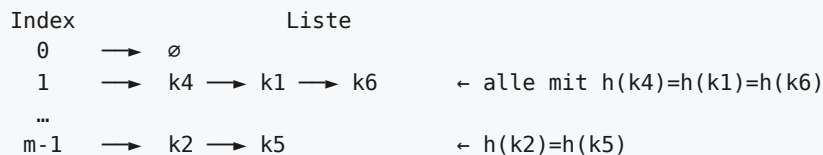
KOLLISIONSBEHANDLUNG

- └ ① VERKETTUNG (= OFFENES Hashing, separate chaining)
- └ ② OFFENE ADRESSIERUNG (= GESCHLOSSENES Hashing)
 - └ Lineare Sondierung
 - └ Quadratische Sondierung
 - └ Doppeltes Hashing

⚠⚠ **ACHTUNG, VERWECHSLUNGSGEFAHR:** „Geschlossenes Hashing = Offene Adressierung ≠ Offenes Hashing“!

F.6 ★★★★★ Strategie 1: VERKETTUNG (Offenes Hashing)

Speicherung ALLER Elemente mit gleichem Hashwert in einer LINEAREN LISTE. *Offenes Hashing (separate chaining) — die Liste ist für beliebig viele Einträge „OFFEN“. *



★ Operationen und Laufzeiten

Operation	Beschreibung	Komplexität
INSERT(T, x)	Füge x an den KOPF der Liste $T[h(x.key)]$ ein (ohne Test auf Vorhandensein)	$O(1)$
DELETE(T, x)	Entferne x aus der Liste (setzt doppelte Verkettung voraus)	$O(1)$
SEARCH(T, k)	Suche in der Liste $T[h(k)]$	$O(T[h(x.key)])$

★★★★ Der BELEGUNGSFAKTOR α (auswendig!)

$$\alpha = \frac{n}{m}$$

Symbol	Bedeutung
α	Füllfaktor, Auslastung, Belegungsfaktor (= durchschnittliche Listenlänge)
$n = K $	Anzahl der in der Hashtabelle gespeicherten Objekte
$m = T $	Größe der Hashtabelle (Anzahl der Speicherplätze/Einträge)

★ α beschreibt die **mittlere Anzahl an Objekten pro Eintrag (Liste)**. α ist **ENTSCHEIDEND**, wenn man die **Laufzeit analysiert** — die Laufzeit hängt von der **durchschnittlichen Anzahl von Objekten in den Listen** ab. "★"

★★ Die Laufzeitsätze

SATZ 1 (ERFOLGLOSE Suche): In einer Hashtabelle mit Verkettung, unter Annahme des einfachen, gleichmäßigen Hashings, benötigt die **erfolglose Suche** $O(1 + \alpha)$ **ERWARTETE Laufzeit**. — Bei erfolgloser Suche wird die Liste $T[h(k)]$ **vollständig** durchlaufen (erwartete Zeit $O(\alpha)$); die Berechnung von $h(k)$ erfolgt in $O(1)$.

SATZ 2 (ERFOLGREICHE Suche): ... benötigt die **erfolgreiche Suche ebenfalls** $O(1 + \alpha)$ **erwartete Laufzeit**. — Bei erfolgreicher Suche wird die Liste in erwarteter Zeit $O(\frac{\alpha}{2})$ durchlaufen:

$$E[T_{\text{SEARCH}}(n)] = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} \approx O(1 + \alpha)$$

★ „Befinden sich **WENIGER** Objekte in der Tabelle als Platz vorhanden ist, dann ist $\alpha < 1 \rightarrow$ **KONSTANTE ZEIT**." ⚠ „Es handelt sich um **KEINE Worst-Case-Laufzeit**, sondern um eine **ERWARTETE Laufzeit** (Erwartung im Mittel)."

Gute Hash-Funktion bei Verkettung: $s_{\text{avg}} = 1 + \frac{\alpha}{2} \approx 1 + \alpha$

Auslastung	Suchschritte
50 %	1,25
100 %	1,5

★ Vor- und Nachteile der Verkettung

✓ Vorteile	✗ Nachteile
★ $\alpha > 1$ MÖGLICH!	Zusätzlicher Speicherbedarf für Zeiger
Entfernung von Objekten direkt möglich	Komplexe Speicherverwaltung (Belegen und Befreien von Listen)
Kein Problem mit Überlauf	Ineffiziente Speichernutzung bei Häufung

📝 ★ BEISPIEL 1 — Offenes Hashing (die 12 Monate)

$$h(k) = (g(1. \text{ Buchstabe von } k) + g(2. \text{ Buchstabe von } k)) \bmod 17$$

wobei g einen Buchstaben auf seine Position im Alphabet abbildet: $a \rightarrow 1$, $b \rightarrow 2$, $c \rightarrow 3$ usw.

Monat	Rechnung	$h(k)$
Januar	$(10 + 1) \bmod 17$	11
Februar	$(6 + 5) \bmod 17$	11
März	$(13 + 1) \bmod 17$	14
April	$(1 + 16) \bmod 17$	0
Mai	$(13 + 1) \bmod 17$	14
Juni	$(10 + 21) \bmod 17$	14
Juli	$(10 + 21) \bmod 17$	14
August	$(1 + 21) \bmod 17$	5
September	$(19 + 5) \bmod 17$	7
Oktober	$(15 + 11) \bmod 17$	9
November	$(14 + 15) \bmod 17$	12
Dezember	$(4 + 5) \bmod 17$	9

(Die Werte stammen aus dem teilweise durch Overlays zerlegten Foliensatz — das **Rechenschema** ist der prüfungsrelevante Punkt: **Position 1. Buchstabe + Position 2. Buchstabe, dann mod 17.**)

F.7 ★★★★★ Strategie 2: OFFENE ADRESSIERUNG (Geschlossenes Hashing)

- EINMALIGES Anlegen von Speicher für die Hashtabelle
- Bei einer Kollision wird ein ANDERER PLATZ (offene Stelle) IN DER HASHTABELLE gesucht — die offene Stelle wird **neu berechnet**
- ★ Zeiger zur Verkettung ist NICHT MEHR NOTWENDIG

★ Die drei Begriffe

Begriff	Definition
Sondieren	Sukzessive Überprüfung der Hashtabelle nach freien Plätzen
Sondierungssequenz	Folge von Hashadressen, die für einen Schlüssel k nach offenen Stellen durchsucht werden (Reihenfolge, in der Speicherzellen überprüft werden)
Sondierungszahl	2. Parameter i , der die ANZAHL der Sondierungsversuche angibt

★★ Die Eigenschaften von Sondierungsfolgen

1. $\forall k \in U: h(k, i)$ ist eine **BIJEKTIVE** Funktion — **ALLE** m Speicherpositionen $(0, \dots, m-1)$ **müssen getestet werden**; die Sondierungsfolge **greift auf jedes Element nur EINMAL zu**
2. Die Sondierungssequenz hängt vom **SCHLÜSSEL** k ab

★★★ Die 3 Sondierungsverfahren

① LINEARE SONDIERUNG

$$h(k, i) = (h(k) + i) \bmod m \quad \text{bzw. allgemein} \quad h(k, i) = (h(k) + c \cdot i) \bmod m$$

Von $h(k)$ aus wird der jeweils nächstgrößere Platz getestet. Bei $c > 1$: c darf keinen gemeinsamen Teiler mit m haben.

⚠️ PROBLEM — PRIMÄRES CLUSTERING:

- Es entstehen **LANGE KETTEN** besetzter Plätze
- Je länger die Kette, desto größer ist die Wahrscheinlichkeit, diese auch zu **TREFFEN**
- Große Wahrscheinlichkeit, eine lange Kette zusätzlich noch zu **VERLÄNGERN**:

$$P\{\text{Kette der Länge } i \rightarrow \text{Kette der Länge } i+1\} = \frac{i+1}{m}$$

★ **PRIMÄRES CLUSTERING**: „Einträge ‚BALLEN‘ sich in einigen Bereichen der Hashtabelle.“

② QUADRATISCHE SONDIERUNG

$$h(k, i) = (h(k) + i^2) \bmod m \quad \text{bzw. allgemein} \quad h(k, i) = (h(k) + c_1 i + c_2 i^2) \bmod m, \quad c_2 \neq 0$$

⚠️ m, c_1 und c_2 müssen so gewählt werden, dass $h(k, i)$ bzgl. i **BIJEKTIV** ist.

✓ Vorteile	⚠️ Problem
Vermeidung des PRIMÄREN Clustering · Vermeidung von langen ZUSAMMENHÄNGENDEN Ketten	★ SEKUNDÄRES CLUSTERING

⚠️ Sekundäres Clustering:

1. Zwei Objekte (Schlüssel), die kollidieren, haben **IMMER DIE GLEICHE** Sondierungsfolge*
2. Die Sondierungsfolge ist **UNABHÄNGIG** von k^*
3. Es entstehen wieder lange Folgen, selbst wenn diese nicht hintereinander stehen → verteilte und zusammenhängende Kette — „Problemverlagerung“

* gilt auch beim linearen Sondieren

③ ★★★ DOPPELTES HASHING

$$h(k, i) = [h_1(k) + h_2(k) \cdot i] \bmod m$$

Funktion	Aufgabe
h_1	Bestimmung des STARTPUNKTES der Sondierungsfolge
h_2	Bestimmung des OFFSETS der Sondierungsfolge

⚠️ Bedingungen:

- $h(k, i)$ ist bzgl. i **NUR DANN BIJEKTIV**, wenn $h_2 \neq 0$ **UND** h_2 **TEILERFREMD** zu m ist
- h_1 und h_2 sollten bzgl. der Kollisionswahrscheinlichkeit **STOCHASTISCH UNABHÄNGIG** voneinander sein

F.8 ★★★★★ Das LÖSCHPROBLEM bei offener Adressierung

```

HASH-Insert(T, k)
  i = 0
  repeat
    j = h(k, i)
    if (T[j] == NIL)
      T[j] = k; return j
    else i = i + 1
  until (i == m)
  error "Hashtable Overflow!"

HASH-Search(T, k)
  i = 0
  repeat
    j = h(k, i)
    if (T[j] == k) return j
    i = i + 1
  until (i == m or T[j] == NIL)
  return NIL

```

? ⚠ „Was passiert, wenn ein Schlüssel k GELÖSCHT wird?" ⚠ „Schlüssel, die NACH k gespeichert wurden, können NICHT WIEDER GEFUNDEN werden, da die Repeat-Schleife bei $T[j] = \text{NIL}$ TERMINIERT." ⚠ „Beim Löschen kann man NICHT NIL schreiben, da dies die STRUKTUR ZERSTÖRT. Man benötigt eine ANDERE MARKIERUNG für die Löschung!"

★★★★★ Die DELETED-Markierung (die Lösung!)

Operation	Behandlung
Löschen	Setze beim Platz $T[i] = k$ das Flag $T[i] = \text{DELETED}$
Suchen	★ Behandle DELETED wie einen BELEGTEN Platz
Einfügen	★ Behandle DELETED wie einen FREIEN Platz

```

HASH-Insert(T, k) // Modifiziert
  if (T[j] is {NIL, DELETED}) T[j] = k; return j
HASH-Deleted(T, k)
  // Modifizierte Search-Funktion; dann: T[j] = DELETED

```

⚠ ★ **Die Konsequenz:** „Die Laufzeit bei der Suche hängt folglich **nicht mehr nur vom Belegungsfaktor** α ab (wie viele Felder belegt sind), **sondern auch davon, WIE VIELE FELDER DIE DELETED-MARKIERUNG BESITZEN.**“

📝★★★★★ Das durchgerechnete Praxisbeispiel

Markierungen: N = NIL · D = DELETED · U = USED · **Hashfunktion:** $h(k, i) = (k + i) \bmod 7$

Aktion	Index 0	1	2	3	4	5	6	Anmerkung
Leere Tabelle	N	N	N	N	N	N	N	
INSERT(15)	N	15/U	N	N	N	N	N	$15 \bmod 7 = 1$
INSERT(36)	N	15/U	36/U	N	N	N	N	$36 \bmod 7 = 1 \rightarrow$ Kollision $\rightarrow i = 1 \rightarrow$ Index 2
SEARCH(71)								$71 \bmod 7 = 1 \rightarrow$ sondiert bis NIL $\rightarrow -1$
INSERT(17)	N	15/U	36/U	17/U	N	N	N	$17 \bmod 7 = 3$
INSERT(29)	N	15/U	36/U	17/U	29/U	N	N	$29 \bmod 7 = 1 \rightarrow$ sondiert bis Index 4
SEARCH(36)								$\rightarrow$ Index 2
DELETE(36)	N	15/U	D	17/U	29/U	N	N	★ DELETED, nicht NIL!
SEARCH(29)								$\rightarrow$ Index 4 ✓ (nur weil DELETED gesetzt ist!)
INSERT(71)	N	15/U	71/U	17/U	29/U	N	N	DELETED wird als frei behandelt

F.9 ★★★★★ Analyse der offenen Adressierung

? *„Wie lange benötigt das Suchen jetzt? DAS lässt sich nicht mehr so einfach beantworten.“*
Entscheidende Faktoren: Keine Listen, sondern **Sequenzen von Hashadressen!** Wie lang sind die Sondierungssequenzen? Wie viele Objekte haben den gleichen Hashwert? (Elemente in den Sequenzen haben teils **unterschiedliche ursprüngliche Hashwerte.**) **Annahmen:** Komplexität nur in Abhängigkeit vom Belegungsfaktor α (DELETED-Felder bleiben unberücksichtigt) · **alle Sondierungssequenzen sind gleich wahrscheinlich**

★★★★ SATZ (Erfolglose Suche)

Für eine Hashtabelle mit offener Adressierung und einem Belegungsfaktor $\alpha < 1$ ist die **ERWARTETE ANZAHL DER SONDIERUNGEN** bei **ERFOLGLOSER** Suche unter der Annahme des gleichmäßigen Hashings **HÖCHSTENS** $\frac{1}{1-\alpha}$.

$$P\{X \geq i\} \leq \left(\frac{n}{m}\right)^{i-1} = \alpha^{i-1} \quad \Rightarrow \quad E[X] = \sum_{i=1}^{\infty} i P\{X = i\} \leq \sum_{i=0}^{\infty} \alpha^i = \boxed{\frac{1}{1-\alpha}}$$

★★★★★ DIE ZAHLENTABELLE (das eindrucksvollste Ergebnis des Kapitels!)

Auslastung	Erwartete Sondierungen $E[X] \leq \frac{1}{1-\alpha}$
$\alpha = 0,5$ — Tabelle zu 50 % voll	$\frac{1}{1-0,5} = 2$
$\alpha = 0,9$ — Tabelle zu 90 % voll	$\frac{1}{1-0,9} = 10$
$\alpha = 0,99$ — Tabelle zu 99 % voll	$\frac{1}{1-0,99} = 100$

🔑 **DIE PRAXISREGEL:** Eine Hashtabelle sollte **NIE ÜBER ~70–80 %** gefüllt sein — die Kosten explodieren am Ende!

★ SATZ (Erfolgreiche Suche)

Für eine Hashtabelle mit offener Adressierung und Belegungsfaktor $\alpha < 1$ ist die erwartete Anzahl der Sondierungen bei **ERFOLGREICHER** Suche höchstens $\frac{1}{\alpha} \cdot \ln\left(\frac{1}{1-\alpha}\right)$.

💡 **Die Beweisidee:** „Hier betrachtet man den Zeitpunkt, zu dem das Objekt **EINGEFÜGT** wurde und nicht den Zeitpunkt, wann man es sucht. **Beim Einfügen hatte man bereits eine ERFOLGLOSE Suche** — und für diesen Fall kennt man bereits die Anzahl der Sondierungsschritte.“

$$E[X] = \frac{1}{n} \sum_{i=0}^{n-1} \frac{1}{1 - \frac{i}{m}} = \frac{m}{n} \sum_{i=0}^{n-1} \frac{1}{m-i} \approx \frac{1}{\alpha} \int_{m-n}^m \frac{dx}{x} = \frac{1}{\alpha} (\ln(m) - \ln(m-n))$$

F.10 ★★ Die große Hashing-Vergleichstabelle

Kriterium	Verkettung (offenes Hashing)	Offene Adressierung (geschlossenes Hashing)
Kollisionsspeicher	externe verkettete Liste	innerhalb der Hashtabelle (Sondierung)
$\alpha > 1$ möglich?	✅ JA	❌ NEIN ($\alpha < 1$ zwingend)
INSERT	$O(1)$	$O\left(\frac{1}{1-\alpha}\right)$
SEARCH (erfolglos)	$O(1 + \alpha)$	$O\left(\frac{1}{1-\alpha}\right)$
SEARCH (erfolgreich)	$O\left(1 + \frac{\alpha}{2}\right)$	$\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$
DELETE	✅ $O(1)$, direkt	⚠️ nur mit DELETED-Markierung
Zusatzspeicher	⚠️ Zeiger	✅ keiner
Überlauf	✅ kein Problem	⚠️ Hashtable Overflow möglich

▼ ? **Selbsttest Teil F**

1. Was ist das Ziel des Hashings (Platz und Laufzeit)?
2. Warum kann man Schlüsselwerte nicht direkt als Index nutzen? Geben Sie das 26-Buchstaben-Beispiel wieder.
3. Nennen Sie die 3 Funktionen der 1. Transformation (String → Zahl).
4. Wie lautet die Java- hashCode -Formel und warum die 31?
5. Nennen Sie die 3 Eigenschaften einer guten Hashfunktion.
6. Was bedeutet uniformes Hashing?
7. Nennen Sie die 4 Typen von Hashfunktionen mit Formel.
8. Welche Eigenschaften sollte m bei der Modulo-Methode haben?
9. Welchen Wert nimmt man für A bei der Multiplikationsmethode und warum?
10. Warum ist die Mittelquadrat-Methode ungeeignet?
11. Wie funktioniert universelles Hashing? Wie hoch ist die Kollisionswahrscheinlichkeit?
12. Warum sind Kollisionen unvermeidbar?
13. Bei welchem n liegt die Kollisionswahrscheinlichkeit beim Geburtstagsparadoxon über 50 %?
14. Was ist Rehashing?
15. Nennen Sie die 2 Strategien der Kollisionsbehandlung mit den 3 Untervarianten.
16. Was ist der Belegungsfaktor? Wie ist er definiert?
17. Welche Laufzeiten hat die Verkettung für INSERT, DELETE, SEARCH?
18. Nennen Sie 3 Vor- und 3 Nachteile der Verkettung.
19. Wie lauten die 3 Sondierungsformeln?
20. Was ist primäres, was sekundäres Clustering?
21. Welche 2 Bedingungen muss h_2 beim doppelten Hashing erfüllen?
22. Warum kann man beim Löschen nicht NIL schreiben? Wie löst man das?
23. Wie wird DELETED beim Suchen und beim Einfügen behandelt?
24. Wie viele Sondierungen erwartet man bei 50 %, 90 % und 99 % Auslastung?
25. Berechnen Sie $h(k)$ für „Januar“ mit der Monatsformel.

TEIL G — KAPITEL 07: GRAPHEN

Die 17 Inhalte: Graphen (allg.) · Euler-Weg/-Zyklus · Traversieren · Spannbäume · Union-Find · Kruskal · Prim · Dijkstra · Bellman-Ford · Floyd-Warshall · Travelling Salesman Problem · Matching · Nearest Neighbor · Greedy · MST-Heuristik · Christofides-Heuristik · k -Opt

G.1 Das SIEBEN-BRÜCKEN-PROBLEM VON KÖNIGSBERG (1736)

Königsberg (bis 1945; danach **Kaliningrad**) am Fluss **Pregel** (danach Pregolja) · **Leonard EULER**, Mathematiker ? „Ist es möglich, durch die Stadt zu gehen, dabei JEDE BRÜCKE GENAU EINMAL zu überqueren und ZUM AUSGANGSPUNKT ZURÜCKZUKOMMEN?“

Eulers Erklärung (wörtlich!):

„Angenommen, solch ein Weg wäre möglich und ein Fußgänger würde ihn gehen. **Wie oft wäre dieser Fußgänger auf der Insel A?**

- **Einmal?** Nein, denn dann hätte er nur **zwei der 5 Brücken** benutzt, die zur Insel A führen.
- **Zweimal?** Wieder nein, dann wären erst **4 Brücken** „verbraucht“.
- **Dreimal?** Immer noch nein, denn dafür wären **6 Brücken** nötig, die ja alle nur einmal begangen werden dürfen.

★ **Also ist KEIN WEG der geforderten Art möglich."**

★ Die Geburt der Graphentheorie

Interesse **NUR** an der **STRUKTUR** des **Gebildes**, die definiert ist durch **Knoten (Punkte)** und **Verbindungen** zwischen diesen Knoten. **KEIN** Interesse daran: **WO** diese Punkte liegen · **WIE LANG** die Verbindungen sind · **WIE** die Verbindungen **GEFORMT** sind → **TOPOLOGIE**

Anwendungen in der Informatik:

- **Modellierung von Netzwerken** (Spielplan WM/EM)
- **Darstellung von Abhängigkeiten** (Produktions- und Logistikprozesse)
- **Strukturierung und Analyse von Ausdrücken**, die nach Grammatikregeln gebildet werden
- **Planung von kürzesten Routen**

G.2 ★★★★★ DEFINITIONEN

Graph

Ein Graph ist ein **PAAR ZWEIER ENDLICHER MENGEN V und E** .

- V (engl. *vertex*) = Menge der **KNOTEN** (auch Eckpunkte, Ecken, Punkte)
- E (engl. *edge*) = Menge der **KANTEN**
- Bedingungen: $V \neq \{\}$ und $V \cap E = \{\}$

🔑 **MERKE:** Ein Graph stellt eine **MENGE VON OBJEKTEN (Knoten)** dar, die zueinander in **BEZIEHUNG/RELATION (Kanten)** stehen.

★ Die Abbildung Ψ (Psi)

Zu einem Graphen gehört die Abbildung Ψ , die jeder Kante die **MENGE IHRER ECKPUNKTE** zuordnet:

$$\Psi : E \rightarrow \mathcal{P}(V) \quad \text{mit} \quad \forall e \in E : 1 \leq |\Psi(e)| \leq 2$$

Die Menge $\Psi(e)$ enthält **mindestens ein Element** (einen Knoten) und **höchstens zwei Elemente**.

	Ungerichteter Graph	Gerichteter Graph
Abbildung	$\Psi : E \rightarrow \mathcal{P}(V)$	$\Psi : E \rightarrow V^2$
Zuordnung	Kante $\leftrightarrow$ MENGE	Kante $\leftrightarrow$ GEORDNETES TUPEL
Notation	$\Psi(a) = \{A, B\}$	$\Psi(a) = (A, B)$
Durchlaufbarkeit	in beide Richtungen	★ NUR in Richtung der Anordnung

★ „Die **REIHENFOLGE** der Eckpunkte gibt an, wie die Kante durchlaufen werden muss!“

★ Erweiterte Definition (gemischte Graphen)

$$\Psi : E \rightarrow V \times V \cup \mathcal{P}(V)$$

Fall	Bedeutung
(i) $\Psi(e) = (A, B) \in V \times V$	e ist eine GERICHTETE Kante mit den Endpunkten A und B ; nur von A nach B durchlaufbar
(ii) $\Psi(e) \in \mathcal{P}(V)$	e ist eine UNGERICHTETE Kante — Ψ hat entweder ein oder zwei Elemente
(iii) $\Psi(e) = \{A\}$	e ist eine GESCHLOSSENE Kante mit nur einem Endpunkt (= SCHLINGE)

★ Teilgraph

G_2 ist ein **Teilgraph** von G_1 , falls: (i) $V_2 \subseteq V_1$ · (ii) $E_2 \subseteq E_1$ · (iii) $\forall e \in E_2: \Psi_2(e) = \Psi_1(e)$

★★ Einfacher Graph

Typ	Bedingungen
Einfach UNGERICHTET	(i) $\forall e \in E \wedge \forall A \in V : \Psi \neq \{A\} \rightarrow$ es gibt KEINE SCHLINGEN (ii) Ψ ist eine INJEKTIVE Abbildung $\rightarrow$ zwischen zwei Knoten gibt es HÖCHSTENS EINE Kante
Einfach GERICHTET	$\forall e \in E \wedge \forall A \in V : \Psi \neq (A, A)$
Einfach ALLGEMEIN	zusätzlich: Es muss ausgeschlossen werden, dass zwischen zwei Punkten SOWOHL eine ungerichtete ALS AUCH eine gerichtete Kante existiert

★★★ „Ein **UNGERICHTETER** Graph ist **IMMER** durch einen **GERICHTETEN** Graphen darstellbar. Ein **GERICHTETER** Graph ist dagegen **NICHT** durch einen ungerichteten Graphen darstellbar.“

★★★★ DER GRAD EINES KNOTENS (auswendig!)

Ungerichteter Graph:

$$\text{Grad}(A) = \underbrace{|\{e \in E \mid A \in \Psi(e) \wedge \{A\} \neq \Psi(e)\}|}_{\text{Kanten zu ANDEREN Punkten}} + 2 \cdot \underbrace{|\{e \in E \mid \{A\} = \Psi(e)\}|}_{\text{Anzahl aller SCHLINGEN}}$$

★★★ „**SCHLINGEN ERHÖHEN DEN GRAD UM +2** (Multiplikation mit 2)!“

Gerichteter Graph:

Begriff	Definition
AUSSENGRAD $g^+(A)$	Anzahl der Kanten, die von A WEGFÜHREN: $g^+(A) = \{e \in E \mid \exists X \in E : \Psi(e) = (A, X)\} $
INNENGRAD $g^-(A)$	Anzahl der Kanten, die zu A HINFÜHREN: $g^-(A) = \{e \in E \mid \exists X \in E : \Psi(e) = (X, A)\} $
GRAD	$\text{Grad}(A) = g^+(A) + g^-(A)$

ISOLIERTER KNOTEN: Ein Knoten vom Grad 0.

★★ Pfad (Weg), Zyklus, Zusammenhang

Begriff	Definition
Pfad (Weg)	Eine Folge von Knoten $(A_1, \dots, A_i)$, wobei aufeinanderfolgende Knoten durch eine Kante verbunden sind und ★ JEDE KANTE HÖCHSTENS EINMAL in dieser Folge vorkommen darf
Verbindung	Der Pfad $(A, \dots, B)$ zwischen zwei Punkten
Länge eines Pfades	Pfad der Länge $(n - 1) = \text{Anzahl Knoten} - 1 \cdot \text{Pfad der Länge } 0 = \text{Einzelknoten}$
ZUSAMMENHÄNGEND	★ Jedes PAAR von Punkten ist durch einen Weg verbunden — zwischen zwei beliebigen Knoten muss es immer einen Pfad geben
STARK VERBUNDEN (connected)	X und Y sind stark verbunden $\iff X = Y$ ODER es gibt einen Weg von X nach Y ALS AUCH einen Weg zurück von Y nach X
ZYKLUS	Ein Weg, der im SELBEN PUNKT beginnt und endet (= geschlossener Weg). ★ Ein Zyklus sollte aus mindestens 3 Knoten bestehen!

! Hinweise zur Pfadnotation:

- Bei gerichteten Kanten müssen die Knoten in der RICHTIGEN REIHENFOLGE notiert werden
- Bei mehreren Kanten zwischen zwei Knoten muss die Kantenauswahl durch eine INDIZIERUNG kenntlich gemacht werden

G.3 ★★★ DIE 7 SPEZIELLEN GRAPHEN

Graph	Definition
BAUM	Ein ZUSAMMENHÄNGENDER, UNGERICHTETER Graph, der KEINE ZYKLEN enthält
WALD	Eine Menge PAARWEISE NICHT ZUSAMMENHÄNGENDER BÄUME, die keine Zyklen enthalten
WURZELGRAPH	Alle Knoten sind vom Wurzelknoten aus ERREICHBAR
Gerichteter AZYKLISCHER Graph (DAG)	Ein gerichteter Graph, der KEINE ZYKLEN enthält
VOLLSTÄNDIGER Graph	Ein ungerichteter Graph, bei dem ALLE PAARE von Knoten ADJAZENT sind und der Graph ein MAXIMUM an Kanten enthält
BIPARTITER Graph	Die Knotenmenge lässt sich in ZWEI DISJUNKTE Teilmengen V_1 und V_2 aufteilen: $V = V_1 \cup V_2$, $V_1 \neq \{\}$, $V_2 \neq \{\}$, $V_1 \cap V_2 = \{\}$; JEDE Kante verbindet einen Knoten aus V_1 mit einem Knoten aus V_2
PLANARER Graph	Ein Graph, der sich OHNE KANTENÜBERSCHNEIDUNGEN in der Ebene darstellen bzw. zeichnen lässt

★ Die Eulersche Polyederformel für planare Graphen

Für einen **planaren und zusammenhängenden** Graphen $G(V, E)$ gilt:

$$|V| - |E| + |A| = 2 \quad \text{oder} \quad |E| \leq 3|V| - 6$$

(A = die Menge der durch die Kanten begrenzten **Flächenstücke**.)

Wozu braucht man planare Graphen? Platinenlayout · Rohrleitungsnetze · teils Bahn- und Straßennetze

G.4 ★★★ REPRÄSENTATION VON GRAPHEN

? „Wie bekommt man einen Graphen in den Rechner?“ → Adjazenzmatrix oder Adjazenliste

① ADJAZENZMATRIX

In einer $n \times n$ -Adjazenzmatrix M wird in der i -ten Zeile und j -ten Spalte die ANZAHL DER KANTEN eingetragen, die zwischen dem i -ten und j -ten Knoten existieren.

$$M[i, j] = |\{e \in E \mid \Psi(e) = \{\varphi(i), \varphi(j)\}\}| \quad A[i, j] = \begin{cases} 1 & (i, j) \in E \\ 1 & (j, i) \in E \\ 0 & \text{sonst} \end{cases}$$

(Die bijektive Abbildung $\Phi : V \rightarrow \{1, \dots, n\}$ ordnet jedem Knoten eine eindeutige natürliche Zahl zu; φ ist die Umkehrung.)

✓	✗
★ Effizienter Test, ob zwei Knoten mit einer Kante verbunden sind: $O(1)$	⚠ Speicherbedarf $\Theta(N^2)$
Symmetrische Matrix bei ungerichteten Graphen: $A[i, j] = A[j, i]^*$	Lineares Auffinden aller Nachfolger eines Knotens: $O(n)$

*bei Graphen ohne Kantengewichte und ohne Mehrfachkanten

② ADJAZENZLISTE

In einer Adjazenliste wird FÜR JEDEN KNOTEN EINE LISTE seiner NACHFOLGER angelegt. Für die Länge des Arrays ergibt sich $|V| = n$. Pro Knoten v gibt es eine Liste mit den Nummern der zu v adjazenten Knoten w .

✓	Speicherbedarf
★ Effizientes Durchlaufen aller zu einem Knoten v adjazenten Knoten	$\Theta(V + E) = \begin{cases} 2E & \text{falls } G \text{ ungerichtet} \\ E & \text{falls } G \text{ gerichtet} \end{cases}$

★★ GEWICHTETE (BEWERTETE) GRAPHEN

Sei $G(V, E, \Psi)$ ein Graph. Falls es zusätzlich eine Funktion Γ (Gamma) gibt, die **JEDER KANTE** eine **BEWERTUNG** aus der Menge der **REELLEN ZAHLEN** zuordnet, dann heißt G ein **GEWICHTETER** oder **BEWERTETER** Graph:

$$\Gamma : E \rightarrow \mathbb{R}$$

Was können das für Werte sein? Entfernung · Kosten · Zeit · Algorithmenschritte · Verlust · Erfolgswahrscheinlichkeit

★★ Die **BEWERTUNGSMATRIX** (Abstandsmatrix) ist **NICHT ZU VERWECHSELN** mit der **Adjazenzmatrix**! ⚠️ *Existiert zwischen zwei Knoten **keine** Kante, so wird die Distanz als **unendlich groß** betrachtet → Symbol ∞ oder $-\infty$.

G.5 ★★★ EULER-WEG und EULER-ZYKLUS

Definitionen

	Definition
EULER-WEG	Ein Weg, der JEDE KANTE des Graphen GENAU EINMAL durchläuft. ★ Es wird NICHT verlangt, dass der Weg am selben Punkt endet, an dem er begonnen hat.
EULER-ZYKLUS	Ein Zyklus, der JEDE KANTE genau einmal durchläuft UND AM SELBEN PUNKT ENDET , an dem er begonnen hat.

★★★ DIE BEDINGUNGEN (auswendig — das ist DIE Prüfungsfrage!)

UNGERICHTETE Graphen (ohne isolierte Punkte)

	Bedingungen
EULER-WEG existiert $\iff$	① G ist ZUSAMMENHÄNGEND ② Der Grad JEDES Punktes ist GERADE ODER GENAU ZWEI Punkte haben einen UNGERADEN Grad
EULER-ZYKLUS existiert $\iff$	① G ist ZUSAMMENHÄNGEND ② Der Grad JEDES Punktes ist eine GERADE ZAHL

GERICHTETE Graphen (ohne isolierte Punkte)

	Bedingungen
EULER-WEG existiert $\iff$	Entweder $g^+(A) = g^-(A)$ für ALLE Punkte ODER es gibt GENAU EINEN Punkt A mit $g^+(A) = g^-(A) + 1$ und GENAU EINEN Punkt E mit $g^+(E) = g^-(E) - 1$
EULER-ZYKLUS existiert $\iff$	$g^+(A) = g^-(A)$ für ALLE Punkte

★ Anders ausgedrückt (wörtlich): „Wenn es einen Punkt gibt, bei dem **zwei Kanten wegführen** und **eine Kante hin**, dann muss es einen anderen Punkt geben, bei dem **zwei Kanten hinführen** und **eine Kante weg**.“

 **Das Beispiel des Dozenten:** $g^+(A) = 2 = 1 + 1 = g^-(A) + 1 \cdot g^+(B) = 1 = g^-(B) \cdot g^+(C) = 1 = 2 - 1 = g^-(C) - 1 \cdot g^+(D) = 1 = g^-(D) \Rightarrow$ **Es existiert ein Euler-WEG** (aber kein Zyklus).

★★ Euler-Zyklus — GRAPHISCHE Konstruktion (Algorithmus von Hierholzer)

- ① Starte in einem Knoten, bis dieser Knoten wieder erreicht wird
- ② Wiederhole Schritt ① bis alle Knoten besucht wurden
- ③ Verbinde die einzelnen Zyklen zu einem großen Zyklus (Eulerkreis)

★★ Euler-Zyklus — MATRIX-Konstruktion

- ① Wähle einen beliebigen Punkt als Startpunkt
 - Nach dessen Wahl steht dieser nicht mehr zur Verfügung!
 - Der entsprechende Eintrag in der Adjazenzmatrix wird auf 0 gesetzt
- ② Auswahl der ersten verfügbaren Kante
- ③ Wiederhole, bis alle Knoten bearbeitet wurden

!★★ Der BREAKOUT:

„Entsteht während der Bearbeitung ein Zyklus, von dessen Ausgangs-/Endpunkt man **nicht weitermachen kann**, weil keine Kanten mehr verfügbar sind, muss man einen **BREAKOUT** durchführen: ① **Wähle den ERSTEN Knoten, von dem noch FREIE Kanten fortführen, als neuen Anfangspunkt** ② **ROTIERE den bisher gewählten Weg entsprechend**“

 **Beispielverlauf:** $(1,2) \rightarrow (1,2,3) \rightarrow (1,2,3,1) \rightarrow$ **BREAKOUT:** $(1|2,3,1) \rightarrow (2,3,1|1) \rightarrow (2,3,1|2) \rightarrow (2,3,1,2) \rightarrow (2,3,1,2,4) \rightarrow \dots \rightarrow (4,3,5,2,3,1,2,4,5,6,4)$

★★ Euler-WEG — Konstruktion mittels HILFSZYKLUS

- ① Füge eine HILFSKANTE zwischen den Punkten mit UNGERADEM Grad hinzu
- ② Konstruiere einen EULER-ZYKLUS, der mit dieser Hilfskante ENDET
- ③ ENTFERNE die Hilfskante aus dem Euler-Zyklus, um den Euler-WEG zu erhalten

★ Laufzeitverhalten

Algorithmus	Komplexität
Algorithmus von EULER	$O(V ^3)$
Algorithmus von FLEURY	$O(E ^2)$
★ Algorithmus von HIERHOLZER	$O(E + V)$

G.6 ★★★★★ TRAVERSIEREN VON GRAPHEN

 „In einem Graphen ist das Traversieren **schwieriger als in einem Baum**, weil: **ein Graph ein GEFLECHT von Kanten ist, KEINE HIERARCHIE (Wurzel) besitzt und jeder Knoten „GLEICHWERTIG“ ist.** Ziel: Alle Knoten eines Graphen in systematischer Reihenfolge zu besuchen und ein „Grundgerüst“ für die späteren Graphenalgorithmen aufzubauen.

★ Die EXPANSION (Voraussetzung für beide Verfahren!)

Die Expansion $X(v)$ eines Graphen G in einem Knoten v ist ein BAUM:

- Ist v ein Blatt, dann ist $X(v)$ der Baum, der nur aus dem Knoten v besteht
- Wenn $v_1, \dots, v_k$ Nachfolger von v sind, so ist $X(v)$ der Baum $(v, X(v_1), \dots, X(v_k))$

⚠ „Der Baum listet ALLE vom Knoten v in G ausgehenden Pfade auf und ist UNENDLICH, wenn \$G\$ ZYKLEN enthält.“ — Das Resultat sind AUFSPANNENDE WÄLDER.

★★★★ BREITENSUCHE vs. TIEFENSUCHE

	BREITENSUCHE (<i>Breadth First Search</i>)	TIEFENSUCHE (<i>Depth First Search</i>)
Vorgehen	① Starte in einem beliebigen Knoten q · ② ITERATIVES Besuchen der noch nicht besuchten NACHBARN von q , dann wieder deren Nachbarn usw. — Durchlauf EBENENWEISE	① Starte in einem beliebigen Knoten q · ② Arbeite den ERSTEN NACHFOLGER ZUERST VOLLSTÄNDIG ab , dann den zweiten usw.
★ Realisierung	mittels einer WARTESCHLANGE (Queue) — hier werden alle Knoten abgelegt, die schon einmal betrachtet wurden, aber noch nicht betrachtete Nachbarn haben	mittels eines STAPELS (Stack) — implizite Nutzung durch REKURSION
★★ Entspricht	LEVEL-ORDER-Durchlauf	PRE-ORDER-Durchlauf
Laufzeit	$O(V + E)$	$O(V + E)$

⚠ Abbruch bei bereits besuchten Knoten!

G.7 ★★★★★ SPANNBÄUME

? „Gibt es auch andere Algorithmen, die dabei helfen, Probleme in der Graphentheorie zu lösen?“ → 💡 Mit Hilfe eines AUFSPANNENDEN BAUMES.

★★★ Warum ein Baum? (Die 3 Begründungen)

1. Die Lösung muss **ZUSAMMENHÄNGEND** sein — es muss einen Weg von jedem Punkt zu jedem anderen geben
2. ★ Die Lösung darf **KEINE ZYKLEN** enthalten — „wenn die Verbindung möglichst billig sein soll, darf es nicht zwei Wege geben, die dasselbe Punktepaar verbinden“
3. **JEDER Punkt des Graphen muss zur Lösung gehören**

Definition Aufspannender Baum

$T(V_T, E_T, \Psi_T)$ heißt **AUFSPANNENDER BAUM** von G , falls: (i) T ist ein BAUM · (ii) $V_G = V_T$ · (iii) $E_T \subset E_G$ · (iv) $\forall e \in E_T: \Psi_T(e) = \Psi_G(e)$ ★ „Wenn G ein **ZUSAMMENHÄNGENDER** Graph ist, dann ist es **STETS MÖGLICH**, einen aufspannenden Baum zu konstruieren.“

Konstruktion: ① Wähle einen beliebigen Punkt · ② Solange es noch Punkte gibt, die nicht zu T gehören: suche einen solchen Punkt, der über eine Kante e mit T verbunden ist, und füge Kante + Punkt hinzu

★★ DIE CAYLEY-FORMEL (Satz von Cayley)

$$\boxed{\#\text{Spannbäume} = n^{n-2}} \quad n = \text{Anzahl Knoten im Graphen}$$

(ohne Beweisführung; siehe z. B. „Proofs from THE BOOK“)

 **Beispiele:** $n = 4 \Rightarrow 4^2 = 16 \cdot n = 5 \Rightarrow 5^3 = 125 \cdot n = 10 \Rightarrow 10^8 = 100.000.000$

★ Minimaler Spannbaum (MST)

Ein Baum, der ALLE Knoten eines Graphen enthält, heißt **MINIMAL AUFSPANNENDER BAUM**, wenn die **SUMME ALLER KANTENGEWICHTE MINIMAL** ist.  „Diese Art von Problemen werden als **OPTIMIERUNGSPROBLEME** bezeichnet und lassen sich unter anderem mit Hilfe von **GREEDY-ALGORITHMEN** lösen.“

★★ GREEDY-ALGORITHMEN

„Ein Greedy-Algorithmus ist ein Verfahren, das in JEDEM SCHRITT LOKAL DIE GIERIGSTE/BESTE WAHL trifft, in der HOFFNUNG, so die GLOBALE Lösung zu finden.“

- Sucht die Lösung in „gieriger Art und Weise“ zusammen
- Die Lösung wird durch EINZELENTSCHEIDUNGEN aufgebaut
- ★ Die einmal getroffene Entscheidung wird NIE WIEDER ZURÜCKGENOMMEN
- ⚠ Häufig KEINE OPTIMALITÄTSGARANTIE
- ⚠ Ist nicht notwendigerweise EINDEUTIG

G.8 ★★★ KRUSKAL-ALGORITHMUS (MST)

- ① Erzeugung eines zunächst LEEREN Baums und SORTIERUNG ALLER KANTEN nach ihrem Gewicht
- ② Wähle die Kante mit dem GERINGSTEN Gewicht und füge sie dem leeren Baum hinzu
- ③ Wähle im weiteren Verlauf IMMER die Kante mit dem geringsten Gewicht
 - ⚠ Es darf dabei KEIN ZYKLUS entstehen
 - Verbindet eine Kante einen bereits im Spannbaum enthaltenen Knoten, so wird sie NICHT hinzugefügt; weiter mit der nächstgrößeren Kante
- ④ Wiederhole Schritt ③, bis alle Knoten hinzugefügt sind

 **Hinweis:** „Sollten mehrere Kanten das GLEICHE GEWICHT haben, ist man in der WAHL DER KANTE FREI.“

 **Das große Beispiel des Dozenten** — Städtetz mit den Knoten **Frankfurt, Mannheim, Karlsruhe, Stuttgart, Ulm, Nürnberg, Basel, Ravensburg, München, Bern, Zürich** und Kantengewichten wie 85, 210, 230, 67, 135, 191, 64, 107, 195, 86, 171, 123, 170, 91, 180, 178, 120.

★★★★ Die UNION-FIND-DATENSTRUKTUR

 „Wie geht man mit den einzelnen, so entstandenen Teilbäumen um?“ **Idee:** Für jeden Knoten des Graphen erzeugt man eine MENGE. Die Menge wird durch einen **gewurzelten Baum** repräsentiert; die **WURZEL** ist der eindeutige **REPRÄSENTANT** des Baums. „Es handelt sich um eine einfache Datenstruktur, bei der **jeder Knoten nur EINEN Vorgänger** hat.“

Operation	Aufgabe
MAKE-SET	Erstellt einmalig eine Menge (von Mengen) — $x.parent = x$ (Wurzel zeigt auf sich selbst), $x.rank = 0$
UNION	Vereinigt zwei DISJUNKTE Mengen — hänge die Wurzel des einen Baumes unter die des anderen
FIND-SET	Liefert ein REPRÄSENTATIVES Element einer Menge — traversiere im Baum bis zur Wurzel und gib die Wurzel zurück

```

MAKE-SET(x)
    x.parent = x
    x.rank = 0
    return x

UNION(x, y)
    LINK( FIND-SET(x), FIND-SET(y) )

LINK(x, y) // Hängt den FLACHEREN Teilbaum unter den anderen
    if (x.rank > y.rank) y.parent = x
    else
        x.parent = y
        if (x.rank == y.rank)
            y.rank = x.rank + 1 // Baum wächst um 1

FIND-SET(x) // rekursiv
    if (x != x.parent)
        x.parent = FIND-SET(x.parent)
    return x.parent

FIND-SET(x) // iterativ mit PFADKOMPRESSION
    r = x
    while (r != r.parent) r = r.parent // Wurzel suchen
    while (x != r) // Pfad-Kompression
        px = x.parent; x.parent = r; x = px
    return r

```

★★ Die 4 Verbesserungen der Union-Find-Datenstruktur

#	Verbesserung	Prinzip
1	Union-By-Size / Union-By-Rank	UNION-Operation: Anhängen des FLACHEREN Baums unter den TIEFEREN Baum
2	Pfadkomprimierung / Pfadkompression	FIND-SET-Operation: Hänge ALLE Teilbäume auf dem Weg zur Wurzel UNTER DIE WURZEL
3	Splitting (Aufteilung)	JEDER Knoten auf dem Pfad zur Wurzel zeigt auf seinen bisherigen GROSSVATER (falls vorhanden)
4	Halving (Halbierung)	NUR JEDER ZWEITE Knoten auf dem Pfad zeigt auf seinen bisherigen Großvater

⚠ „Diese Methoden garantieren **KEINE spezielle Eigenschaft** — sie **beschleunigen lediglich das Verfahren.**“

$$O(m \cdot \alpha(m, n) + n) \quad (\text{bei Union-By-Rank + Pfadkompression})$$

★★ Die ACKERMANN-FUNKTION und ihre Inverse

$$A(m, n) = \begin{cases} n + 1 & \text{für } m = 0 \\ A(m - 1, 1) & \text{für } m > 0 \wedge n = 0 \\ A(m - 1, A(m, n - 1)) & \text{für } m > 0 \wedge n > 0 \end{cases}$$

⚠ ★ „Wächst EXTREM SCHNELL: $A(4, 2) > 10^{19729}$ “ ★ Die Inverse $\alpha(m, n) = \min\{i \geq 1 : A(i, 1) > n\}$ wächst EXTREM LANGSAM — in der Praxis ≤ 5 für jedes realistische n !

★ Laufzeit von Kruskal

Implementierung	Komplexität
Naiv (Warteschlange/Liste)	$O(V ^2)$
UNION-FIND (normal)	$O(E \cdot \log(V))$
★ UNION-FIND (verbessert)	$O(E \cdot \alpha(V))$

🔑 „Der Algorithmus von Kruskal benötigt diese Datenstruktur zur **Verwaltung von DISJUNKTEN MENGEN** — sie **entscheidet für je zwei Knoten effizient, ob diese zum GLEICHEN BAUM gehören oder nicht.**“

G.9 ★★★★★ PRIM-ALGORITHMUS (MST)

- ① Wähle einen STARTKNOTEN aus
- ② Wähle die Kante VOM STARTKNOTEN aus, welche das geringste Gewicht besitzt und noch nicht im neuen Baum enthalten ist
- ③ Füge die neue Kante und den neuen Knoten zum Baum hinzu
- ④ Vom AKTUELL AUFSPANNENDEN BAUM wählt man wiederum die Kante mit dem geringsten Gewicht, sofern Kante und Knoten noch nicht zum Spannbaum gehören

★★★★ KRUSKAL vs. PRIM — DER UNTERSCHIED

	KRUSKAL	PRIM
Ausgangspunkt	ALLE Kanten global nach Gewicht sortiert	EIN Startknoten; wächst von dort
Struktur	erzeugt einen WALD, der zusammenwächst	erzeugt EINEN wachsenden Baum
★ Hilfsdatenstruktur	UNION-FIND-Datenstruktur (speichert, ob zwei Knoten zum Spannbaum gehören)	★ PRIORITÄTSWARTESCHLANGE (realisiert durch einen HEAP) — speichert alle Knoten, die noch nicht mit dem Baum verbunden sind; die Knoten werden nach der Priorität des minimalen Kantengewichts gespeichert. Dadurch entfällt die Notwendigkeit der Union-Find-Datenstruktur!
Laufzeit	$O(E \cdot \alpha(V))$	Naiv: $O(V ^2)$ · ★ Fibonacci-Heap: $O(E \cdot \log(V))$

G.10 ★★★★★ KÜRZESTE PFADE

Definitionen

Länge/Gewicht eines Pfades $p = (v_0, v_1, \dots, v_k)$:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

Gewicht des KÜRZESTEN Pfades:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \rightarrow v\} & \text{Pfad existiert} \\ \infty & \text{sonst} \end{cases}$$

Falls $|\delta(u, v)| < \infty$ gilt:

- ★ Es existiert ein kürzester Pfad mit HÖCHSTENS $|V| - 1$ KANTEN (alle Pfade gesehen)
- ★ Es existiert ein kürzester Pfad OHNE ZYKLEN

Begründung: Enthielte der Pfad $|V|$ oder mehr Kanten, käme **ein Knoten mindestens doppelt** vor $\implies$ der Pfad enthält einen **ZYKLUS**.

★★ Die 3 PROBLEM VARIANTEN

Variante	Definition
Single-Pair-Shortest-Path	Kürzester Pfad zwischen ZWEI ausgewählten Knoten $u, v \in G(V, E)$
Single-Source-Shortest-Path	Kürzeste Pfade von EINEM FIXEN Knoten u zu ALLEN anderen Knoten
All-Pairs-Shortest-Path	Kürzeste Pfade zwischen ALLEN KNOTENPAAREN $\forall u, v \in G(V, E), u \neq v$

!★★ NEGATIVE KANTENGEWICHTE

$\delta(u, v)$ ist **prinzipiell auch für negative Kantengewichte definiert** — d. h. die **Pfadlängen können durch die Addition insgesamt negativ werden**.

!! „Es ist **PROBLEMATISCH**, wenn es auf dem Pfad von u nach v einen **ZYKLUS MIT NEGATIVER GESAMTLÄNGE** gibt:

- Mit JEDEM DURCHLAUF des negativen Zyklus **VERKÜRZT SICH DER PFAD**
- Der kürzeste Pfad ist bzgl. der Anzahl der Kanten **UNENDLICH LANG** und streng genommen **NICHT MEHR DEFINIERT**
- Es gilt $\delta(u, v) = -\infty$

★ **ABER:** „Ein Zyklus mit negativer Gesamtlänge bedeutet **NICHT**, dass **ALLE** Distanzen im Graphen $-\infty$ sind. Es sind **NUR** die Pfade davon betroffen, die auch den negativen Zyklus **ENTHALTEN**.“*

 **Beispiel des Dozenten:** $w(u, a, e, d, c, v) = 9 \cdot w(u, a, b, c, v) = -2 \cdot w(u, a, b, c, d, e, a, b, c, v) = -3 \leftarrow$ immer kürzer!

★★ ZYKLUS-EIGENSCHAFTEN (auswendig!)

Zyklusgewicht	Konsequenz
$w(\text{Zyklus}) < 0$	! $\delta(u, v) = -\infty$
$w(\text{Zyklus}) = 0$	Der Zyklus kann aus dem Pfad GELÖSCHT werden — $w(\text{Pfad})$ bleibt unverändert
$w(\text{Zyklus}) > 0$	Durch Löschen des Zyklus entsteht ein NEUER, KÜRZERER Pfad : $w(\text{Pfad}_{\text{neu}})$

Die 3 Arten von Pfadlängen: reelle endliche Werte (können auch negativ sein) · **Zyklen mit negativer Gesamtlänge** ($-\infty$) · **Knoten, die man nie erreichen kann** (∞)

★★ SPEICHERUNG der kürzesten Pfade

Möglichkeit	Beschreibung	Bewertung
① Speicherung aller Pfade als LISTEN	$n - 1$ viele Listen (für jeden Knoten außer dem Startknoten)	⚠ Problem: Listen sind im Schnitt relativ lang $\rightarrow n^2$ Ausgabegröße $\Rightarrow$ Der Algorithmus kann NICHT SCHNELLER ALS n^2 werden!
★ ② Speicherung in einem FELD (BAUM)	Speichert man einen kürzesten Pfad von u nach v , dann sind auch ALLE TEILPFADE von u nach v OPTIMAL. Jeder Knoten hat in einem kürzesten Pfad MAXIMAL EINEN VORGÄNGER $\rightarrow$ Speicherung als SHORTEST-PATHS-TREE, Länge n	★ „Man muss nicht jeden Pfad explizit speichern, sondern lediglich EINEN BAUM“ · ⚠ Die kürzesten Pfade sind <i>nicht eindeutig</i> , daher ist auch <i>der Baum nicht eindeutig</i> .

G.11 ★★★ DIJKSTRA-ALGORITHMUS

- ① Wähle einen Startknoten aus
- ② Der Weg zum Startknoten hat KOSTEN 0
- ③ Setze den Weg zu allen anderen Knoten auf UNENDLICH
- ④ Der Algorithmus wählt schrittweise über die als nächstes erreichbaren Knoten die MOMENTAN GÜNSTIGSTEN Wege aus – dabei kann er auch VERBESSERUNGEN vornehmen, so lange, bis alle Knoten besucht wurden und kein besserer Weg gefunden wurde
- ⑤ In die WARTESCHLANGE werden alle bereits gefundenen Knoten eingefügt
- ⑥ Der Knoten mit dem momentan GÜNSTIGSTEN Weg vom Startknoten steht GANZ VORNE
- ⑦ Der VORDERSTE Knoten wird aus der Warteschlange ENTNOMMEN
- ⑧ Dann werden alle NACHBARKNOTEN des entnommenen Knotens betrachtet und geprüft:
 - i) Ist der Knoten in der Warteschlange enthalten UND sind die Kosten über die neue Kante GERINGER als die bisherigen? $\rightarrow$ Kosten VERRINGERN
 - ii) Falls nein: wurde der Knoten überhaupt schon besucht?
 $\rightarrow$ Falls nicht: in die Warteschlange EINFÜGEN

★ „Ein Knoten erhält die Kosten, die sich aus der SUMME der Kosten zu seinem VORGÄNGERKNOTEN und den Kosten der NEU ENTDECKTEN KANTE ergeben.“ ⚠⚠ „Der Algorithmus funktioniert NUR MIT NICHT-NEGATIVEN KANTEN.“

📝 ★★★ Das durchgerechnete Beispiel (Start in Knoten D)

Schritt	Aktueller Knoten (Dist.)	Besuchte Kanten	Knoten-Update	Warteschlange danach
1	(D, 0)	(D,B,12), (D,C,2), (D,E,6), (D,F,3)	(B,12), (C,2), (E,6), (F,3)	(C,2), (F,3), (E,6), (B,12)
2	(C, 2)	(C,E,3)	(E, 5) $\leftarrow$ Verbesserung von 6!	(F,3), (E,5), (B,12)
3	(F, 3)	(F,A,1)	(A, 4)	(A,4), (E,5), (B,12)
4	(A, 4)	(A,B,7)	(B, 11) $\leftarrow$ Verbesserung von 12!	(E,5), (B,11)
5	(E, 5)	(E,B,5), (E,G,8)	(B, 10) $\leftarrow$ Verbesserung! $\cdot$ (G, 13)	(B,10), (G,13)
6	(B, 10)	(B,H,6), (B,G,2)	(G, 12) $\leftarrow$ Verbesserung! $\cdot$ (H, 16)	(G,12), (H,16)
7	(G, 12)	(G,H,3)	(H, 15) $\leftarrow$ Verbesserung!	(H,15)
8	(H, 15)			✓ fertig

Vorgänger-Tabelle (Shortest-Paths-Tree): $A \leftarrow F \cdot C \leftarrow D \cdot F \leftarrow D \cdot E \leftarrow C \cdot B \leftarrow E \cdot G \leftarrow B \cdot H \leftarrow G$

★ Laufzeitverhalten

Implementierung	Komplexität
Naiv ($ V \times V $ -Matrix)	$O(V ^2)$
Binärer Heap	$O((E + V) \cdot \log(V))$
★ Fibonacci-Heap	$O(E + V \cdot \log(V))$

G.12 ★★★ BELLMAN-FORD-ALGORITHMUS

„Der Bellman-Ford-Algorithmus ist eine VERALLGEMEINERUNG des Dijkstra-Algorithmus.“

★★★★ Bellman-Ford vs. Dijkstra (die Unterschiedstabelle!)

	DIJKSTRA	BELLMAN-FORD
Knotenstatus	Knoten werden abschließend betrachtet	★ Die Knoten werden zu KEINEM ZEITPUNKT als abschließend betrachtet
Endgültigkeit	sofort nach Entnahme	★ Die kürzeste Entfernung steht ERST MIT BEENDIGUNG des Algorithmus fest
Negative Kanten	✗ NICHT erlaubt	✓ ERLAUBT
Negative Zyklen	—	✓ ERKENNT negative Zyklen — ⚠ funktioniert nur korrekt, wenn KEINE negativen Zyklen auftreten
Laufzeit	$O(E + V \log V)$	$O(V \cdot E)$

G.13 ★★★ FLOYD-WARSHALL-ALGORITHMUS (All-Pairs)

Idee: „Der Algorithmus geht davon aus, dass **wenn der kürzeste Weg von u nach v durch w geht, dann sind die Teilpfade von u nach w und von w nach v ebenfalls MINIMAL.** Um den kürzesten Weg selbst zu konstruieren, wird **parallel zu A eine weitere Matrix F** geführt, in der $F[i, j]$ jeweils den **vorherigen Knoten** auf dem kürzesten Weg von i nach j enthält.“

Vorgehen: ① Aufstellen einer BEWERTUNGSMATRIX · ② In jeder Iteration werden alle möglichen Pfade zwischen allen Paaren einer **FIXEN ZEILE UND SPALTE** schrittweise verglichen — nur die **BESTEN** Werte werden gespeichert/verändert

★★★★ Die Kernformel (auswendig!)

$$m_{i,j}^{(k)} = \min \left[m_{i,j}^{(k-1)}; m_{i,k}^{(k-1)} + m_{k,j}^{(k-1)} \right]$$

Symbol	Bedeutung
i	i -te Zeilennummer
j	j -te Spaltennummer
k	k -te Iteration (= der „Zwischenknoten“)

Initialisierung ($k = 0$): Existiert **keine Verbindung** zwischen Knoten, wird deren Distanz auf ∞ gesetzt. **I. d. R. existiert keine Schleife auf den eigenen Knoten — die Distanz auf sich selbst wird auf 0 gesetzt.**

 **Beispielrechnungen aus $k = 1$:**

```

min(m2,2 ; m2,1 + m1,2) = min(0 ; 1 + ∞) = min(0 ; ∞) = 0
min(m2,3 ; m2,1 + m1,3) = min(∞ ; 1 + ∞) = min(∞ ; ∞) = ∞
min(m2,4 ; m2,1 + m1,4) = min(2 ; 1 + ∞) = min(2 ; ∞) = 2
min(m2,5 ; m2,1 + m1,5) = min(∞ ; 1 + (-1)) = min(∞ ; 0) = 0 ← Verbesserung!
min(m4,5 ; m4,1 + m1,5) = min(3 ; -4 + (-1)) = min(3 ; -5) = -5 ← Verbesserung!

```

✓ „Der Algorithmus funktioniert AUCH DANN, wenn die Kanten ein **NEGATIVES GEWICHT** haben!“ ⚠ „ZYKLEN mit negativer Länge können dagegen **NICHT ERKANNT** werden.“ Laufzeit:

$$O(|V|^3)$$

G.14 ★★ SCHWIERIGE VERFAHREN — Hamilton

Beispiel 1: Der LÄNGSTE Weg

Aufgabe: Finden Sie den längsten Weg in einem Graphen. ⚠ **FAZIT:** „Es gibt **WEDER** effiziente Methoden **NOCH** Algorithmen, um einen solchen längsten Weg herauszufinden.“

Beispiel 2: HAMILTON

	Definition
HAMILTON-WEG	Ein Weg, der JEDEN KNOTEN des Graphen GENAU EINMAL durchläuft. Es wird nicht verlangt, dass der Weg am selben Punkt endet.
HAMILTON-ZYKLUS	Ein Zyklus, der jeden Knoten genau einmal durchläuft UND am selben Punkt endet, wie er begonnen hat.

⚠⚠ **FAZIT:** „Es gibt **WEDER** effiziente Methoden **NOCH** Algorithmen, um festzustellen, ob ein Graph einen Hamilton-Weg oder Hamilton-Zyklus enthält.“

★★★★ EULER vs. HAMILTON — die zentrale Gegenüberstellung!

	EULER	HAMILTON
Was wird genau einmal durchlaufen?	★ JEDE KANTE	★ JEDER KNOTEN
Existenzbedingung	✓ einfach prüfbar (Knotengrade!)	⚠ kein effizientes Kriterium bekannt
Komplexität	✓ $O(E + V)$ (Hierholzer)	⚠ NP-schwer

G.15 ★★★★★ DAS TRAVELLING SALESMAN PROBLEM (TSP)

Die zwei Formulierungen

Allgemein: Gegeben seien n Orte, jeder Ort i sei mit jedem Ort j durch eine Strecke $S[i][j]$ verbunden. **Gesucht ist eine RUNDREISE, auf der JEDER ORT BESUCHT wird und die die MINIMALE LÄNGE hat.**

Graphentheoretisch: Gegeben sei ein **ungerichteter, positiv bewerteter Graph** mit n Knoten, in dem **alle Knoten durch eine Kante miteinander verbunden** sind. **Gesucht ist ein ZYKLUS MINIMALER LÄNGE, der JEDEN KNOTEN BERÜHRT.**

★★★★★ DIE ANZAHL DER MÖGLICHKEITEN (auswendig!)

$$\text{Versuche} = \frac{(n-1)!}{2} \quad (\text{symmetrisches TSP})$$

# Knoten	# Möglichkeiten
4	$(4-1)! = 6$
5	$(5-1)! = 24$
6	$(6-1)! = 120$
10	$(10-1)! = 362.880$
20	$(20-1)! = 121.645.100.408.832.000$

⚠ „Das Problem ist **NUR SEHR AUFWÄNDIG** zu lösen.“ 🔑 „In der Informatik möchte man jedoch **nur Algorithmen einsetzen, deren Laufzeit in Bezug zur Anzahl der betrachteten Objekte HÖCHSTENS POLYNOMIAL wächst.**“

★★★★★ DIE PROBLEMKLASSEN P und NP

Klasse	Definition
P	Ein Problem gehört zur Klasse P, wenn es ein $k \in \mathbb{N}$ aus der Menge $O(x^k)$ gibt, so dass dieses mit einem DETERMINISTISCHEN Algorithmus in POLYNOMIALER ZEIT lösbar ist. ★ Die Anzahl der Operationen lässt sich durch ein POLYNOM von n darstellen.
NP	Ein Problem gehört zur Klasse NP, wenn es ... mit einem deterministischen Algorithmus NICHT in polynomialer Zeit lösbar ist. ★ NP-Probleme benötigen einen NICHT-DETERMINISTISCHEN Algorithmus. Die Anzahl der Operationen lässt sich NICHT durch ein Polynom von n darstellen.

$$\underbrace{O(1) \quad O(\log n) \quad O(n) \quad O(n^2) \quad O(n^3)}_{\text{P-Problem}} \quad \underbrace{O(2^n) \quad O(n!)}_{\text{NP-Problem}}$$

P = „Easy to find“ NP = „Easy to check“

★ „Bisher ist KEIN Algorithmus bekannt, der dieses Problem in der Klasse P einordnet. Diese Art von Problem, bei dem Lösungsalgorithmen alle mindestens ein EXPONENTIELLES WACHSTUM haben, bezeichnet man als ALGORITHMISCH UNLÖSBAR.“

★★★ MERKE DAS ABER: „JEDER ZUFÄLLIG AUSGEWÄHLTE ZYKLUS ist in POLYNOMIALER Laufzeit ÜBERPRÜFBAR, ob er ein Zyklus ist, zu dem jeder Knoten des Graphen gehört, und ob seine Länge (Summe aller Kanten) kleiner als eine geforderte Länge ist.“ ← Das ist genau die Definition von NP!

★ Wie löst man ein NP-schweres Problem?

Möglichkeit	Beschreibung
1	Es gibt einen effizienten Algorithmus, den man erst noch finden muss
2	Es gibt KEINEN effizienten Algorithmus — Nachweis durch eine untere Laufzeitschranke (extrem schwer) ODER Nachweis, dass das Problem zu einer Klasse von Problemen gehört, die scheinbar schwer zu lösen sind

★★ Exakte Verfahren vs. Approximationsverfahren

	① Exakte Verfahren	② Approximationsverfahren	③ Heuristik
Laufzeit	exponentiell	polynomiell	polynomiell
Voraussetzung	Eingabegrößen hinreichend klein ; exakte Lösung (optimales Ergebnis) notwendig ; Rechenkapazität verfügbar	wenn ① nicht möglich ist	—
Garantie	✓ optimal	★ GÜTEGARANTIE — liefert eine wahrscheinlich gute Lösung	⚠ KEINE Gütegarantie — Näherungsverfahren

Die 6 TSP-Lösungsverfahren des Kurses: Paarungsprobleme (Matching) · Nearest-Neighbour-Algorithmus · Greedy-Algorithmus · Minimum-Spanning-Tree-Heuristik · k -Opt · Christofides-Algorithmus

G.16 ★★ PAARUNGSPROBLEME (MATCHING)

Definition: „Unter dem **Paarungsproblem** versteht man die **Bildung von PAAREN von Objekten** in einem Graphen, in dem die Objekte, die möglicherweise zu Paaren zusammengefasst werden können, **durch Kanten miteinander verbunden sind**.“ ⚠ „Das Problem besteht darin, dass die Kanten meist **PRÄFERENZEN** repräsentieren, die untereinander **KOLLIDIEREN** können.“

📝 Beispiele:

- **Zuordnung von Arbeitssuchenden**, die jeweils spezifische Qualifikationen haben
- **Zuordnung zwischen den Anschlüssen zweier Netzwerke/Haushalte**
- **Optimale Zuordnung von freien Speicherplätzen** für mehrere Datenmengen mit spezifischen Größen und Eigenschaften
- **Zuordnung von Pärchen (Heiratssatz)**

Vorgehen: *Man macht aus den Objekten des Problems Punkte des Graphen und teilt den Graphen in zwei disjunkte Mengen V_1 und V_2 auf. Die Kanten gehen grundsätzlich immer von einem Punkt aus V_1 zu einem Punkt aus V_2 — der Graph ist **BIPARTIT** („zweigeteilt“).*

★ Definitionen

Begriff	Definition
Matching	Ein Teilgraph M von G heißt Matching genau dann, wenn in M JEDER PUNKT HÖCHSTENS DEN GRAD 1 hat — d. h. die Zerlegung erfolgt in so viele Teilgraphen, dass zwischen je zwei Knoten nur eine Kante existiert
PERFEKTE Paarung	zusätzlich: $2 \cdot M = V $
MAXIMALE S Matching	Wenn es NICHT MÖGLICH ist, einer ECHTEN OBERMENGE von V_1 -Punkten aus M einen Partner zuzuordnen: $V_1 \cap V_M \subseteq V_1 \cap V_{M'} \rightarrow V_1 \cap V_M = V_1 \cap V_{M'}$

★★ Der UNGARISCHE ALGORITHMUS

Harold W. KUHN — Veröffentlichung „*The Hungarian Method for the Assignment Problem*“. Die Arbeit basiert auf den Ideen **ungarischer Mathematiker: Dénes KÖNIG** (Graphentheorie) und **Eugene EGervÁRY** (Kombinatorische Optimierung).

★ Der ERWEITERNDE WEG:

Ein Weg ζ (zeta) in G heißt **ERWEITERNDER WEG**, falls: (i) **Anfangs- und Endpunkt des Weges ζ gehören NICHT zum Matching M** (ii) **Die Kanten von ζ sind ABWECHSELND Element von M bzw. KEIN Element von M** (Wege mit Eigenschaft (ii) werden auch als **ALTERNIERENDE Wege** bezeichnet.)

★★ Die ERWEITERUNGSREGEL für ein Matching:

„**ENTFERNE** alle Kanten aus dem **ALTEN Matching**, die auch Teil des erweiternden Weges sind, und **FÜGE** alle **ANDEREN** Kanten des erweiternden Weges zu dem Matching **HINZU**.“

Fazit des Algorithmus:

- **Zu Beginn kann man ohne Probleme Knoten einander zuordnen**
- **Anschließend muss man mit Hilfe erweiternder Wege bereits getroffene Zuordnungen REVIDIEREN**, um das Matching erweitern zu können
- **Gegen Ende sieht man mit Hilfe eines Baumes, dass das Matching NICHT MEHR ERWEITERT werden kann** — Knoten, die zum wachsenden Baum gehörten, wurden nicht mehr zugelassen (andernfalls hätte man **Zyklen** erzeugt)

☺ **Das Fazit des Dozenten:** „Wenn getanzt wird und jede Frau darauf besteht, nur mit einem Mann zu tanzen, der ihr sympathisch ist, bleibt eine der Frauen ohne Partner.“

Laufzeiten:

Algorithmus	Komplexität
Ungarischer Algorithmus	$O(V ^3)$
Algorithmus von Hopcroft-Karp	$O(E \cdot \sqrt{ V })$
Blossom-Algorithmus	$O(V ^2 \cdot E)$

G.17 ★★ TSP-HEURISTIKEN

① NEAREST-NEIGHBOR-ALGORITHMUS

- ① Wähle einen beliebigen Knoten aus
- ② Wähle die Kante mit dem GERINGSTEN Gewicht vom Startknoten aus
- ③ Der neue Knoten wird als NEUER Startknoten betrachtet
- ④ Von diesem Knoten analog verfahren, bis alle Knoten enthalten sind

⚠ „Dieses Beispiel zeigt, dass der Algorithmus in diesem Fall NICHT FUNKTIONIERT! Allgemein kann man sagen, dass der Nearest-Neighbor-Algorithmus NICHT IN JEDEM FALL funktioniert.“
 Laufzeit: $O(|V|^2)$

② GREEDY-ALGORITHMUS (für TSP)

- ① Greift sich immer die KÜRZESTE Kante des Graphen heraus, OHNE AUF DAS GESAMTERGEBNIS ZU ACHTEN
- ② Beginnt mit einem leeren Baum und der Sortierung aller Kanten nach Gewicht
- ③ Beginnend mit der kürzesten Kante werden alle Kanten durchlaufen und hinzugefügt
- ④ Nach jedem Hinzufügen wird geprüft:
 - (i) ob ein ZYKLUS entstanden ist ODER
 - (ii) ob die zugehörigen Knoten einen $\text{GRAD} > 2$ haben
 → Trifft (i) oder (ii) zu, wird die Kante wieder ENTFERNT; weiter mit ③
- ⑤ Wenn ALLE Knoten den $\text{GRAD} 2$ haben → fertig

⚠ „Der Greedy-Algorithmus funktioniert NICHT IN JEDEM FALL — insbesondere, wenn der Graph ZU WENIGE KANTEN besitzt!“

③ ★★ MST-HEURISTIK (Minimum Spanning Tree)

- ① Konstruiere einen MINIMALEN SPANNBAUM vom Graphen G
- ② VERDOPPELE alle Kanten im Spannbaum
- ③ Suche einen EULERKREIS im modifizierten Spannbaum und ersetze DOPPELT BESUCHTE Knoten durch DIREKTE Kanten

 **Beispiel:** $1 \rightarrow 4 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 2 \rightarrow 5 \rightarrow 2 \rightarrow 1 \Rightarrow 1 \rightarrow 4 \rightarrow \sim 1 \sim \rightarrow 2 \rightarrow 3 \rightarrow \sim 2 \sim \rightarrow 5 \rightarrow \sim 2 \sim \rightarrow$

1

④ ★★ CHRISTOFIDES-HEURISTIK

- ① Berechne den MINIMALEN SPANNBAUM
- ② Bestimme im Spannbaum die MINIMAL PERFEKTE PAARUNG (Minimal Cost Matching, MCM) zwischen Knoten mit UNGERADEM GRAD
- ③ Füge diese Kanten dem Baum hinzu → EULERSCHER Graph
- ④ Konstruiere eine EULERTOUR und anschließend einen HAMILTONKREIS ab einem beliebigen Knoten
- ⑤ Ersetze dabei bereits besuchte Knoten durch DIREKTE Verbindungen zum nächsten noch nicht besuchten Knoten

 **Beispiel:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 1 \Rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \sim 2 \sim \rightarrow 5 \rightarrow 4 \rightarrow 1$

Lösungsart	Komplexität
Exakte Lösungen	(exponentiell)
Approximative Lösungen	$O(V \cdot \log(V))$

⑤ ★★ 2-OPT und k -OPT (VERBESSERUNGSheuristik!)

★ „Alle bisher gezeigten Algorithmen sind KONSTRUKTIONSheURISTIKEN, die versuchen, eine optimale Tour zu KONSTRUIEREN.“ 💡 „Durch **LOKALE OPTIMIERUNGSVERFAHREN** können die heuristisch gewonnenen Lösungen etwas verbessert werden. Sie werden auch als **AUSTAUSCH- oder VERBESSERUNGSheURISTIK** bezeichnet.“

Idee: Eine **bestehende TSP-Lösung** wird **SCHRITTWEISE VERBESSERT**, bis keine Verbesserung mehr erreicht werden kann. Mittels einer **lokalen Suche** wird geprüft, ob eine geringfügige lokale Veränderung bereits zu einer besseren Lösung führt. **Ziel:** Verkürzung der Tour durch **Austausch von Kanten**, wobei sich eine **neue GESCHLOSSENE Tour** ergeben muss. Beim k -Opt-Verfahren werden k **Kanten aus der aktuellen Tour ENTFERNT** und durch **geeignete andere k Kanten ERSETZT**.

- ① Entnimm aus der TSP-Lösung k Kanten
- ② Verbinde die "freien" Knoten mit neuen Kanten mit möglichst kürzerer Strecke
- ③ Wiederhole ② für alle Kanten
- ④ Sind keine weiteren Austauschmöglichkeiten mit bis zu k Kanten mehr gegeben, um die Tourlänge zu reduzieren, so wird diese Tour als k -OPTIMAL bezeichnet

⚠ „Für (ca.) $k > 4$ entsteht bereits ein **HOHER Rechenaufwand**. Für große k : **BRUTE-FORCE-Aufwand**.“

G.18 ★★ Die Verbesserung von Christofides (2020)

 „Computer Scientists Break the ‚Traveling Salesperson‘ Record“ — „Finally, there's a better way to find approximate solutions to the notorious optimization problem.“

Nicos Christofides (1976):

- Sein Algorithmus fand Lösungen, die **50 % länger** sind als die besten bekannten Algorithmen zu dieser Zeit
- Fand einen Weg, **Knoten (Städte)** mit **UNGERADER Kantenanzahl effizient zu verbinden**
- **Informatiker erwarteten, dass sein einfacher Algorithmus schnell verbessert werden könnte** — es dauerte **44 Jahre!**

⚠ **Wo hat der Algorithmus seine Schwachstelle?**

- Der **MINIMALE SPANNBAUM** ist **NICHT IMMER DAS BESTE RÜCKGRAT (backbone)**
- **Hat der Baum viele Äste, muss jeder Knoten am Ende solch einer Kante mit einer anderen Stadt übereinstimmen, die wiederum eine Vielzahl an Verbindungen hervorruft**

💡 **Die neue Idee:**

„Vielleicht lässt sich Christofides' Algorithmus verbessern, indem man **nicht den minimalen Spannbaum als Grundlage verwendet, sondern einen ZUFALLS-SPANNBAUM aus einer Menge sorgfältig ausgewählter Kanten**. Die Auswahl einer Kante erfolgt so, dass **eine gegebene Kante im Spannbaum der Wahrscheinlichkeit entspricht, mit der der Kantenanteil in der besten anteiligen Route auftaucht**. Anschließend wird der Zufalls-Spannbaum mit dem Christofides-Algorithmus bearbeitet.“

Der Zahlenvergleich des Dozenten:

Verfahren	Baum-Länge	Rundreise-Länge
Christofides	5,4	8,4
Christofides — Alternative	5,9	★ 6,8
Neue Methode	5,6	7,6

⚠ **Einschränkungen:** „Funktioniert **nur mit Netzwerken, bei denen Kanten die selbe Länge haben**. Die Wissenschaftler konnten bisher **noch keine Möglichkeit finden, ihre Lösung zu verallgemeinern**.“ ☹
Anna Karlin: „If we have to add a super expensive edge to the matching then we're screwed, basically.“ ☹
 Um wie viel ist der Algorithmus schneller? 50 % - **0,2 Milliardenstel einer Billionstel einer Billionstel von einem Prozent** ☹ **David Williamson (Cornell University):** „The new result is the **first step towards showing that the frontiers of efficient computation are in fact BETTER than what we thought**.“

G.19 ★★ BACKTRACKING-ALGORITHMEN

⚠ „Greedy-Algorithmen haben das Problem, dass sie oft ‚FEHLSCHLAGEN‘ und damit kein gewünschtes Ergebnis liefern.“ → 💡 **Lösung: Backtracking-Algorithmen**

Definition

„Backtracking-Algorithmen bauen aus **TEILLÖSUNGEN** schrittweise eine **GESAMTLÖSUNG** auf. Führt eine verfolgte Lösungsstrategie zu **KEINER** Lösung, so wird der **LETZTE SCHRITT** (die letzten Schritte) **RÜCKGÄNGIG** gemacht und es wird ein **ALTERNATIVER** Lösungsweg gesucht.“
 🗝 Man spricht auch vom **TRIAL-AND-ERROR-Prinzip** (Versuch-und-Irrtum-Prinzip).

★ **Die Garantie:** „Mit dieser Strategie ist **sichergestellt, dass ALLE in Frage kommenden Lösungswege ausprobiert werden können**. D. h. eine vorhandene Lösung wird **entweder GEFUNDEN** oder es kann **DEFINITIV** ausgesagt werden, dass **KEINE Lösung existiert**.“

BACKTRACKING(Stufe, Vektor)

1. wiederhole, solange es noch neue Teil-Lösungsschritte gibt:
 - a) wähle einen neuen Teil-Lösungsschritt;
 - b) falls Wahl gültig ist:
 - I) erweitere Vektor um Wahl;
 - II) falls Vektor vollständig ist, return true // Lösung gefunden
 - sonst: falls (BACKTRACKING(Stufe+1, Vektor)) return true
 - sonst mache Wahl rückgängig // SACKGASSE (Backtracking)
2. Da es keinen neuen Teil-Lösungsschritt gibt: return false // Keine Lösung

Vorgehen: ① Führe den Algorithmus aus, bis ein **DEAD END** (Sackgasse) erreicht wird · ② Kehre zum **letzten Schritt** zurück und teste einen **anderen Folgeschritt** · ③ Versuche eine **neue gültige Teillösung** zu finden

Bildhaft: „Den Aufbau von Teillösungen kann man sich wie **Knoten in einem BAUM** vorstellen. Jeder Knoten symbolisiert gewissermaßen **mögliche Folgeschritte**.“

★ **Laufzeit:** Es handelt sich um eine **TIEFENSUCHE** — sucht **höchstens einmal im gleichen Dead End**. Bei maximal k Verzweigungen und maximaler Tiefe n :

$$1 + k + k^2 + k^3 + \dots + k^n = \boxed{O(k^n)}$$

⚠ „Je größer die Suchtiefe n , desto länger dauert die Suche. Daher ist Backtracking **PRIMÄR FÜR PROBLEME MIT EINEM KLEINEN LÖSUNGSBAUM** geeignet.“

G.20 ★ TSP in der Praxis — lohnt sich der Aufwand?

 **Das Beispiel des Dozenten (Paketdienst):**

- **55.000 Fahrten pro Tag** mit je **120 Stationen**
- **Optimierung nach Verkehrsfluss, Abholzeiten, Lieferfristen, Kundenklassen**
- **Zusätzlich: LINKSABBIEGEN VERMEIDEN**
- ★★ 📦 **30.000.000 US-\$ ERSPARNIS PRO JAHR**, wenn jeder Lieferwagen nur **1 MEILE PRO TAG WENIGER** fährt!

G.21 ★★★★★ Die große Graphen-Laufzeittabelle

Algorithmus	Problem	Komplexität	Besonderheit
Breitensuche (BFS)	Traversieren	$O(V + E)$	Queue · Level-Order
Tiefensuche (DFS)	Traversieren	$O(V + E)$	Stack/Rekursion · Pre-Order
Hierholzer	Euler-Zyklus	★ $O(E + V)$	schnellster Euler-Algorithmus
Fleury	Euler-Zyklus	$O(E ^2)$	
Euler	Euler-Zyklus	$O(V ^3)$	
Kruskal (naiv)	MST	$O(V ^2)$	
Kruskal (Union-Find)	MST	★ $O(E \cdot \alpha(V))$	braucht Union-Find
Prim (naiv)	MST	$O(V ^2)$	
Prim (Fibonacci-Heap)	MST	★ $O(E \cdot \log V)$	braucht Prioritätswarteschlange
Dijkstra (naiv)	Single-Source	$O(V ^2)$	⚠ keine negativen Kanten
Dijkstra (Fibonacci-Heap)	Single-Source	★ $O(E + V \log V)$	
Bellman-Ford	Single-Source	$O(V \cdot E)$	✅ negative Kanten + erkennt neg. Zyklen
Floyd-Warshall	All-Pairs	$O(V ^3)$	✅ negative Kanten · ⚠ erkennt keine neg. Zyklen
Ungarischer Algorithmus	Matching	$O(V ^3)$	
Hopcroft-Karp	Matching	$O(E \sqrt{ V })$	
Blossom	Matching	$O(V ^2 E)$	
Nearest Neighbor	TSP	$O(V ^2)$	⚠ funktioniert nicht immer
Christofides / MST-Heuristik	TSP	$O(V \log V)$	approximativ
Backtracking	allgemein	$O(k^n)$	Tiefensuche über den Lösungsbaum
TSP exakt	TSP	$\frac{(n-1)!}{2}$	⚠ NP

▼ ? **Selbsttest Teil G**

1. Erklären Sie das Königsberger Brückenproblem und Eulers Argumentation.
2. Definieren Sie einen Graphen. Was ist Ψ ?
3. Was ist der Unterschied in Ψ bei gerichteten und ungerichteten Graphen?
4. Wie berechnet man den Grad in ungerichteten Graphen? Was ist mit Schlingen?
5. Was sind Außengrad und Innengrad?
6. Definieren Sie: zusammenhängend, stark verbunden, Zyklus.
7. Nennen Sie die 7 speziellen Graphen mit Definition.
8. Wie lautet die Eulersche Polyederformel? Wozu braucht man planare Graphen?
9. Adjazenzmatrix vs. Adjazenzliste: Speicher und Zugriffszeiten?
10. Nennen Sie die Bedingungen für Euler-Weg und Euler-Zyklus (ungerichtet + gerichtet).
11. Was ist ein BREAKOUT bei der Matrix-Konstruktion?
12. Wie konstruiert man einen Euler-Weg aus einem Euler-Zyklus?
13. BFS vs. DFS: Datenstruktur, Baumdurchlauf, Laufzeit?
14. Nennen Sie die 3 Gründe, warum eine MST-Lösung ein Baum sein muss.
15. Wie lautet die Cayley-Formel? Wie viele Spannbäume hat ein Graph mit 6 Knoten? (1296)
16. Was ist ein Greedy-Algorithmus? Nennen Sie 3 Eigenschaften.
17. Beschreiben Sie Kruskal in 4 Schritten.
18. Nennen Sie die 3 Union-Find-Operationen.
19. Nennen Sie die 4 Verbesserungen der Union-Find-Datenstruktur.
20. Was ist die Ackermann-Funktion und warum ist ihre Inverse relevant?
21. Nennen Sie 3 Unterschiede zwischen Kruskal und Prim.
22. Nennen Sie die 3 Problemvarianten der kürzesten Pfade.
23. Was passiert bei negativen Zyklen? Nennen Sie die 3 Zyklus-Eigenschaften.
24. Warum speichert man kürzeste Pfade als Baum und nicht als Listen?
25. Beschreiben Sie Dijkstra. Warum funktioniert er nicht mit negativen Kanten?
26. Nennen Sie 4 Unterschiede Bellman-Ford $\leftrightarrow$ Dijkstra.
27. Schreiben Sie die Floyd-Warshall-Formel auf.
28. Euler vs. Hamilton: Was wird jeweils genau einmal durchlaufen? Komplexität?
29. Wie viele Rundreisen gibt es beim TSP mit 10 Knoten?
30. Definieren Sie P und NP. Was bedeutet „easy to find/easy to check“?
31. Nennen Sie die 3 Lösungsarten (exakt, approximativ, heuristisch) mit ihren Garantien.
32. Was ist ein erweiternder Weg? Wie lautet die Erweiterungsregel?
33. Beschreiben Sie die MST-Heuristik und die Christofides-Heuristik.
34. Was ist der Unterschied zwischen Konstruktions- und Verbesserungsheuristik?
35. Wie funktioniert Backtracking? Welche Laufzeit?
36. Wie viel spart der Paketdienst pro Jahr pro Meile?

TEIL H — KAPITEL 08: FORMALE SPRACHEN UND AUTOMATENTHEORIE ★★

Inhalte: Sprachen (allg.) · Deterministischer endlicher Automat (DEA) · Nichtdeterministischer endlicher Automat (NEA) · Äquivalenz von DEA und NEA · Reguläre Ausdrücke · Pumping-Lemma · Sprachentypen

H.1 ★★ Die Geschichte der Automatentheorie

Automatentheorie = „Studium abstrakter Rechenggeräte oder Maschinen“

Zeit	Persone	Beitrag
30er-Jahre	★ Alan Turing	Definierte eine abstrakte Maschine, die über sämtliche Fähigkeiten heutiger Computer verfügte → TURING-MASCHINE . Wollte die Grenze zwischen dem, was eine Rechenmaschine berechnen kann, und dem, was sie nicht kann, genau beschreiben (berechenbar / nicht berechenbar). ★ Seine Schlussfolgerungen treffen nicht nur auf seine abstrakten Turing-Maschinen zu, sondern auch auf die heutigen REALEN Maschinen.
50er-Jahre	★ Noam Chomsky	Linguist — FORMALE GRAMMATIKEN → CHOMSKY-HIERARCHIE . Enge Verwandtschaft zu abstrakten Automaten → Grundlage einiger wichtiger Softwarekomponenten (COMPILER)
1969	★ Stephen Cook	Führte Turings Untersuchung fort — konnte die Probleme, die sich EFFIZIENT mit Computern lösen lassen, von jenen trennen, die PRINZIPIELL lösbar sind, deren Lösung jedoch in der Praxis SO ZEITAUFWÄNDIG ist, dass Computer sich lediglich zur Lösung SEHR KLEINER Beispiele eignen → nicht handhabbar / NP-HART

★ Wozu braucht man die Automatentheorie? (4 Anwendungen)

1. **Software zum ENTWURF und zur ÜBERPRÜFUNG des Verhaltens DIGITALER SCHALTSTRECKEN**
2. ★ **Die „LEXIKALISCHE ANALYSEKOMPONENTE“ von COMPILERN** — Aufschlüsseln eines Eingabetextes in **logische Einheiten** wie Schlüsselwörter und Satzzeichen
3. **Software zum DURCHSUCHEN umfangreicher Texte** — Vorkommen von Wörtern oder Mustern
4. **Software zur VERIFIZIERUNG** — (Kommunikations-)Protokolle

H.2 ★★★★★ DIE DEFINITIONEN (das Fundament!)

#	Begriff	Definition
1	ALPHABET Σ	Ein Alphabet Σ ist eine ENDLICHE, NICHT LEERE MENGE . Die Elemente heißen Buchstaben, Zeichen oder Symbole .
2	KLEENESCHE HÜLLE Σ^*	Die Menge ALLER Wörter beliebiger Länge, EINSCHLIESSLICH des LEEREN Wortes ε , das Länge Null hat
3	TRANSITIVE HÜLLE Σ^+	Die Menge aller Wörter beliebiger Länge, die das leere Wort ε NICHT enthält
4	ZEICHENREIHE ω	Eine ENDLICHE FOLGE von Symbolen eines bestimmten Alphabets. <i>Synonyme: Wort, Zeichenkette, String</i> . Die leere Zeichenreihe enthält kein Symbol $\rightarrow \varepsilon$
5	LÄNGE $ \omega $	Die Anzahl der Positionen für die Symbole einer Zeichenreihe
6	POTENZ Σ^k	Die Menge aller Zeichenketten einer bestimmten LÄNGE k über Σ . Es gilt: $\Sigma^0 = \{\varepsilon\}$
7	★ SPRACHE L	Eine MENGE VON ZEICHENREIHEN (Wörtern) aus Σ^* . Ist Σ ein Alphabet und $L \subseteq \Sigma^*$, dann ist L eine Sprache über Σ .
8	★ FORMALE SPRACHE	Eine formale Sprache ist eine TEILMENGE $L \subseteq \Sigma^*$ über einem ENDLICHEN Alphabet Σ .

🔑 **Merkhilfe für die Hüllen:** ***** $\rightarrow$ Zeichen kann 0-mal oder beliebig oft vorkommen $\cdot + \rightarrow$ Ein oder mehrere Vorkommen

📝 **Beispiele Alphabet:** $\Sigma = \{0, 1\} \rightarrow$ binäres Alphabet $\cdot \Sigma = \{a, b, \dots, z\} \rightarrow$ alle Kleinbuchstaben $\cdot \Sigma = \{0, 1, \dots, 9\} \rightarrow$ alle Ziffern $\cdot$ alle ASCII-Zeichen

📝 **Beispiel Potenz ($\Sigma = \{0, 1\}$):**

Σ^0	$\{\varepsilon\}$
Σ^1	$\{0, 1\}$ (Worte, nicht Symbole!)
Σ^2	$\{00, 01, 10, 11\}$
Σ^3	$\{000, 001, 010, 011, 100, \dots\}$

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, \dots\} \quad \Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \dots$$

$$\Sigma^* = \{\varepsilon\} \cup \Sigma^+$$

★ Konkatenation von Zeichenreihen

Seien $x = a_1 a_2 \dots a_i$ ($|x| = i$) und $y = b_1 b_2 \dots b_j$ ($|y| = j$), dann ist $xy = a_1 a_2 \dots a_i b_1 b_2 \dots b_j$ die **Konkatenation**.

📝 $\Sigma = \{0, 1\}, x = 110, y = 001: xy = 110001 \cdot yx = 001110 \cdot x\varepsilon = \varepsilon x = x = 110$

! ★ ★ ★ Die wichtigen Feinheiten (Prüfungsfallen!)

- Σ^* ist Sprache für JEDES Alphabet Σ
- Die LEERE SPRACHE $\emptyset$ ist Sprache über ALLE Σ — $\emptyset$ enthält KEINE Zeichenketten
- $\{\varepsilon\}$ enthält DIE LEERE ZEICHENKETTE und ist Sprache über jedes Alphabet
- ★ ★ ★ $\{\varepsilon\} \neq \emptyset$!!!
- ★ Sprachen können UNENDLICH VIELE Zeichenreihen enthalten. Ein ALPHABET muss dagegen ENDLICH sein.

H.3 ★ ★ ★ Operationen auf Sprachen

Operation	Definition	Beispiel
Vereinigung $L \cup M$	Die Menge aller Zeichenreihen, die entweder in L oder M oder in beiden enthalten sind	$L = \{001, 10, 111\}, M = \{\varepsilon, 001\} \rightarrow L \cup M = \{\varepsilon, 10, 001, 111\}$
Verkettung (Konkatenation) LM	Die Menge aller Zeichenreihen, die gebildet werden, indem eine Zeichenreihe aus L mit einer beliebigen Zeichenreihe aus M verkettet wird: $\forall v \in L, w \in M \Rightarrow LM \rightarrow vw$	$LM = \{001, 10, 111, 001001, 10001, 111001\}$
Kleenesche Hülle L^*	Die Menge aller Zeichenreihen, die durch Verkettung einer BELIEBIGEN ANZAHL von Zeichenreihen aus L gebildet werden: $L^* = \bigcup_{i \geq 0} L^i$ mit $L^0 = \{\varepsilon\}, L^1 = L, L^n = LL \dots L$	$L = \{0, 11\}: L^0 = \{\varepsilon\} \cdot L^1 = \{0, 11\} \cdot L^2 = \{00, 011, 110, 1111\} \cdot L^3 = \{000, 0011, 0110, 01111, 1100, 11011, 11110, 111111\}$

★ Spezialfälle:

- Sei L die Menge aller aus Nullen bestehenden Zeichenketten $\Rightarrow L^* = L$
- Sei $L = \{\varepsilon\} \Rightarrow L^* = \{\varepsilon\} \rightarrow L$ hat eine ENDLICHE Hülle

★ Mengenschreibweise

$$\{w \mid \text{Aussage über } w\}$$

Beispiele:

- $\{w \mid w \text{ enthält eine gleiche Anzahl von Nullen und Einsen}\}$
- $\{w \mid w \text{ ist eine binäre ganze Zahl, die eine Primzahl ist}\}$
- $\{w \mid w \text{ ist ein syntaktisch fehlerfreies C-Programm}\}$
- $\{0^n 1^n \mid n \geq 1\}$ — „die Menge von 0 hoch n 1 hoch n , derart dass $n \geq 1$ “
- $\{0^i 1^j \mid 0 \leq i \leq j\} = \{01, 0011, 000111, \dots\}$ — „einige (oder keine) Nullen, gefolgt von mindestens ebenso vielen Einsen“

H.4 ★ ★ ★ GRAMMATIK und die CHOMSKY-HIERARCHIE

★ ★ ★ Definition Grammatik (auswendig — 4 Bestandteile!)

Eine Grammatik besteht aus: (i) einem endlichen ALPHABET Σ von TERMINALSYMBOLEN $\rightarrow$ Kleinbuchstaben (ii) einer endlichen Menge V von NICHTTERMINALSYMBOLEN (Variablen) mit $V \cap \Sigma = \emptyset \rightarrow$ Großbuchstaben (iii) einem STARTSYMBOL $S \in V$ (iv) einer endlichen Menge P von PRODUKTIONEN $u \rightarrow v$ mit $u \in (V \cup \Sigma)^* V (V \cup \Sigma)^*$ und $v \in (V \cup \Sigma)^*$

★ Produktion / Ableitungsregel

- Sie definiert eine ERSETZUNGSREGEL für ein Zeichen eines Wortes aus einer beliebigen Mischung von Variablen und Terminalsymbolen
- Die Ersetzung BEGINNT beim STARTSYMBOL S
- Produktionen können so lange HINTEREINANDER GESCHALTET werden, bis KEINE REGEL MEHR ANWENDBAR ist
- ★ Besteht am Ende das Wort NUR NOCH AUS TERMINALSYMBOLEN, dann ist dies ein WORT der durch die Grammatik erzeugten SPRACHE

Zwei Schreibweisen:

Möglichkeit 1:

$P = \{ S \rightarrow aA \\ A \rightarrow aS \\ A \rightarrow a \}$

Möglichkeit 2:

$P = \{ S \rightarrow aA \\ A \rightarrow aS \mid a \}$

🔑 „Die **Gesamtheit der auf die Menge der Wörter wirkenden Ableitungsregeln** wird auch als **SYNTAX** oder **ABLEITUNGSSTRUKTUR** bezeichnet.“

★★ Das WORTPROBLEM

„Ein Problem in der Automatentheorie ist die Frage, OB EINE GEGEBENE ZEICHENREIHE ELEMENT EINER BESTIMMTEN SPRACHE IST. Dieses Problem wird als WORTPROBLEM bezeichnet.“

Wenn $\omega \in \Sigma^*$, ist dann $\omega \in L$?

Wesentliche Beiträge zur Klassifizierung formaler Sprachen:

- **Axel THUE** (norwegischer Mathematiker) — **Wortproblem**
- **Noam CHOMSKY** (amerikanischer Linguist) — **Chomsky-Grammatiken** (Sprachwissenschaft + Informatik)

★ „Für praktische Zwecke ist der bisherige Begriff der formalen Sprache **noch viel zu weit gefasst**. Im Folgenden betrachten wir daher nur formale Sprachen, **deren PRODUKTIONSREGELN EINGESCHRÄNKT sind**.“

★★★★ DIE CHOMSKY-HIERARCHIE (die zentralste Tabelle des Kapitels!)

Typ	Sprachtyp	Grammatik	★ Akzeptierende Maschine
Typ 0	semi-entscheidbare / rekursiv aufzählbare Sprachen	ALLE Grammatiken	★ TURINGMASCHINE
Typ 1	KONTEXTSENSITIVE Sprachen	Kontextsensitiv e	★ (nichtdeterministische) LINEAR BESCHRÄNKTE Turingmaschine
Typ 2	KONTEXTFREIE Sprachen	Kontextfreie	★ KELLERAUTOMAT
Typ 3	REGULÄRE Sprachen	Reguläre	★ ENDLICHE AUTOMATEN

$$\text{Typ } 3 \subsetneq \text{Typ } 2 \subsetneq \text{Typ } 1 \subsetneq \text{Typ } 0$$

★ **Chomsky-Grammatik:** Eine Sprache L_i heißt vom **Typ** i ($i = 0, 1, 2, 3$), falls eine Chomsky-Grammatik G_i vom Typ i existiert mit $L(G_i) = L_i$. ★★ „Eine Sprache bleibt auch dann vom Typ i bestehen, wenn man dafür eine Grammatik vom Typ j mit $\$j$ angibt. D. h. eine reguläre Sprache kann eine kontextsensitive Grammatik definieren, aber NICHT UMGEKEHRT.“ ★ „ALLE Sprachen vom Typ 1, 2 und 3 sind ENTSCHEIDBAR — aber NICHT alle Sprachen vom Typ 0.“

★★★★ Die 4 Grammatiktypen im Detail

Chomsky-0 — ALLGEMEINE Grammatik

- Die Produktionsregeln unterliegen KEINEN Einschränkungen
- (rekursiv) aufzählbare Sprache — semi-entscheidbar
- Die zur Sprache gehörigen Wörter können durch einen Algorithmus NACHEINANDER ERZEUGT werden
- ⚠️ „Wenn ein Wort NICHT zur Sprache gehört, dann liefert der Automat AUCH KEINE ANTWORT darüber. D. h. man kann die Wörter der Sprache zwar AUZÄHLEN, aber NICHT die Wörter, die NICHT zur Sprache gehören. D. h. wiederum, dass es KEINE GRAMMATIK für die Sprache derjenigen Wörter gibt, die nicht zu dieser Sprache gehören.“

	Regel
Links vom Pfeil	Mindestens EIN Nichtterminal
Rechts vom Pfeil	Gar nichts (ε) ODER Terminale und Nichtterminale in beliebiger Folge und Anzahl

Chomsky-1 — KONTEXTSENSITIVE Grammatik

- Die Variable darf nur IM KONTEXT einer Zeichenkette ersetzt werden — der KONTEXT SELBST BLEIBT UNVERÄNDERT
- ★ Wörter dieser Form werden IMMER LÄNGER bzw. behalten ihre Länge bei (außer es gilt $S \rightarrow \varepsilon$) → MONOTONE Grammatik
- Akzeptierende Maschine: (nichtdeterministische) linear beschränkte Turingmaschine — die Bandlänge ist auf die Länge des Eingabeworts beschränkt

	Regel
Links vom Pfeil	Mindestens ein Nichtterminal · NIE MEHR ZEICHEN als rechts vom Pfeil
Rechts vom Pfeil	Terminale und Nichtterminale in beliebiger Folge

Chomsky-2 — KONTEXTFREIE Grammatik

- Die Produktionen hängen NICHT von einem Kontext ab → Klammersetzung
- Beschreibt die Sprache aller Wörter, in denen VERSCHIEDENE ZEICHEN GLEICH OFT vorkommen
- ★★ ALLE PROGRAMMIERSPRACHEN können hiermit beschrieben werden
- Akzeptierende Maschine: KELLERAUTOMAT

	Regel
Links vom Pfeil	★ IMMER NUR EIN EINZIGES Nichtterminal

Chomsky-3 — REGULÄRE Grammatik

- **Akzeptierende Maschine: ENDLICHE AUTOMATEN**
- **Alle Produktionen sind ENTWEDER RECHTSLINEAR ODER LINKSLINEAR:**

rechtslinear: $A \rightarrow xB$ linkslinear: $A \rightarrow Bx$ terminal: $A \rightarrow x$

★ „Wir arbeiten nur mit **RECHTSLINEAR**.“

	Regel
Links vom Pfeil	Immer nur ein einziges Nichtterminal
Rechts vom Pfeil	GENAU EIN Terminal ODER genau ein Terminal GEFOLGT VON genau einem Nichtterminal

H.5 ★★☆☆ AUTOMATEN

Definition

„Ein **AUTOMAT** kann als eine allgemeine **ALGEBRAISCHE STRUKTUR** aufgefasst werden, bei dem eine Menge von Elementen durch eine **VORSCHRIFT** miteinander verknüpft sind.“

★★★ **DAS 5-TUPEL (auswendig!)**

$$A = (Q, \Sigma, \delta, q_0, F)$$

Symbol	Bedeutung
Q	Menge von ZUSTÄNDEN
Σ	Menge von EINGABESYMBOLN
δ	ÜBERGANGSFUNKTION
q_0	STARTZUSTAND
F	Menge FINALER/AKZEPTIERENDER Zustände ($F \subseteq Q$)

Jeder Automat hat: einen Anfangszustand · mindestens einen akzeptierenden Zustand (Endzustand) · Übergänge zwischen den Zuständen ★ „Ein Startzustand darf auch **GLEICHZEITIG** ein Endzustand sein.“

★★ Die Übergangsfunktion δ

„Die Übergangsfunktion δ ordnet JEDEM PAAR (q_i, σ_j) aus Zustand q_i und Eingabezeichen σ_j einen NEUEN FOLGEZUSTAND q_k zu.“

$$\delta(q_i, \sigma_j) = q_k \quad \text{oder} \quad (q_i, \sigma_j) \rightarrow q_k$$

- Eine Übergangsfunktion bestimmt, WIE SICH DER ZUSTAND auf die Verarbeitung eines EINGABESYMBOLS hin VERÄNDERT
- Ist für eine Eingabe kein Übergang definiert, kann die LEERE MENGE eingetragen werden oder $(-)$
- ⚠️ „Die Übergangsfunktion δ MUSS NICHT UMKEHRBAR sein. D. h. aus dem aktuellen Zustand muss NICHT unbedingt auf den vorhergehenden Zustand geschlossen werden können.“

Zwei Darstellungsarten: ① ÜBERGANGSTABELLE · ② ÜBERGANGSDIAGRAMM

🔑 „Übergangsdigramme sind viel anschaulicher als Tabellen von Übergangsfunktionen.“

★★★★ Die ERWEITERTE Übergangsfunktion $\tilde{\delta}$

„Damit der Automat GANZE WÖRTER verarbeiten kann, und nicht nur ein einzelnes Symbol, muss die Übergangsfunktion δ ERWEITERT werden. Die erweiterte Übergangsfunktion $\tilde{\delta}$ beschreibt, was passiert, wenn man von einem beliebigen Knoten ausgeht und einer beliebigen FOLGE VON EINGABEN folgt.“

$$(i) \tilde{\delta}(q, \varepsilon) = q \quad (ii) \tilde{\delta}(q, \omega) = \delta(\tilde{\delta}(q, x), a)$$

wobei ω eine Zeichenreihe der Form $x a$ ist, a das LETZTE Symbol von ω und x die Zeichenreihe aller Eingabesymbole MIT AUSNAHME des letzten.

Berechnung in 2 Schritten: ① Berechne $\tilde{\delta}(q, x) = q_x$ — der Zustand nach Verarbeitung aller Symbole außer dem letzten · ② Berechne $\delta(q_x, a)$ — der Übergang auf das letzte Symbol

★ Unvollständige Darstellungsart (der FANGZUSTAND)

★ „Der Übersichtlichkeit halber lässt man den oder die FANGZUSTÄNDE oft WEG und vereinbart, dass ALLE nicht berücksichtigten Übergänge zu einem FANGZUSTAND führen. In der Übergangstabelle kann man dies durch einen STRICH $(-)$ kennzeichnen.“

📝 Beispiel: $L = \{0110\}$ mit q_{error} als Fangzustand · $L(A) = \{10^n 1 \mid n \in \mathbb{N}_0\}$

★ Kleine Bausteine

Ausdruck	Automat
a^*	Zustand mit Selbstschleife bei a (auch 0-mal)
a^+	ein a -Übergang, danach Selbstschleife
$a?$	ein optionaler a -Übergang

★★ ENDLICHER Automat

„Ein Automat gilt dann als **ENDLICH**, wenn die **ZUSTANDSMENGE** Q **UND** das **ALPHABET** Σ **ENDLICH** sind.“ *Endliche Automaten umfassen Zustände und Übergänge zwischen Zuständen, die in Reaktion auf EINGABEN erfolgen.*

	Definition
★ DETERMINISTISCH	„Der Automat kann nach jeder Eingabe von seinem aktuellen Zustand in GENAU EINEN Zustand übergehen.“
★ NICHTDETERMINISTISCH	„Der Automat kann GLEICHZEITIG IN MEHRERE ZUSTÄNDE übergehen.“

★ **Der Vorteil:** „Mit einer **endlichen Anzahl von Zuständen zu arbeiten**, bietet den Vorteil, dass das betrachtete System mit einer **FIXEN MENGE VON RESSOURCEN implementiert werden kann.**“

★★ Die 2 FOLGERUNGEN

Folgerung 1: „Unter all den Wörtern, die sich aus den Eingabezeichen bilden lassen, muss es für **JEDEN ENZZUSTAND** mindestens **EINEN Zustand** geben, der den Automaten von einem Startzustand — auch über Zwischenzustände — in einen **ENZZUSTAND** überführt.“

Folgerung 2: ★★ „Die Menge **ALLER** aus den Zeichen des Alphabets Σ bildbaren Wörter, die einen Automaten vom **ANFANGSZUSTAND** in einen **ENZZUSTAND** überführen, bildet die **AKZEPTIERTE SPRACHE** $L(A)$ des Automaten A .“

★★ Endliche Automaten und das WORTPROBLEM

„Ein konzipierter Automat für eine bestimmte Sprache L ist in der Lage, anhand eines Wortes $w \in \Sigma^*$ zu erkennen, ob dieses Wort zu L gehört oder nicht (**WORTPROBLEM**).“

Vorgehen: ① Die einzelnen Zeichen von w werden **NACHEINANDER** als Eingabezeichen auf den zu Beginn im Anfangszustand befindlichen Automaten „geladen“ — ⚠ „Es ist **NICHT** möglich, große Zeichenblöcke zu initiieren.“ ② Wenn nach Eingabe des **LETZTEN** Zeichens der Automat in einen **ENZZUSTAND** gelangt, ist $w \in L$ — andernfalls nicht.

★ Der Bezug zur Praxis (die schöne Analogie!)

Automaten-Element	Programm-Entsprechung
Q	Menge der Programmzeilen
Σ	Alle zulässigen Wörter (der Programmiersprache)
δ	Funktionsaufruf, Parameterübergabe
q_0	Zeile 0 des Programms ohne Variablen
F	Letzte Zeile des Programms (return)

H.6 ★★★★★ DEA — DETERMINISTISCHER ENDLICHER AUTOMAT

$$\text{DEA} = (Q, \Sigma, \delta, q_0, F) \quad \text{mit} \quad \boxed{\delta(q, a) = p \quad (q, p \in Q, a \in \Sigma)}$$

★ „Bei einem DEA wird die Übergangsfunktion δ als **EINDEUTIG** vorausgesetzt. D. h. ein Übergang resultiert in **GENAU EINEN** neuen Zustand — daher auch die Bezeichnung **DETERMINISTISCH**.“

★ Spracherkennung beim DEA

- ① Wir gehen davon aus, dass sich der DEA in seinem Startzustand q_0 befindet
- ② Mithilfe der Übergangsfunktion δ wird für JEDES Eingabesymbol der neue Zustand ermittelt, in den er wechselt
- ③ Wenn der LETZTE Zustand q_n ein ELEMENT VON F ist, dann wird die Eingabe **AKZEPTIERT**, andernfalls nicht

★★ Die SPRACHE eines DEA

$$L(A) = \{\omega \mid \tilde{\delta}(q_0, \omega) \in F\}$$

„Die Sprache von A ist die **MENGE DER ZEICHENREIHEN** ω , die vom Startzustand q_0 in einen **AKZEPTIERENDEN** Zustand führen.“ ★ „Wenn L für einen DEA A $L(A)$ ist, dann wird L als **REGULÄRE SPRACHE** bezeichnet.“

📝 ★★★★★ **BEISPIEL 1:** $\Sigma = \{0, 1\}$, $L = \{x01y \mid x, y \in \Sigma^*\}$

Die Denkweise des Dozenten:

„01 schon gelesen?“ — JA → nur noch akzeptierende Zustände
 |
 NEIN
 |
 „Eingabe einer 0?“ — JA → „Folgt eine 1?“ — JA → akzeptierend

Die 3 Zustände:

Zustand	Eingabe 0	Eingabe 1
q_0	Wechsel nach q_1	Bleibe in q_0
q_1	Bleibe in q_1	Wechsel nach q_2
q_2	Bleibe in q_2	Bleibe in q_2

📝 ★★★★★ **BEISPIEL 2:** $L = \{\omega \mid \omega \text{ enthält eine GERADE Anzahl von Nullen UND Einsen}\}$

Zustand	Bedeutung
q_0	Anzahl 0 GERADE, Anzahl 1 GERADE ← auch der akzeptierende Zustand!
q_1	Anzahl 0 gerade, Anzahl 1 UNGERADE
q_2	Anzahl 0 UNGERADE, Anzahl 1 gerade
q_3	Anzahl 0 ungerade, Anzahl 1 ungerade

$$A = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_0\})$$

★ **Die erweiterte Übergangsfunktion durchgerechnet für $\omega = 110101$:**

$$\begin{aligned}
 \tilde{\delta}(q_0, \varepsilon) &= q_0 \\
 \tilde{\delta}(q_0, 1) &= \delta(\tilde{\delta}(q_0, \varepsilon), 1) = \delta(q_0, 1) = q_1 \\
 \tilde{\delta}(q_0, 11) &= \delta(\tilde{\delta}(q_0, 1), 1) = \delta(q_1, 1) = q_0 \\
 \tilde{\delta}(q_0, 110) &= \delta(\tilde{\delta}(q_0, 11), 0) = \delta(q_0, 0) = q_2 \\
 \tilde{\delta}(q_0, 1101) &= \delta(\tilde{\delta}(q_0, 110), 1) = \delta(q_2, 1) = q_3 \\
 \tilde{\delta}(q_0, 11010) &= \delta(\tilde{\delta}(q_0, 1101), 0) = \delta(q_3, 0) = q_1 \\
 \tilde{\delta}(q_0, 110101) &= \delta(\tilde{\delta}(q_0, 11010), 1) = \delta(q_1, 1) = \mathbf{q_0} \in F \quad \checkmark \text{ akzeptiert!}
 \end{aligned}$$

H.7 ★★★★★ NEA — NICHTDETERMINISTISCHER ENDLICHER AUTOMAT

$$\delta(q, a) = p \quad (q \in Q, p \subseteq Q, a \in \Sigma)$$

★ „Bei einem NEA steht bei MINDESTENS EINEM Zustand nicht nur EIN Folgezustand q_k , sondern eine MENGE von Folgezuständen $\{q_{k1}, q_{k2}, \dots\}$ zur Verfügung. D. h. der NEA ist in der Lage, GLEICHZEITIG über MEHRERE Zustände zu verfügen.“ ★ „Die Übergangstabelle des NEA unterscheidet sich lediglich darin, dass die einzelnen Tabelleneinträge MENGEN angeben — auch wenn es sich um eine EINELEMENTIGE Menge handelt.“

★★ Die SPRACHE eines NEA

$$L(A) = \{\omega \mid \tilde{\delta}(q_0, \omega) \cap F \neq \emptyset\}$$

„Die Sprache von A ist die Menge ALLER Zeichenreihen ω aus Σ^* , sodass $\tilde{\delta}(q_0, \omega)$ MINDESTENS EINEN akzeptierenden Zustand enthält.“

🔑 **DER UNTERSCHIED IN EINEM ZEICHEN: DEA:** $\tilde{\delta}(q_0, \omega) \in F$ • **NEA:** $\tilde{\delta}(q_0, \omega) \cap F \neq \emptyset$

📝 ★ **BEISPIEL:** $L = \{x01 \mid x \in \Sigma^*\}$

$$\delta = \{ \delta(q_0, 0) = \{q_0, q_1\}, \quad \delta(q_0, 1) = \{q_0\}, \quad \delta(q_1, 0) = \emptyset, \quad \delta(q_1, 1) = \{q_2\}, \quad \delta(q_2, 0) = \emptyset, \quad \delta(q_2, 1) = \emptyset \}$$

Erweiterte Übergangsfunktion:

$$\begin{aligned}
 \tilde{\delta}(q_0, \varepsilon) &= \{q_0\} \\
 \tilde{\delta}(q_0, 0) &= \{q_0, q_1\} \\
 \tilde{\delta}(q_0, 00) &= \delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\} \\
 \tilde{\delta}(q_0, 001) &= \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{\mathbf{q_0}, \mathbf{q_2}\} \quad \checkmark
 \end{aligned}$$

🔑 **Die anschauliche Merkhilfe des Dozenten: „Der NEA ‚VERMUTET‘ bzw. KENNT den richtigen Übergang.“**

H.8 ★★★★★ ÄQUIVALENZ VON DEA UND NEA

„Wann sind zwei Automaten ÄQUIVALENT? — Wenn sie ein IDENTISCHES EIN-/AUSGABEVERHALTEN zeigen und damit DIESELBE SPRACHE akzeptieren/erkennen.“

★★★ **DER SATZ:** „JEDE Sprache, die sich durch einen NEA beschreiben lässt, kann AUCH durch einen DEA beschrieben werden. D. h. JEDER NEA lässt sich durch einen DEA und JEDER DEA durch einen NEA darstellen.“

!★★★ Die Zustandsexplosion

„Beide Verfahren enden nach ENDLICH VIELEN Schritten — MAXIMAL 2^n Schritte.“ ⚠ „Im schlimmsten Fall kann der KLEINSTE DEA über 2^n ZUSTÄNDE verfügen, während der kleinste NEA für dieselbe Sprache jedoch NUR n Zustände aufweist.“ (2^n = Anzahl der Teilmengen der Potenzmenge.)

★★★★ Verfahren 1: POTENZMENGENKONSTRUKTION (Teilmengenkonstruktion)

$$\text{NEA: } N = (Q_N, \Sigma, \delta_N, q_0, F_N) \quad \longrightarrow \quad \text{DEA: } D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$$

Beide Automaten haben dasselbe Eingabealphabet; beim Startzustand von D handelt es sich um die MENGE, die den Startzustand von N enthält.

Konstruktion von ...	Regel
Q_D	★ Q_D ist die Menge der TEILMENGEN von Q_N (= die POTENZMENGE): $ Q_N = n \Rightarrow Q_D = 2^n$. Die tatsächliche Anzahl kann sehr viel kleiner sein — alle NICHT VON q_0 AUS ERREICHBAREN Zustände können ELIMINIERT werden
F_D	★ F_D umfasst ALLE Teilmengen von Q_N , die MINDESTENS EINEN akzeptierenden Zustand von N enthalten: $S \cap F_N \neq \emptyset$
δ_D	★ Für jede Menge $S \subseteq Q_N$ und jedes Eingabesymbol a : $\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$ — Betrachtung ALLER in S enthaltenen Zustände p ; Untersuchung, zu welchen Zuständen N auf a hin übergeht; ÜBERNAHME DER VEREINIGUNG aller dieser Zustände

📝★★★★ Das durchgerechnete Beispiel

NEA: 3 Zustände $\Rightarrow$ DEA: $2^3 = 8$ Zustände (theoretisches Maximum)

Zustand/Eingabe	0	1
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
* $\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
* $\{q_2\}$, * $\{q_1, q_2\}$, * $\{q_0, q_1, q_2\}$	...	...

→ markiert den Startzustand · * markiert einen Endzustand ★ „Die orange markierten Felder können GESTRICHEN werden, da diese die LEERE MENGE $\emptyset$ enthalten bzw. diese Zustände vom Startzustand q_0 aus NICHT ZU ERREICHEN sind.“

Berechnung mit der erweiterten Übergangsfunktion:

$$\begin{aligned}\delta_D(\{q_0\}, 0) &= \{q_0, q_1\} & \delta_D(\{q_0\}, 1) &= \{q_0\} \\ \delta_D(\{q_0, q_1\}, 0) &= \delta_N(q_0, 0) \cup \delta_N(q_1, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\} \\ \delta_D(\{q_0, q_1\}, 1) &= \delta_N(q_0, 1) \cup \delta_N(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\} \\ \delta_D(\{q_0, q_2\}, 0) &= \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\} & \delta_D(\{q_0, q_2\}, 1) &= \{q_0\} \cup \emptyset = \{q_0\}\end{aligned}$$

⇒ Der resultierende DEA hat nur 3 statt 8 Zustände: $\{q_0\}$, $\{q_0q_1\}$, $\{q_0q_2\}$

★★ Verfahren 2: LAZY EVALUATION

? ⚠️ * „Der Rechenaufwand kann mit 2^n zu berechnenden Zuständen sehr zeitintensiv werden. Kann diese ZUSTANDSEXPLSION vermieden werden?“ * 💡 „Der exponentielle Zeitaufwand lässt sich oft vermeiden, wenn man die Teilmengen durch ‚ABWARTENDE AUSWERTUNG‘ berechnet.“
★ „Diese Auswertungsstrategie, bei der SO WENIG WIE MÖGLICH ausgewertet wird, bezeichnet man als LAZY EVALUATION.“

- ① Starte im Startknoten und konstruiere NUR die Knoten, die vom Startknoten aus ERREICHBAR sind
- ② Fahre in gleicher Weise mit den konstruierten Knoten fort

★ Der umgekehrte Weg: DEA → NEA

- ① Abänderung der Übergangstabelle des DEA
→ Zustände in GESCHWEIFTE KLAMMERN setzen, um sie als MENGEN zu kennzeichnen

$$\tilde{\delta} = \begin{cases} \{\delta(q, a)\}, & \text{falls } \delta(q, a) \text{ definiert} \\ \emptyset & \text{sonst} \end{cases}$$

★ „Die Umwandlung erfordert $O(n)$ Zeit für einen DEA mit n Zuständen.“

★★ Die Praxisregel

★★ „NEAs sind VIEL EINFACHER ZU ENTWERFEN als DEAs. Aus diesem Grund wird MEIST ZUERST EIN NEA ENTWORFEN, der dann in einen ÄQUIVALENTEN DEA TRANSFORMIERT wird.“

H.9 ★★★★★ REGULÄRE AUSDRÜCKE

★ „Reguläre Ausdrücke können **GENAU DIESELBEN REGULÄREN SPRACHEN** definieren, die von den verschiedenen Arten von Automaten beschrieben werden. Im Gegensatz zu Automaten stellen sie ein **DEKLARATIVES** Verfahren zur Beschreibung der Zeichenreihen zur Verfügung.“

Definition: „Reguläre Ausdrücke beschreiben die Mengen von Zeichenreihen, die als **BESCHRIFTUNGEN BESTIMMTER PFADE** im Übergangsdiagramm des NEA oder DEA auftreten.“

★★ **DER SATZ:** „Wenn für einen beliebigen DEA A $L = L(A)$ gilt, dann gibt es einen regulären Ausdruck R derart, dass gilt: $L = L(R)$.“

★★★★ Die 5 OPERATOREN

#	Operator	Definition
1	Konstanten ε und $\emptyset$	$L(\varepsilon) = \{\varepsilon\} \cdot L(\emptyset) = \emptyset$ • wenn a ein beliebiges Symbol ist, dann ist a ein regulärer Ausdruck mit $L(a) = \{a\}$
2	VEREINIGUNG $E + F$	$L(E + F) = L(E) \cup L(F)$
3	VERKETTUNG EF	$L(EF) = L(E) \cdot L(F)$
4	KLEENESCHE HÜLLE E^*	$L(E^*) = (L(E))^*$
5	KLAMMERUNG (E)	$L(E) = L((E))$ — beschreibt die gleiche Sprache

★★ Die AUSWERTUNGSREIHENFOLGE (Priorität!)

„Das Arbeiten mit regulären Ausdrücken entspricht dem Arbeiten mit **arithmetischen Ausdrücken**, wobei auch hier bestimmte **Vorrangs- bzw. Prioritätsregeln** gelten.“

Priorität	Operator	Bezeichnung
1 (höchste)	*	Sternoperator (Stern)
2	Verkettung	Verkettungsoperator (Punkt)
3 (niedrigste)	+	Vereinigungsoperator (Plus)

! ★ Die klassische Falle:

$$(01)^* = 01010101\dots \quad \text{ABER} \quad 01^* = 01111\dots$$

(Analog: $xy + z = (xy) + z \neq x(y + z)$ **und** $x - y - z = (x - y) - z \neq x - (y - z)$.)

★★ Die GESETZE für reguläre Ausdrücke

*„Viele dieser Gesetze ähneln den Gesetzen der Arithmetik, wenn man die **Vereinigung von Mengen der Addition** und die **Verkettung der Multiplikation** gleichsetzt. Es besteht jedoch **NICHT IMMER eine 1:1-Analogie**.“*

Gesetz	Formel
Kommutativgesetz (nur Vereinigung!)	$L + M = M + L$
Assoziativgesetz der Mengenvereinigung	$(L + M) + N = L + (M + N)$
Assoziativgesetz der Verkettung	$(LM)N = L(MN)$
Identitätsgesetz der Vereinigung	$\emptyset + L = L + \emptyset = L$
Identitätsgesetz der Verkettung	$\varepsilon L = L\varepsilon = L \cdot \emptyset L = \emptyset = \emptyset$
Distributivgesetz der Verkettung	$L(M + N) = LM + LN \cdot (M + N)L = ML + NL$
Idempotenzgesetz für die Vereinigung	★ $L + L = L$ — „Die Anwendung auf zwei Operanden mit dem gleichen Wert ergibt wieder diesen Wert“

★★ Die GESETZE der HÜLLENBILDUNG (auswendig!)

#	Gesetz
(i)	$(L^*)^* = L^*$
(ii)	$\emptyset^* = \varepsilon$
(iii)	$\varepsilon^* = \varepsilon$
(iv)	$L^* = \varepsilon + L^+ = \varepsilon + L + LL + LLL + \dots$
(v)	$L^+ = LL^* = L\varepsilon + LL + LLL + LLLL + \dots$
(vi)	$L? = \varepsilon + L$

📝★★★★ BEISPIEL 1 (das große durchgerechnete Beispiel!)

Aufgabe: Beschreibe die Menge der Zeichenreihen, die aus in abwechselnder Reihenfolge stehenden Nullen und Einsen besteht.

Schritt 1: $L(0) = \{0\}$, $L(1) = \{1\}$ **Schritt 2:** $L(0)L(1) = L(01) = \{01\}$ **Schritt 3:** Hülle: $(01)^*$

❓ „Ist $L((01)^*)$ schon die Sprache, die wir suchen?“ ⚠️ **NEIN**, weil:

- die Sprache muss **NICHT** mit Null beginnen und mit Eins enden!
- es muss die Möglichkeit betrachtet werden, dass eine Eins am Anfang und/oder eine Null am Ende steht
- $\Rightarrow$ Es müssen mindestens **DREI WEITERE** reguläre Ausdrücke gebildet werden!

Ausdruck	Bedeutung
$(01)^*$	Beginnt mit Null und endet mit Eins
$(10)^*$	Beginnt mit Eins und endet mit Null
$0(10)^*$	Beginnt mit Null und endet mit Null
$1(01)^*$	Beginnt mit Eins und endet mit Eins

$$\Rightarrow (01)^* + (10)^* + 0(10)^* + 1(01)^*$$

? „Lässt sich dieser Ausdruck noch PRÄGNANTER schreiben?" 💡 Idee: Hinzufügen einer **OPTIONALEN EINS** am Anfang und einer **OPTIONALEN NULL** am Ende.

$$L(\varepsilon + 1) = \{\varepsilon\} \cup \{1\} = \{\varepsilon, 1\} \quad L(\varepsilon + 0) = \{\varepsilon\} \cup \{0\} = \{\varepsilon, 0\}$$

$$(\varepsilon + 1)(01)^*(\varepsilon + 0)$$

⚠️ „Die Ausdrücke mit ε müssen für eine **korrekte Auswertung der Operatoren in KLAMMERN** gesetzt werden."

📝 BEISPIEL 2 (aus einem Automaten)

$$(0 + 1)^*1(0 + 1) + (0 + 1)^*1(0 + 1)(0 + 1)$$

📝 BEISPIEL 3 (Übungsaufgaben)

- $\Sigma = \{a, b, c\}, L = \{\omega \mid \omega \text{ mit mindestens je ein } a \text{ und } b\}$
- $\Sigma = \{0, 1\}, L = \{\omega \mid \omega \text{ enthält mindestens einmal } 11\}$

📝 BEISPIEL 4 (Welche Sprache beschreibt ...?)

- $(1 + \varepsilon)(00^*1)^*0^*$
- $(0^*1^*)000(0 + 1)^*$
- $(0 + 10)^*1^*$

H.10 ★★ Die UNIX-NOTATION für erweiterte reguläre Ausdrücke

⚠️ Das Problem

„Bisher wurden nur **kleine Alphabete** verwendet (z. B. $\{0, 1\}$). Bei Verwendung von **ASCII (128 Zeichen)** würde derselbe Ausdruck eine **Liste ALL DIESER ZEICHEN** beinhalten → **sehr umfangreich.**"

★ **Vorteile der Unix-Notation:** „Leistungserweiterungen" · **prägnante Darstellung komplexer regulärer Ausdrücke** · ★ **ERKENNUNG NICHTREGULÄRER SPRACHEN ist möglich!**

★★ Die Operatorentabelle

Symbol	Bedeutung
.	Beliebiges Zeichen
*	Zeichen kann 0-mal oder beliebig oft vorkommen
+	Ein oder mehrere Vorkommen
?	Kein oder ein Vorkommen
\$	Ende einer Zeichenkette oder Zeilenende
^	Anfang einer Zeichenkette oder Zeilenanfang
[^{}]	NEGIERT den Inhalt („Alles außer {}“)
	Trennt Alternativen — anstelle des + für Vereinigung
()	Zusammenfassung der Ausdrücke
[]	Genau EIN Zeichen, das zwischen den Klammern steht
{n}	Zeichen kommt GENAU n-mal vor
{min,}	Zeichen kommt MINDESTENS min-mal vor
{min,max}	Mindestens min-mal und maximal max-mal
{0,max}	Zeichen darf MAXIMAL max-mal vorkommen
\	Schützt das nächste Zeichen (Escape)
\\ \. \+ \- \\$ \] \[\) \[	die jeweils literalen Zeichen

★ Zeichenklassen

Klasse	Entspricht
[:alpha:]	[A-Za-z]
[:digit:]	[0-9]
[:alnum:]	[0-9A-Za-z]
[:lower:]	Kleinbuchstaben
[:upper:]	Großbuchstaben
\d	Beliebiges Ziffernzeichen — identisch mit [0-9]
\D	Alle Zeichen AUSSER Ziffern — identisch mit [^0-9]
\w	Jedes alphanumerische Zeichen — identisch mit [a-zA-Z0-9_]
\W	Alle Zeichen außer alphanumerische — identisch mit [^\w]
[a1a2a3]	$a_1 + a_2 + a_3$
[0-9]	[0123456789]
[^ {abc}]	Alles außer a, b oder c (Negierung)
[a-zA-Z]	a bis z oder A bis Z

★ Die Übersetzungstabelle Theorie ↔ Unix

Theorie	Unix
$\varepsilon + a$	a?
aa^*	a+
aaaaa	a{5}

★ Fazit und Anwendungen

★★ „JEDE Sprache, die durch einen REGULÄREN AUSDRUCK definiert wird, wird AUCH durch einen ENDLICHEN AUTOMATEN definiert.“

Anwendungen — Beschreibung vage definierter Klassen von Textmustern:

- Zeichenkettensuche → `grep`, `find`
- Eingabevalidierung
- Lexikalische Analyse → `lex` / `flex`

H.11 ★★★ DAS PUMPING-LEMMA FÜR REGULÄRE SPRACHEN

! Die Motivation: das Klammerproblem

✎ *, „Ein kleines Beispiel bzgl. der Klammersetzung: (((((((((((... und)))))))))). Wie kann man feststellen, wie viele Klammern es waren?“* ! ! „Mit den bisher kennengelernten endlichen Automaten kann man z. B. NICHT PRÜFEN, OB KLAMMERN AUCH WIEDER GESCHLOSSEN WERDEN — unter der Voraussetzung, dass es beliebig viele Klammern sein dürfen.“

? „Warum kann man so ein Problem nicht mit einem endlichen Automaten prüfen?“ ★★ „EIN ENDLICHER AUTOMAT KANN NICHT BELIEBIG MITZÄHLEN. D. h. diese Art von Sprachen können NICHT von einem endlichen Automaten erkannt werden.“

💡 „Es gibt ein allgemeineres Argument, das sich **PUMPING-LEMMA** nennt. Es wird insbesondere **IMMER DANN VERWENDET, WENN VON EINER SPRACHE GEZEIGT WERDEN SOLL, DASS SIE NICHT REGULÄR IST.**“

★★ Die IDEE

Annahme: Das Wort w sei aus einer regulären Sprache — $w = xyz$. **Ist das Wort w LANG GENUG, so kann man es immer in DREI TEILE zerlegen bzw. zusammensetzen. Idee:** „Ist die Sprache regulär, so MUSS es möglich sein, das Wort w zu PUMPEN. Die so entstandenen Wörter sind dann wieder in der Sprache L enthalten.“ **PUMPEN:** Der y -Teil des Wortes wird **VERVIELFACHT**:

$$w = xyz \rightarrow w' = xyyz \rightarrow w'' = xyxyz \rightarrow \dots$$

★ Die Folgerung aus der Idee:

- „JEDER endliche Automat, dessen akzeptierte Sprache L **UNENDLICH VIELE WÖRTER** umfasst, muss **NOTWENDIGERWEISE ZYKLEN** enthalten, die beliebig oft durchlaufen werden können, da $|w| > |V|$ “
- Die Mächtigkeit $|L|$ dieser Sprache ist dann **ABZÄHLBAR UNENDLICH**
- ★★ „Sobald die **ANZAHL DER ZEICHEN** eines Wortes w **GRÖßER ALS** (oder gleich groß wie) die **ANZAHL DER ZUSTÄNDE** eines endlichen Automaten ist, MUSS ES **SCHLEIFEN GEBEN** — also eine Stelle, an der das Wort ‚AUFGEPUMPPT‘ werden kann. Der endliche Automat erkennt dann **AUCH** die aufgepumpten Wörter.“

 **Anschauliches Beispiel:** Die Wörter `aa` und `ba` gehören zur akzeptierten Sprache — sind aber zu kurz, um gepumpt werden zu können. Das Wort `abba` gehört zur Sprache und ist lang genug: `abbba`, `abbbba`, `abbbbbba` usw.

★★★★ DIE DEFINITION (auswendig!)

Ist L die durch eine REGULÄRE Grammatik erzeugte Sprache, dann gibt es eine von L abhängige KONSTANTE n , sodass es für JEDE Zeichenkette $\omega \in L$ mit $|\omega| \geq n$ eine ZERLEGUNG $\omega = xyz$ gibt. Es gilt:

$$(i) \ y \neq \varepsilon \text{ bzw. } |y| > 0 \quad (ii) \ |xy| \leq n \quad (iii) \ \forall k \geq 0 : xy^kz \in L$$

★★★★ DAS BEWEISVERFAHREN (der Widerspruchsbeweis!)

- ① ANNAHME: Die Sprache L IST regulär
- ② Dann MUSS es nach dem Pumping-Lemma eine Länge $n \in \mathbb{N}_0$ geben, sodass es für jedes Wort $w \in L$ mit $|w| \geq n$ eine Zerlegung $w = xyz$ gibt mit
 - ① $y \neq \varepsilon$ bzw. $|y| > 0$
 - ② $|xy| \leq n$
 - ③ $\forall k \geq 0 : xy^kz \in L$
- ③ ★ Wenn man nun für solch eine Zerlegung EIN k angeben kann, so dass $xy^kz \notin L$ gilt, dann MUSS DIE ANNAHME, DASS L REGULÄR IST, FALSCH SEIN!

★★★★★ DAS VOLLSTÄNDIG DURCHGERECHNETE BEISPIEL

Frage: Ist die Sprache $L := \{0^n 1^n \mid n \in \mathbb{N}_0\}$ über dem Alphabet $\Sigma = \{0, 1\}$ REGULÄR? (Die Sprache umfasst alle Zeichenreihen aus einer oder mehreren Nullen, gefolgt von der gleichen Anzahl an Einsen: `01`, `0011`, `000111` etc.)

Schritt 1 — Annahme: L ist regulär. Dann existiert eine Zerlegung $w = xyz$ mit den drei Bedingungen.

Schritt 2 — Zerlegung konstruieren: Da $|xy| \leq n$ gelten muss, gibt es ein $l \in \mathbb{N}$ mit $1 \leq l \leq k$, so dass gilt:

$$x = 0^{k-l} \quad y = 0^l \quad z = 1^k$$

Nebenrechnung: $0^{k-l} \cdot 0^l = 0^{k-l+l} = 0^k$ ✓ (z. B. $w = 00001111$)

Schritt 3 — Pumpen mit $k = 2$:

$$xy^2z = 0^{k-l}(0^l)^2 1^k = 0^{k-l} 0^{2l} 1^k = 0^{k-l+2l} 1^k = \boxed{0^{k+l} 1^k}$$

Schritt 4 — Widerspruch:

- Damit müsste auch $0^{k+l} 1^k \in L$ gelten
- Da aber $y > 0$ und damit auch $l > 0$ ist $\implies k+l > k \implies k+l \neq k$
- ✗ Es gibt MEHR NULLEN als EINSEN in dem Wort!

☞ Damit ist $0^{k+l} 1^k \notin L$ — und die Sprache ist NICHT REGULÄR. ■

H.12 ★★ KONTEXTFREIE GRAMMATIK

$$G = (V, T, P, S)$$

Symbol	Bedeutung
V	Variablen (Nichtterminale)
T	Terminale
P	Produktion
S	Startsymbol

📝★ BEISPIEL 1: PALINDROME

Beispiele für Palindrome: *Otto, Lehel, ABBA, Ebbe, Elle, Gag, Lagerregal, Regallager, Reliefpfeiler, Retsinakanister, „Eine Blase salbe nie“*

Wie könnte ein Palindrom über $\Sigma = \{0, 1\}$ aussehen?

- ε , 0 und 1 sind Palindrome
- Wenn ω ein Palindrom ist, dann ist auch $0\omega 0$ und $1\omega 1$ ein Palindrom

$$P = \{ S \rightarrow \varepsilon \mid S \rightarrow 0 \mid S \rightarrow 1 \mid S \rightarrow 0S0 \mid S \rightarrow 1S1 \}$$

📝 BEISPIEL 2: Deutsche Grammatik

Satz → <Subjekt> <Verb> [<Objekt>]
 Subjekt → <Substantiv> | <Pronomen>
 Verb → <Klein>
 Objekt → <Substantiv> | <Pronomen>
 Substantiv → <Groß><Klein>
 Pronomen → ich | du | er | sie | es | wir | ihr | ...
 Klein → a|b|...|z | a<Kleinbuchstabe> | b<Kleinbuchstabe> | ...
 Groß → A<Kleinbuchstabe> | B<Kleinbuchstabe> | ...

📝★ BEISPIEL 3: Einfache Rechnung

Term → <Term> <Operator> <Term> | <Zahl>
 Operator → + | - | * | /
 Zahl → <Zahl><Ziffer> | <Ziffer>
 Ziffer → 0|1|2|3|4|5|6|7|8|9

? Ist $1 + 2 \cdot 32$ eine gültige einfache Rechnung? Ist $1 + 2 \cdot 23$ eine gültige einfache Rechnung? Ableitungsbaum: <Term> <Operator> <Term> → <Zahl> → $1 + 2 \cdot 3$

★ Anwendungen kontextfreier Grammatiken

- ★ Beschreibung von PROGRAMMIERSPRACHEN → PARSERBAU
- Analyse korrekt geklammelter arithmetischer Ausdrücke
- ★ XML ist eine kontextfreie Grammatik

★★ BACKUS-NAUR-FORM (BNF)

John BACKUS und Peter NAUR · ALGOL (Algorithmic Language) — ★ „C und Pascal sind Weiterentwicklungen von Algol 60“ ★ „BNF ist die STANDARDBESCHREIBUNGSTECHNIK für Programmiersprachen.“

Bestandteile: Ableitungsregeln (Produktionen) · Nichtterminalsymbole · Terminalsymbole

Symbol	Bedeutung
(vertikaler Strich)	entspricht dem ODER-Zeichen
::=	entspricht dem DEFINITIONSZEICHEN
< ... >	umschließt NICHTTERMINALSYMBOLE

BNF-Regeln:

```

Ziffer      = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
Vorzeichen = "+" | "-"

<Ziffer ausser Null> ::= 1|2|3|4|5|6|7|8|9
<Ziffer>             ::= 0 | <Ziffer ausser Null>
<Zweistellige Zahl>  ::= <Ziffer ausser Null> <Ziffer>
<Zehn bis Neunzehn> ::= 1 <Ziffer>
<Zweiundvierzig>    ::= 42
<Ziffernfolge>      ::= <Ziffer> | <Ziffer> <Ziffernfolge>
<Positive Zahl>      ::= <Ziffer ausser Null> | <Ziffer ausser Null> <Ziffernfolge>

```

★ ERWEITERTE BACKUS-NAUR-FORM (EBNF)

! „Die BNF benötigt teilweise umständliche Konstrukte, um optionale Elemente oder sich wiederholende Elemente darzustellen.“

Verwendung von VIER Klammerarten:

Klammer	Bedeutung
()	Gruppierung
[]	Optionalen Inhalt
{ }	Beliebige Wiederholungen
< >	Beliebige, aber mindestens einmalige Wiederholung

★ „Terminale und Nichtterminale werden NICHT STRENG UNTERSCHIEDEN.“

BNF vs. EBNF:

```

BNF:  <Ganze Zahl>      ::= <Positive Zahl> | - <Positive Zahl> | 0
      <Positive Zahl>   ::= <Ziffer ausser Null><Optionale Ziffernfolge>
      <Optionale Ziffernfolge> ::= <Ziffer> <Optionale Ziffernfolge> | ε

EBNF: GanzeZahl = ( [ "-" ], ZifferAusserNull, { Ziffer } ) | "0";

```

Weiterführend: The syntax of C in Backus-Naur Form · The syntax of Java in Backus-Naur Form

H.13 ★★ KONTEXTSENSITIVE GRAMMATIK — das Beispiel

Aufgabe: Was für eine Sprache beschreibt die folgende Produktion? $\Sigma = \{a, b, c\}$, $V = \{X, Y, Z\}$, Startsymbol: X

$X \rightarrow aXYZ \mid aYZ$
 $ZY \rightarrow YZ$
 $aY \rightarrow ab$
 $bY \rightarrow bb$
 $bZ \rightarrow bc$
 $cZ \rightarrow cc$

 **Ableitung (Variante 1 — das lange Wort):**

$X \rightarrow aXYZ \rightarrow aaXYZY \rightarrow aaaYZYZY$
 $\rightarrow aaaYYZZY \rightarrow aaaYYZY \rightarrow aaaYYYZZZ$ (ZY $\rightarrow$ YZ mehrfach anwenden)
 $\rightarrow aaabYYZZ \rightarrow aaabbYZZ \rightarrow aaabbbZZ$ (aY $\rightarrow$ ab, bY $\rightarrow$ bb)
 $\rightarrow aaabbbcZ \rightarrow aaabbbccZ \rightarrow aaabbbccc$ (bZ $\rightarrow$ bc, cZ $\rightarrow$ cc)

 **Ableitung (Variante 2 — das kurze Wort):** $X \rightarrow aYZ \rightarrow abZ \rightarrow abc$

★★ DAS ERGEBNIS

$$L(G) = \{a^n b^n c^n \mid n \geq 1\} \quad (\text{ohne Beweis})$$

 **Das ist DAS Standardbeispiel: $\{a^n b^n c^n\}$ ist KONTEXTSENSITIV (Typ 1), aber NICHT kontextfrei — genau wie $\{0^n 1^n\}$ kontextfrei (Typ 2), aber NICHT regulär ist!**

H.14 ★★★ DIE GROSSE ZUSAMMENFASSUNG ZU KAPITEL 08

Typ	Sprache	Grammatikregel	Automat	Beispielsprache	Beweis der Nicht-Zugehörigkeit
3	regulär	$A \rightarrow xB$ oder $A \rightarrow Bx$, $A \rightarrow x$	endlicher Automat (DEA/NEA)	$(01)^*$, $\{x01y\}$	Pumping-Lemma für reguläre Sprachen
2	kontextfrei	genau EIN Nichtterminal links	Kellerautomat	$\{0^n 1^n\}$, Palindrome, Klammerausdrücke, alle Programmiersprachen	Pumping-Lemma für kontextfreie Sprachen
1	kontextsensitiv	links nie mehr Zeichen als rechts (monoton)	linear beschränkte Turingmaschine	$\{a^n b^n c^n\}$	—
0	semi-entscheidbar / rekursiv aufzählbar	keine Einschränkung	Turingmaschine	Halteproblem	—

▼ ? Selbsttest Teil H

1. Was leisteten Turing (30er), Chomsky (50er) und Cook (1969)?
2. Nennen Sie 4 Anwendungen der Automatentheorie.
3. Definieren Sie: Alphabet, Kleenesche Hülle, transitive Hülle, Zeichenreihe, Sprache, formale Sprache.
4. Was ist der Unterschied zwischen $\emptyset$ und $\{\varepsilon\}$?
5. Berechnen Sie Σ^2 und Σ^3 für $\Sigma = \{a, b\}$.
6. Nennen Sie die 3 Operationen auf Sprachen mit je einem Beispiel.
7. Berechnen Sie L^2 für $L = \{0, 11\}$.
8. Nennen Sie die 4 Bestandteile einer Grammatik.
9. Was ist das Wortproblem?
10. Nennen Sie die Chomsky-Hierarchie: Typ, Sprache, Automat — für alle 4 Typen.
11. Welche Sprachtypen sind entscheidbar?
12. Was ist eine monotone Grammatik?
13. Welcher Typ beschreibt alle Programmiersprachen?
14. Schreiben Sie das 5-Tupel eines Automaten auf und erklären Sie alle Symbole.
15. Was macht die erweiterte Übergangsfunktion? Schreiben Sie beide Gleichungen auf.
16. Was ist ein Fangzustand?
17. DEA vs. NEA: Wie unterscheiden sich δ und $L(A)$?
18. Konstruieren Sie einen DEA für „gerade Anzahl von Nullen und Einsen“.
19. Beschreiben Sie die Potenzmengenkonstruktion in 3 Teilen (Q_D , F_D , δ_D).
20. Was ist Lazy Evaluation und warum braucht man sie?
21. Warum entwirft man meist zuerst einen NEA?
22. Nennen Sie die 5 Operatoren regulärer Ausdrücke.
23. Wie lautet die Prioritätenreihenfolge? Was ist der Unterschied zwischen $(01)^*$ und 01^* ?
24. Nennen Sie die 6 Gesetze der Hüllenbildung.
25. Entwickeln Sie den regulären Ausdruck für „abwechselnde Nullen und Einsen“.
26. Übersetzen Sie in Unix-Notation: $\varepsilon + a, aa^*, aaaaaa$.
27. Warum kann ein endlicher Automat keine Klammern zählen?
28. Formulieren Sie das Pumping-Lemma vollständig.
29. Beweisen Sie mit dem Pumping-Lemma, dass $\{0^n 1^n\}$ nicht regulär ist.
30. Geben Sie die Grammatik für Palindrome über $\{0, 1\}$ an.
31. Was bedeuten $::=$, $|$ und $< >$ in der BNF?
32. Nennen Sie die 4 Klammerarten der EBNF.
33. Welche Sprache erzeugt die Grammatik mit $X \rightarrow aXYZ \mid aYZ$, $ZY \rightarrow YZ$, ...?

TEIL I — DIE 3 GEDANKENEXPERIMENTE (mit Musterlösungen)

I.1 ★ Gedankenexperiment 1: Die schwerste Kugel

Aufgabe: „Wie oft muss man wiegen, um herauszufinden, welche die **SCHWERSTE** Kugel ist?“ (mit einer Balkenwaage)

 **Lösungsansatz (ergänzt):** Bei n Kugeln und der Suche nach dem **Maximum** benötigt man mit einem Turnierbaum $n - 1$ **Wägungen** — jede Wägung eliminiert genau eine Kugel. Das entspricht exakt dem **Maximum-Problem** $O(n)$ aus Kapitel 04 (Pivotelement)!

I.2 ★★ Gedankenexperiment 2: Die 12 Kugeln — 3 Wägungen

Aufgabe: *, „Die zwölf Kugeln sehen alle gleich aus. Das Gewicht **EINER** Kugel weicht von dem aller anderen ab — d. h. sie ist **entweder leichter ODER schwerer** als alle anderen. Die Balkenwaage darf **maximal DREI MAL** zum Wiegen benutzt werden. **Wie findet man diese Kugel?**“*

 **DIE MUSTERLÖSUNG** (Quelle: SPIEGEL, Rätsel der Woche)

Wägung 1: Vier Kugeln links, vier Kugeln rechts.

FALL 1: Die Waage ist **NICHT** im Gleichgewicht

Dann muss die gesuchte Kugel **eine der acht auf der Waage** sein. Nehmen wir an, die **linken vier** sind zusammen **schwerer** als die vier rechts.

Wägung 2:

- **Nimm DREI der VIER Kugeln rechts** von der Waage und lege sie daneben (*merke sie dir und die auf der Waage verbleibende Kugel!*)
- **Ersetze sie durch DREI der VIER Kugeln, die bei Wägung 1 LINKS lagen** (*auch hier: merke die links verbleibende Kugel*)
- **Lege links DREI der VIER Kugeln, die bei Wägung 1 NICHT DABEI waren** — wir wissen, dass diese drei **KEINE** abweichende Masse haben können

Unterfall	Beobachtung	Schlussfolgerung	Wägung 3
1.1	Linke Seite schwerer	Entweder die LINKS verbliebene Kugel ist die gesuchte (und SCHWERER) oder die RECHTS verbliebene ist die gesuchte (und LEICHTER)	Vergleiche diese beiden Kugeln miteinander
1.2	Waage im Gleichgewicht	Die gesuchte Kugel ist eine der DREI, die bei Wägung 1 RECHTS lagen — und da links die schwerere Seite war, ist sie LEICHTER	Nimm zwei dieser drei Kugeln und lege sie links und rechts. Ist eine leichter → gefunden. Sind sie gleich schwer → die dritte ist die gesuchte
1.3	Rechte Seite schwerer	Die gesuchte Kugel ist eine der DREI, die bei Wägung 1 LINKS lagen — und sie ist SCHWERER	Vergleiche zwei der drei Kugeln. Ist eine schwerer → gefunden. Gleich schwer → Kugel Nummer drei

FALL 2: Die Waage ist bei Wägung 1 IM GLEICHGEWICHT

Dann muss sich die gesuchte Kugel unter den VIER Kugeln befinden, die bei Wägung 1 NICHT dabei waren.

Wägung 2: Lege drei dieser vier Kugeln in die leere linke Waagschale; in der rechten Schale liegen drei der acht Kugeln aus Wägung 1, von denen keine die gesuchte sein kann.

Unterfall	Beobachtung	Schlussfolgerung	Wägung 3
2.1	Linke Seite schwerer	Die gesuchte ist eine der drei links und SCHWERER	Vergleich von zwei der drei
2.2	Rechte Seite schwerer	Die gesuchte ist ebenfalls eine der drei links und LEICHTER	Vergleich von zwei der drei
2.3	Waage im Gleichgewicht	Die gesuchte ist jene, die weder bei Wägung 1 noch bei Wägung 2 in einer Waagschale lag	Vergleiche sie mit einer beliebigen anderen Kugel, um herauszufinden, ob sie schwerer oder leichter ist

🔑 **Der informationstheoretische Hintergrund:** Es gibt 24 mögliche Ausgänge (12 Kugeln × {leichter, schwerer}); eine Wägung hat 3 mögliche Ergebnisse (links, rechts, Gleichgewicht) $\Rightarrow 3^3 = 27 \geq 24$ ✓ — drei Wägungen reichen theoretisch gerade aus. (Das ist exakt das Entscheidungsbaummodell aus Kapitel 04, nur mit einem ternären statt binären Baum!)

I.3 ★★★★★ Gedankenexperiment 3: Der perfekte Stadtrundgang

Aufgabe: „Sie sind zu Besuch in einer kleinen Stadt, die berühmt ist für ihre Fachwerkgebäude. Damit Ihnen kein Haus entgeht, wollen Sie JEDE STRASSE durchlaufen — die Richtung ist egal. Um nicht zu viel unterwegs zu sein, möchten Sie aber auf keinen Fall eine Straße oder einen Straßenabschnitt DOPPELT durchlaufen. Nur KREUZUNGEN dürfen Sie mehr als einmal betreten. Start- und Endpunkt dürfen Sie frei wählen — diese können identisch sein oder unterschiedlich. Wenn Sie die Tour einzeichnen, dürfen Sie den Stift nicht absetzen — so wie beim ‚Haus des Nikolaus‘.“

✓ DIE MUSTERLÖSUNG (Quelle: SPIEGEL, Rätsel der Woche)

★★ „ES IST NICHT MÖGLICH, alle Straßen in einem Zug genau einmal zu durchlaufen.“

Warum nicht?

- **Entscheidend sind die KREUZUNGEN**, an denen man als Fußgänger verschiedene Wege nehmen kann
- ★ „Wenn von einer Kreuzung eine **UNGERADE** Anzahl von Straßen abgeht, **MUSS** diese Kreuzung **START- ODER ENDPUNKT** des Weges sein.“ Eine solche Kreuzung kann nicht **ZUGLEICH** Start- und Endpunkt sein, weil sonst eine von ihr abgehende Straße nicht abgelaufen werden kann
- Eine Kreuzung mit **GERADER** Anzahl abgehender Straßen macht **KEINE PROBLEME** — entweder man passiert sie zwei- oder mehrfach, wobei alle Straßen genommen werden, oder sie ist zugleich Start- und Endpunkt eines geschlossenen Weges

Die konkrete Stadt hat **NEUN** Kreuzungen:

- **VIER** mit drei abgehenden Straßen (⚠ ungerade — in der Zeichnung rot markiert)
- **FÜNF** mit vier Straßen (gerade)

★★ „VIER Kreuzungen mit ungerader Straßenzahl sind leider **ZWEI ZU VIEL!** Denn ein Weg hat **NUR EINEN ANFANG UND EIN ENDE**. Also können nur an **ZWEI** der vier Kreuzungen alle von dort abgehenden Straßen abgelaufen werden.“

🔑 **DER HINWEIS DES DOZENTEN:** „Diese Art von Problemlösung besprechen wir in **KAPITEL 7 — EULERZYKLUS bzw. EULERWEG!**“

★★ Die Verbindung zur Theorie (die Brücke, die dieses Modul schlägt!)

Gedankenexperiment	Kapitel	Konzept
1 — schwerste Kugel	Kap. 04	Maximum-Problem $O(n)$, Turnierbaum
2 — 12 Kugeln / 3 Wägungen	Kap. 04	Entscheidungsbaummodell, untere Schranke ($3^3 \geq 24$)
3 — Stadtrundgang	Kap. 07	★ EULER-WEG: höchstens 2 Knoten mit ungeradem Grad!

TEIL J — QUERSCHNITT: ALLE KOMPLEXITÄTEN AUF

EINEN BLICK

J.1 ★★★★★ Die Master-Komplexitätstabelle

Bereich	Algorithmus / Operation	Best	Average	Worst
Datenstruktur	Array-Zugriff über Index	$O(1)$	$O(1)$	$O(1)$
	Array Einfügen/Löschen	—	$O(n)$	$O(n)$
	Liste Einfügen/Löschen (Position bekannt)	$O(1)$	$O(1)$	$O(1)$
	Liste: Zugriff auf i -tes Element	$O(1)$	$O(n)$	$O(n)$
	Stack PUSH / POP / HEAD	$O(1)$	$O(1)$	$O(1)$
	Queue ENQUEUE / DEQUEUE	$O(1)$	$O(1)$	$O(1)$
	Binärbaum-Höhe	—	—	$\Theta(\log K)$
Sortieren	Selectionsort	$O(n^2)$	$O(n^2)$	$O(n^2)$
	Insertionsort	$O(n)$	$O(n^2)$	$O(n^2)$
	Bubblesort (verbessert)	$O(n)$	$O(n^2)$	$O(n^2)$
	Quicksort	$O(n \log n)$	$O(n \log n)$	⚠ $O(n^2)$
	Mergesort	$O(n \log n)$	$O(n \log n)$	★ $O(n \log n)$
	Heapsort	$O(n \log n)$	$O(n \log n)$	★ $O(n \log n)$
	Shellsort	—	—	$O(n^2)/O(n^{1.5})/O(n \log^2 n)$
	Bucketsort	$O(n)$	$O(n)$	⚠ $O(n^2)$
	Radixsort	$O(n \cdot m)$	$O(n \cdot m)$	$O(n \cdot m)$
	Countingsort	$O(n + k)$	$O(n + k)$	$O(n + k)$
	Untere Schranke (vergleichend)	—	—	★ $\Omega(n \log n)$
	Heap BUILD-MAX-HEAP	—	—	$O(n)$
Heap	MAX-HEAPIFY	—	—	$O(\log n)$
Suchen	Lineare Suche	$O(1)$	$O(n/2)$	$O(n)$
	Binäre Suche	$O(1)$	$O(\log n)$	$O(\log n)$
	Exponentielle Suche	$O(1)$	$O(\log k)$	$O(\log n)$
	Interpolationssuche	★ $O(\log \log n)$	$O(\log \log n)$	⚠ $O(n)$
	Quadratische Binärsuche	—	—	$O(\sqrt{n})$
	Binärer Suchbaum	$O(1)$	$O(\log n)$	⚠ $O(n)$
Hashing	Verkettung: INSERT/DELETE	$O(1)$	$O(1)$	$O(1)$
	Verkettung: SEARCH	$O(1)$	$O(1 + \alpha)$	$O(n)$
	Offene Adressierung: SEARCH erfolglos	—	$\frac{1}{1-\alpha}$	—
	Offene Adressierung: SEARCH erfolgreich	—	$\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$	—
Graphen	BFS / DFS	—	—	$O(V + E)$
	Hierholzer (Euler)	—	—	$O(E + V)$
	Kruskal (Union-Find)	—	—	$O(E \alpha(V))$
	Prim (Fibonacci-Heap)	—	—	$O(E \log V)$

Bereich	Algorithmus / Operation	Best	Average	Worst
	Dijkstra (Fibonacci-Heap)	—	—	$O(E + V \log V)$
	Bellman-Ford	—	—	$O(V \cdot E)$
	Floyd-Warshall	—	—	$O(V ^3)$
	Ungarischer Algorithmus	—	—	$O(V ^3)$
	Backtracking	—	—	$O(k^n)$
	TSP exakt	—	—	⚠ $\frac{(n-1)!}{2}$
Automaten	NEA → DEA (Potenzmenge)	—	—	⚠ $O(2^n)$
	DEA → NEA	—	—	$O(n)$

J.2 ★★ Die 8 wiederkehrenden Ideen des ganzen Kurses

#	Idee	Wo taucht sie auf?
1	DIVIDE AND CONQUER	Quicksort · Mergesort · Binäre Suche · Master-Theorem
2	HALBIERUNG $\Rightarrow \log(n)$	Quicksort · Mergesort · Binäre Suche · Heap-Höhe · Binärbaum-Höhe
3	GREEDY (lokal beste Wahl)	Kruskal · Prim · Nearest Neighbor · TSP-Greedy · recursive binary splitting
4	ZUFALL statt Determinismus	RAND-SELECT (Quicksort-Pivot) · Universelles Hashing · Bogosort
5	ENTSCHEIDUNGSBAUM / untere Schranke	Sortier-Untergrenze $\Omega(n \log n)$ · 12-Kugeln-Rätsel · Backtracking
6	TRADE-OFF Zeit $\leftrightarrow$ Speicher	Mergesort (out-of-place) · Hashing (α) · Adjazenzmatrix vs. -liste · Bucketsort
7	GEOMETRISCHE REIHE $\sum x^k = \frac{1}{1-x}$	Quicksort-Halbierung · Rekursionsbaum · BUILD-MAX-HEAP · Sondierungsanalyse
8	STACK $\Rightarrow$ Tiefe · QUEUE $\Rightarrow$ Breite	Tiefensuche (Stack/Rekursion) · Breitensuche (Queue) · Pre-Order vs. Level-Order

J.3 ★★ Die „Unmöglichkeitsresultate" des Kurses

Resultat	Kapitel
Das HALTEPROBLEM ist unentscheidbar	02
Vergleichendes Sortieren braucht $\Omega(n \log n)$	04
Für Shellsort existiert KEINE h-Folge mit $O(n \log n)$	04
Für Shellsort existiert bis heute KEINE vollständige Laufzeitbeschreibung	04
KOLLISIONEN beim Hashing sind unvermeidbar ($ U > m$)	06
Kein effizienter Algorithmus für den LÄNGSTEN Weg	07
Kein effizienter Algorithmus für HAMILTON-Weg/-Zyklus	07
TSP ist NP — kein Algorithmus der Klasse P bekannt	07
Ein endlicher Automat kann NICHT BELIEBIG MITZÄHLEN (Klammerproblem)	08
$\{0^n 1^n\}$ ist NICHT regulär (Pumping-Lemma)	08
Für Typ-0-Sprachen gibt es KEINE Grammatik der Nicht-Wörter	08

TEIL K — SELBSTTESTFRAGEN (140 Fragen mit

Kurzantworten)

K.1 Grundlagen (1–30)

#	Frage	Antwort
1	Informatik = ?	Information + Mathematik ; Wissenschaft von der automatisierten Verarbeitung von Daten und Informationen
2	Information = ?	Daten + Kontext
3 ★	Definition Algorithmus?	Vollständige, präzise, endliche Beschreibung eines schrittweisen Problemlösungsverfahrens aus eindeutigen, ausführbaren Aktionen
4	Algorithmus vs. Programm?	Algorithmus = allgemeine Folge von Anweisungen · Programm = Umsetzung in eine Programmiersprache
5 ★	Die 7 Eigenschaften?	Korrektheit, Vollständigkeit, Eindeutigkeit, Ausführbarkeit, statische Endlichkeit, dynamische Endlichkeit, Effizienz
6 ★	Statische vs. dynamische Endlichkeit?	Länge der BESCHREIBUNG vs. Anzahl der SCHRITTE bis zum Halt
7	Was ist eine Schleifeninvariante?	Bedingung, die vor, während (nach jeder Iteration) und nach der Schleife gilt
8 ★	Halteproblem-Ergebnis?	Unentscheidbar — Widerspruchsbeweis über $Q(Q)$
9	Variable vs. Konstante?	verändert sich während der Ausführung / verändert sich nicht
10 ★	3 Arten von Datentypen?	Elementar (nicht zerlegbar), strukturiert (zusammengesetzt), Referenzen (Pointer)
11	k Bits → wie viele Zustände?	$n = 2^k$
12	Crumb/Nibble/Byte?	2 / 4 / 8 Bit
13	1 KByte = ?	$2^{10} = 1024$ Byte (nicht 1000!)
14 ★	MSB/LSB, Big-/Little-Endian?	höchst-/niederwertigstes Bit · höherwertiges/niederwertiges Byte zuerst
15 ★	Stellenwertformel?	$z = \sum_{v=0}^{n-1} a_v B^v$
16	2 Umwandlungsmethoden dezimal→dual?	Subtraktionsmethode (B^n) und Divisionsmethode (durch B)
17 ★	3 Verfahren für negative Zahlen?	Vorzeichen-Betrag, Einserkomplement, Zweierkomplement
18 ★	Warum Zweierkomplement?	Einfache Addition/Subtraktion bitweise unabhängig vom Vorzeichen ; Null nur einmal
19 ★	Was ist das Hidden Bit?	Die immer 1 lautende Vorkommastelle der normalisierten Mantisse — wird nicht gespeichert , erhöht die Genauigkeit um 1 Bit
20 ★	IEEE 754 float: V/E/M/Bias?	1 / 8 / 23 Bit, Bias 127
21 ★	IEEE 754 double: V/E/M/Bias?	1 / 11 / 52 Bit, Bias 1023
22	$E = 255, M = 0 / M \neq 0$?	$\pm\infty$ / NaN

#	Frage	Antwort
23 ★	Die 4 Umwandlungsschritte?	Normalisieren · Hidden Bit entfernen + auf 23 auffüllen · Bias addieren · eintragen
24	Ariane 5?	double-Gleitkommazahl auf 16-Bit-Integer zugewiesen → Bereichsüberschreitung
25	ASCII/Unicode(UCS-2)/UTF-8?	7 Bit / 16 Bit, 65536 Zeichen / Mehrbyte, Länge im 1. Byte
26 ★	Speicherabbildungsfunktion?	$\text{AddrOf}(a[i]) = \text{AddrOf}(a) + (i - \text{first}) \cdot \text{elementSize}$
27 ★	Feld vs. Liste (3 Kriterien)?	Größe fix/veränderlich · Zugriff $O(1)/O(n)$ · Einfügen $O(n)/O(1)$
28 ★	Wozu die Wächter-Strategie?	Sonderfälle am Listenanfang/-ende entfallen — die Liste ist nie leer
29 ★	Stack vs. Queue?	LIFO (PUSH/POP/HEAD, Tiefensuche) vs. FIFO (ENQUEUE/DEQUEUE/HEAD, Breitensuche)
30 ★	Binärbaum: Knoten je Ebene / gesamt / Höhe?	$2^i / 2^{h+1} - 1 / h = \log(K + 1) - 1$

K.2 Komplexität (31–55)

#	Frage	Antwort
31 ★	Komplexität = ?	Aufwand = Speicherbedarf + Rechenzeit
32 ★	Effektivität vs. Effizienz?	Wirksamkeit (wie gut gelöst) vs. Wirtschaftlichkeit (mit wie wenig Aufwand) — Effizienz setzt Effektivität voraus
33	4 Gründe für Laufzeitanalyse?	Vorhersage · Vergleich · Optimierungspotential · Ressourcen
34 ★	Fein- vs. Grobanalyse?	Jedes Detail (nicht praktikabel, Echtzeit) vs. nur signifikante Teile (Normalfall)
35 ★	Die 3 Fälle?	Best / Worst / Average Case
36	Wer führte O ein? Wer machte es bekannt?	Paul Bachmann (1892) / Edmund Landau
37 ★	Definition $O(g(n))$?	$\{f \mid \exists c, n_0 : \forall n \geq n_0 : 0 \leq f(n) \leq c \cdot g(n)\}$
38 ★★	Was sagt O NICHT?	Nichts über die TATSÄCHLICHE Laufzeit — nur über das Wachstum im Verhältnis zur Problemgröße
39 ★	Die 5 Schranken?	O (höchstens), o (langsamer), Ω (mindestens), ω (schneller), Θ (genauso)
40 ★	Die Komplexitätskette?	$1 < \log n < \sqrt{n}$
41	n^2 bei $n = 10^6$? 2^n bei $n = 1000$?	$10^{12} / 10^{300}$
42 ★	Warum ist die Log-Basis egal?	Basiswechsel ergibt nur einen konstanten Faktor
43 ★	Kosten von Schleife/Rekursion/Funktionsaufruf?	$O(\# \text{Durchläufe}) / T(n) / O(1)$
44 ★	Rekursionsgleichung?	$T(n) = a \cdot T(n/b) + f(n)$
45	Was ist $a, b, f(n)$?	Anzahl Teilprobleme · Verkleinerungsfaktor · nicht-rekursiver Aufwand (Teilen + Zusammenfügen)
46 ★	Die 3 Verfahren in empfohlener Reihenfolge?	Master-Theorem $\Rightarrow$ Substitutionsmethode $\Rightarrow$ Rekursionsbaum
47	Substitutionsmethode: 3 Schritte?	Einsetzen bis Vermutung · Anzahl Schritte berechnen · Induktionsbeweis
48	Rekursionsbaum: 4 Schritte?	Baum zeichnen · Höhe/Knoten/Kosten abschätzen · Formel für letzte Ebene · Induktion
49 ★★	Master-Theorem E ?	$E = \log_b a$
50 ★★	Fall 1?	$f \in O(n^{E-\epsilon}) \Rightarrow T \in \Theta(n^E)$
51 ★★	Fall 2?	$f \in \Theta(n^E) \Rightarrow T \in \Theta(n^E \log n)$
52 ★★	Fall 3?	$f \in \Omega(n^{E+\epsilon})$ und $af(n/b) \leq cf(n) \Rightarrow T \in \Theta(f(n))$
53	Was, wenn kein Fall greift?	Komplexitätsklasse anderweitig berechnen
54	Geometrische Reihe?	$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$
55	Untere Schranke des Problems vs. obere Schranke des Algorithmus?	Mindestens benötigte Ressourcen vs. Ressourcen eines konkreten Algorithmus

K.3 Sortieren (56–85)

#	Frage	Antwort
56 ★	Stabil = ?	Reihenfolge GLEICHER Elemente bleibt erhalten
57 ★	In-Place vs. Out-of-Place?	<i>in situ</i> /intern (direkt modifizierbar) vs. <i>ex situ</i> /extern
58	3 Sortier-Operationen?	Zugriff c_Z , Vergleich c_V , Datenaustausch c_D
59	Selectionsort: Idee, Stabilität, Besonderheit?	Kleinstes Element nach vorne · instabil · ★ optimal bzgl. Swaps ($O(n)$)
60	Insertionsort: Idee, Stabilität, Best Case?	Element in sortierten Bereich einfügen · stabil · $O(n)$
61 ★	Was verursacht i. d. R. eine Fallunterscheidung?	Eine While-Schleife
62	Bubblesort-Verbesserung?	Boolsche Variable <code>noSwap</code> + <code>break</code> → Best Case $O(n)$
63	Shakersort-Idee?	Bidirektional — größte nach oben, kleinste nach unten
64 ★	Divide-and-Conquer definieren?	Rekursive Zerlegung in identische kleinere Probleme, dann Zusammensetzen der Teillösungen
65 ★	Quicksort: Erfinder, Stabilität, Worst Case?	Hoare · nicht stabil · $O(n^2)$ bei Degeneration
66 ★	Wann degeneriert Quicksort?	Wenn das Pivot immer das kleinste/größte Element ist → bereits sortierte Eingabe
67	2 degenerationsfreie Alternativen?	Mergesort und Heapsort
68 ★	Die 6 Pivotwahlen?	Mittleres Element · Minima löschen · Feld sortieren · RAND-SELECT · Median-of-three · SELECT
69 ★	SELECT-Algorithmus?	Fünfer-Gruppen bilden → Fünfer-Mediane → rekursiv den Median der Mediane — deterministisch linear im Worst Case
70 ★	Mergesort: Stabilität, Platz, Besonderheit?	stabil · ⚠ out-of-place (doppelter Speicher) · parallelisierbar
71 ★★	Quicksort vs. Mergesort?	Quicksort: Teilen = Kernidee · Mergesort: Rekombination = Kernidee
72 ★	Warum bringt k -fache Teilung bei Mergesort nichts?	Minimum aus k Listenköpfen kostet k Schritte
73 ★	Max-Heap-Eigenschaft?	$A[\text{PARENT}(i)] \geq A[i]$ für alle Knoten außer der Wurzel
74 ★	Heap-Array-Formeln?	links $2i$ · rechts $2i + 1$ · Elter $\lfloor i/2 \rfloor$
75 ★	Wo startet BUILD-MAX-HEAP?	$\lfloor N/2 \rfloor$ — die zweite Hälfte sind Blätter (= triviale Heaps)
76 ★	Komplexität BUILD-MAX-HEAP / MAX-HEAPIFY?	$O(n)$ / $O(\log n)$
77	Heapsort-Phasen?	Aufbauphase und Abbauphase
78 ★	Die 3 Shellsort-Folgen?	Shell $2^n \dots 4, 2, 1 \rightarrow O(n^2)$ · Hibbard $2^n - 1 \rightarrow O(n^{1.5})$ · Pratt $2^p 3^q \rightarrow O(n \log^2 n)$
79	Combsort = ?	Bubblesort (verbessert) + Shellsort, Shrink-Faktor 1,3
80 ★★	Das Theorem zur unteren Schranke?	Jedes vergleichende Sortiervverfahren braucht im Worst Case $\Omega(n \log n)$ Vergleiche ($h \geq \log(n!)$)
81 ★	Die 4 Annahmen für lineares Sortieren?	Paarweise verschieden aus $\{1..n\}$ (Weirdsort) · gleichverteilt (Bucketsort) · m-adisch (Radixsort) · $k = O(n)$ (Countingsort)

#	Frage	Antwort
82 ★	Bucketsort Worst Case und wann?	$O(n^2)$ — alles in einem Bucket bei fehlender Gleichverteilung
83 ★	Radixsort: Phasen und Richtung?	Partitionierungs- und Sammelphase, von der niederwertigsten Stelle (rechts) beginnend; $O(n \cdot m)$
84 ★★	Countingsort: 3 Schritte und Stabilitätsbedingung?	Zählen · Akkumulieren · von hinten einsortieren — ★ nur bei RÜCKWÄRTS durchlaufener Schleife stabil
85	Bubblesort vs. Quicksort bei 1 Mio. Elementen?	3.167 s vs. 0,25 s — Faktor ~12.900

K.4 Suchen und Hashing (86–110)

#	Frage	Antwort
86	3 elementare Suchverfahren?	Sequentiell, binär, hashing-basiert
87 ★	Lineare Suche Average Case?	$\frac{n+1}{2} \approx \frac{n}{2}$
88 ★	2 Voraussetzungen der binären Suche?	Sortiert + wahlfreier Zugriff
89 ★	Wann ist die lineare schneller als die binäre Suche?	Wenn die Position des Elements $< \log n$ ist
90	Exponentielle Suche?	Rechten Rand verdoppeln , dann binär in $[\frac{r}{2}, r]$
91 ★	Interpolationsformel?	$m = \text{left} + \frac{\text{key} - A[\text{left}]}{A[\text{right}] - A[\text{left}]} (\text{right} - \text{left})$
92 ★	Interpolationssuche Best/Worst?	$O(\log \log n)$ / ⚠ $O(n)$ — setzt Gleichverteilung voraus
93	Quadratische Binärsuche?	Sprünge um $\sqrt{n} \rightarrow O(\sqrt{n})$
94 ★	Binärer Suchbaum: Merkformel?	Linkes Kind $<$ Eltern-Knoten $\leq$ Rechtes Kind
95 ★	Catalansche Zahl?	$C(n) = \frac{1}{n+1} \binom{2n}{n}$ — für $n = 4$: 14
96 ★	Die 3 Löschfälle?	Blatt $\rightarrow$ löschen $\cdot$ 1 Nachfolger $\rightarrow$ hochziehen $\cdot$ 2 Nachfolger $\rightarrow$ durch TREE-SUCCESSOR ersetzen
97 ★	Die 4 Rotationsfälle?	LL$\rightarrow$rechts $\cdot$ LR$\rightarrow$Doppel (links-rechts) $\cdot$ RR$\rightarrow$links $\cdot$ RL$\rightarrow$Doppel (rechts-links)
98 ★	Welcher Durchlauf sortiert?	In-Order
99 ★	Hashing-Ziel?	INSERT/DELETE/SEARCH in $O(1)$ bei Platzbedarf $\Theta(K)$
100	Wie heißt Hashing noch?	Streuspeicherung (scatter storage)
101 ★	Java-hashCode-Formel?	$\sum \text{Int}(K[i]) \cdot 31^{\text{KLen}-i}$ — breitere Streuung bei kurzen Ketten
102 ★	3 Eigenschaften guter Hashfunktionen?	Gleichverteilung $\cdot$ Zufälligkeit (ähnliche Schlüssel $\rightarrow$ stark unterschiedliche Indizes) $\cdot$ effizient berechenbar
103 ★	Die 4 Hashfunktionstypen?	Modulo $\cdot$ Multiplikation $\cdot$ Mittelquadrat $\cdot$ universell
104 ★	m bei der Modulo-Methode?	Primzahl, nicht nahe an einer Zweierpotenz, wenige Teiler
105 ★	A bei der Multiplikationsmethode?	$A = \frac{\sqrt{5}-1}{2} = 0,618\dots$ (goldener Schnitt, Knuth)
106 ★	Universelles Hashing: Formel + Kollisionswahrscheinlichkeit?	$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m \cdot P = \frac{1}{m}$
107 ★	Belegungsfaktor?	$\alpha = \frac{n}{m}$ = durchschnittliche Listenlänge
108 ★	Primäres vs. sekundäres Clustering?	Lineare Sondierung: lange zusammenhängende Ketten $\cdot$ Quadratische Sondierung: gleiche Sondierungsfolge für kollidierende Schlüssel
109 ★★	Warum DELETED statt NIL?	Sonst terminiert die Suche zu früh — Schlüssel, die nach k eingefügt wurden, wären unauffindbar
110 ★★	Sondierungen bei $\alpha = 0,5 / 0,9 / 0,99$?	2 / 10 / 100

K.5 Graphen (111–125)

#	Frage	Antwort
111 ★	Grad mit Schlingen?	Schlingen zählen doppelt (+2)
112 ★	Euler-Zyklus (ungerichtet)?	Zusammenhängend + ALLE Grade gerade
113 ★	Euler-Weg (ungerichtet)?	Zusammenhängend + alle gerade ODER genau ZWEI ungerade
114 ★	Euler-Zyklus (gerichtet)?	$g^+(A) = g^-(A)$ für alle Knoten
115 ★	Schnellster Euler-Algorithmus?	Hierholzer , $O(E + V)$
116 ★	BFS/DFS: Datenstruktur + Baumdurchlauf?	Queue / Level-Order · Stack / Pre-Order
117 ★	Cayley-Formel?	n^{n-2} Spannbäume
118 ★	Kruskal vs. Prim (Hilfsstruktur)?	Union-Find vs. Prioritätswarteschlange (Heap)
119 ★	Die 3 Union-Find-Operationen?	MAKE-SET, UNION, FIND-SET
120 ★	Warum die Ackermann-Inverse?	Wächst extrem langsam $\Rightarrow$ Kruskal praktisch $O(E)$
121 ★	Dijkstra: Einschränkung?	⚠ Nur nicht-negative Kanten
122 ★	Bellman-Ford: 3 Vorteile?	Negative Kanten erlaubt · erkennt negative Zyklen · Knoten nie abschließend
123 ★	Floyd-Warshall-Formel?	$\min[m_{i,j}; m_{i,k} + m_{k,j}]$, $O(V ^3)$
124 ★★	Euler vs. Hamilton?	jede KANTE (einfach prüfbar) vs. jeder KNOTEN (NP-schwer)
125 ★★	TSP-Möglichkeiten und Problemklasse?	$\frac{(n-1)!}{2}$ · NP („easy to check, not easy to find“)

K.6 Sprachen und Automaten (126–140)

#	Frage	Antwort
126 ★	Σ^* vs. Σ^+ ?	mit vs. ohne das leere Wort ε
127 ★★	$\emptyset$ vs. $\{\varepsilon\}$?	Leere Sprache (keine Zeichenketten) vs. Sprache, die die leere Zeichenkette enthält — nicht gleich!
128 ★	Die 4 Grammatikbestandteile?	Σ (Terminale), V (Nichtterminale), S (Startsymbol), P (Produktionen)
129 ★★	Chomsky-Hierarchie?	Typ 0 Turingmaschine · Typ 1 kontextsensitiv, linear beschränkte TM · Typ 2 kontextfrei, Kellerautomat · Typ 3 regulär, endlicher Automat
130 ★	Welcher Typ beschreibt Programmiersprachen?	Typ 2 (kontextfrei)
131 ★★	Das Automaten-5-Tupel?	$A = (Q, \Sigma, \delta, q_0, F)$
132 ★	Erweiterte Übergangsfunktion?	$\tilde{\delta}(q, \varepsilon) = q$ und $\tilde{\delta}(q, \omega) = \delta(\tilde{\delta}(q, x), a)$
133 ★★	$L(A)$ bei DEA vs. NEA?	$\tilde{\delta}(q_0, \omega) \in F$ vs. $\tilde{\delta}(q_0, \omega) \cap F \neq \emptyset$
134 ★	Potenzmengenkonstruktion: δ_D ?	$\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a); Q_D \leq 2^n$
135 ★	Was ist Lazy Evaluation?	Nur die vom Startknoten erreichbaren Zustände konstruieren
136 ★	Prioritäten bei RegEx?	Stern > Verkettung > Vereinigung
137 ★	$(01)^*$ vs. 01^* ?	010101... vs. 01111...
138 ★★	Pumping-Lemma: 3 Bedingungen?	$ y > 0 \cdot xy \leq n \cdot \forall k \geq 0 : xy^kz \in L$
139 ★★	Beweisstrategie mit dem Pumping-Lemma?	Annehmen, L sei regulär → ein k finden, für das $xy^kz \notin L$ → Widerspruch
140 ★	BNF: $::=$, $ $, $<$, $>$?	Definitionszeichen · ODER · Nichtterminalsymbole

TEIL L — SPICKZETTEL

L.1 ★★ Die Formeln, die man blind können muss

KOMPLEXITÄT

$1 < \log(n) < \sqrt{n} < n < n \cdot \log(n) < n^2 < n^3 < 2^n < n!$
 $O(g(n)) = \{f \mid \exists c, n_0 : \forall n \geq n_0 : 0 \leq f(n) \leq c \cdot g(n)\}$
 $T(n) = a \cdot T(n/b) + f(n)$ mit $E = \log_b(a)$
 Fall 1: $f \in O(n^{E-\epsilon}) \rightarrow T \in O(n^E)$
 Fall 2: $f \in \Theta(n^E) \rightarrow T \in \Theta(n^E \cdot \log n)$
 Fall 3: $f \in \Omega(n^{E+\epsilon})$ und $a \cdot f(n/b) \leq c \cdot f(n) \rightarrow T \in \Theta(f(n))$
 Geometrische Reihe: $\sum x^k = 1/(1-x)$

DATENSTRUKTUREN

Bits: $n = 2^k$
 Feld-Adresse: $\text{AddrOf}(a[i]) = \text{AddrOf}(a) + (i - \text{first}) \cdot \text{elementSize}$
 Binärbaum: Knoten/Ebene 2^i · gesamt $2^{(h+1)} - 1$ · $h = \log(|K|+1) - 1$
 Heap/Array: links $2i$ · rechts $2i+1$ · Elter $\lfloor i/2 \rfloor$ · Start $\lfloor N/2 \rfloor$
 IEEE 754 float: 1 | 8 | 23 Bit, Bias 127 double: 1 | 11 | 52, Bias 1023

SORTIEREN

Untere Schranke: $h \geq \log(n!) \approx \Omega(n \cdot \log n)$
 Selectionsort: $\Sigma(n-i) = n(n-1)/2 \approx n^2/2$

HASHING

Belegungsfaktor: $\alpha = n/m$
 Verkettung: $O(1 + \alpha)$
 Offene Adr.: erfolglos $\leq 1/(1-\alpha)$ · erfolgreich $\leq (1/\alpha) \cdot \ln(1/(1-\alpha))$
 Modulo: $hc = k \bmod m$ (m Primzahl!)
 Multiplikation: $hc = \lfloor (k \cdot A \bmod 1) \cdot m \rfloor$ ($A = (\sqrt{5}-1)/2$)
 Universell: $h_{\{a,b\}}(k) = ((ak+b) \bmod p) \bmod m$

GRAPHEN

Grad (ungerichtet): #Kanten zu anderen + 2 · #Schlingen
 Grad (gerichtet): $g^+(A) + g^-(A)$
 Planar: $|V| - |E| + |A| = 2$ bzw. $|E| \leq 3|V| - 6$
 Cayley: #Spannbäume = $n^{(n-2)}$
 Floyd-Warshall: $\min[m(i,j) ; m(i,k) + m(k,j)]$
 TSP: $(n-1)! / 2$

AUTOMATEN

Automat: $A = (Q, \Sigma, \delta, q_0, F)$
 $\delta(q, \epsilon) = q$ $\delta(q, w) = \delta(\delta(q, x), a)$
 DEA: $L(A) = \{w \mid \delta(q_0, w) \in F\}$
 NEA: $L(A) = \{w \mid \delta(q_0, w) \cap F \neq \emptyset\}$
 $\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$ $|Q_D| \leq 2^n$
 Pumping-Lemma: $w = xyz$ mit $|y| > 0$, $|xy| \leq n$, $\forall k \geq 0: xy^kz \in L$

L.2 ★★ Die Entscheidungshilfen

Welches SORTIERVERFAHREN?

Kleine/fast sortierte Daten	→ INSERTIONSORT	$O(n)$ best
Allgemeiner Fall, Praxis	→ QUICKSORT	schnellster real
Worst-Case-GARANTIE nötig	→ HEAPSORT	in-place + $O(n \log n)$
STABILITÄT nötig	→ MERGESORT	stabil, parallelisierbar
Ganzzahlige Schlüssel, $k = O(n)$	→ COUNTINGSORT	$O(n+k)$
Mehrstellige Schlüssel	→ RADIXSORT	$O(n \cdot m)$
Gleichverteilte reelle Zahlen	→ BUCKETSORT	$O(n)$

Welches SUCHVERFAHREN?

Unsortierte Daten	→ LINEARE SUCHE	$O(n)$
Sortiert + wahlfreier Zugriff	→ BINÄRE SUCHE	$O(\log n)$
Sortiert + GLEICHVERTEILT	→ INTERPOLATION	$O(\log \log n)$
Element vermutlich weit vorne	→ EXPONENTIELLE S.	$O(\log k)$
Dynamisches Einfügen/Löschen	→ BINÄRER SUCHBAUM	$O(\log n)$ (balanciert!)
Maximale Geschwindigkeit	→ HASHING	$O(1)$

Welcher GRAPHEN-Algorithmus?

Alle Knoten besuchen (ebenenweise)	→ BFS (Queue)
Alle Knoten besuchen (tief zuerst)	→ DFS (Stack)
Jede KANTE genau einmal	→ EULER (Hierholzer)
Jeden KNOTEN genau einmal	→ HAMILTON ⚠ NP-schwer
Minimaler Spannbaum, viele Kanten	→ PRIM
Minimaler Spannbaum, wenige Kanten	→ KRUSKAL
Kürzester Weg, nicht-negativ	→ DIJKSTRA
Kürzester Weg, negative Kanten	→ BELLMAN-FORD
Kürzeste Wege ALLER Paare	→ FLOYD-WARSHALL
Rundreise (TSP)	→ Heuristik (Christofides, k-Opt) ⚠ NP

Welche KOLLISIONSBEHANDLUNG?

α kann > 1 werden, häufiges Löschen	→ VERKETTUNG
Kein Zusatzspeicher, $\alpha < 1$	→ OFFENE ADRESSIERUNG
... einfach, aber Clustering	→ Lineare Sondierung
... kein primäres Clustering	→ Quadratische Sondierung
... kein sekundäres Clustering	→ DOPPELTES HASHING

L.3 ★ Die Zahlen, die man kennen sollte

Zahl	Bedeutung
$2^{10} = 1024$	1 KByte
127 / 1023	IEEE-754-Bias (float/double)
8 / 23 bzw. 11 / 52	Exponent/Mantisse (float/double)
31	Multiplikator in Javas <code>hashCode</code>
0,618033980	A = goldener Schnitt (Multiplikationsmethode)
1,3	Shrink-Faktor bei Combsort
$1,4 \leq c \leq 2$	Konstante im Quicksort-Average-Case
2/3 : 1/3	Heapify-Worst-Case-Verhältnis
23 / 50,7 %	Geburtstagsparadoxon
2 / 10 / 100	Sondierungen bei $\alpha = 0,5 / 0,9 / 0,99$
$3^3 = 27 \geq 24$	12-Kugeln-Rätsel: warum 3 Wägungen reichen
$A(4, 2) > 10^{19729}$	Ackermann-Funktion
121.645.100.408.832.000	TSP-Möglichkeiten bei 20 Knoten
30 Mio. US-\$	TSP-Ersparnis pro Jahr pro Meile/Tag
5.000 / 1,742 · 10¹⁸ FLOPS	ENIAC (1946) / El Capitan (2025)

L.4 ★★ Prüfungsstrategie (Portfolio!)

- ① THEMENWAHL bis 13.10.
- Wählen Sie ein Thema, zu dem es EINE IMPLEMENTIERUNG geben kann – die stärksten Projekte der Liste kombinieren THEORIE + CODE + MESSUNG (z. B. Gruppe 4: GUI für Sortiervorgänge, Gruppe 9: Dijkstra/Bellman-Ford/Floyd-Warshall im Performance-Vergleich)
- ② PROJEKTBERICHT (60 Punkte, Deadline 10.11.)
- 3–6 Seiten PRO Gruppenmitglied
 - Gestaltungsrichtlinien wissenschaftlichen Arbeitens einhalten
 - Selbstständigkeitserklärung nicht vergessen!
 - KI-Einsatz VOLLSTÄNDIG dokumentieren: nicht nur den Lösungsweg, sondern auch den ZWECK der Anwendung
 - Abgabe als PDF über Moodle oder per Mail
- Empfohlene Gliederung:
1. Einleitung + Problemstellung
 2. Theoretische Grundlagen (Definitionen, Pseudocode, Komplexitätsanalyse)
 3. Implementierung / Methodik
 4. Ergebnisse (★ MESSUNGEN! Laufzeittabellen wie bei Prof. Eggendorfer)
 5. Diskussion / Grenzen
 6. Fazit + Ausblick
 7. Literatur- und Hilfsmittelverzeichnis (inkl. KI!)
- ③ PRÄSENTATION (40 Punkte, Deadline 10.11., Vortrag 11.11.)
- 20–30 Min. pro Gruppe, ca. 10 Min. pro Teilnehmer
 - ⚠ KAMERA AN während des Vortrags – Prüfungsbedingung!
 - ⚠ ANWESENHEITSPFLICHT bei ALLEN Präsentationen
 - ⚠ JEDE Gruppe muss MINDESTENS EINE FRAGE pro Präsentation stellen
- ④ BESTEHENSGRENZE: $\Sigma \geq 50$ von 100 Punkten

L.5 ★ Die Kapitel in je einem Satz

Ka p.	Kernaussage
01	Von der Kerbe (30.000 v. Chr.) über Al-Chwarizmi (Algorithmus), Leibniz (binär), Jacquard (Lochkarte), Babbage/Lovelace (programmierbar), Turing (berechenbar) bis El Capitan (1,742 EFLOPS).
02	Ein Algorithmus ist eine endliche, eindeutige, schrittweise Problemlösung — und das Halteproblem beweist, dass es Grenzen gibt .
03	Die O-Notation beschreibt nicht die Laufzeit, sondern das Wachstum ; das Master-Theorem löst Rekursionen in drei Fällen.
04	Vergleichendes Sortieren kann nie besser als $\Omega(n \log n)$ werden — schneller geht es nur, wenn man nicht vergleicht (Counting-/Radix-/Bucketsort).
05	Halbieren macht aus $O(n)$ ein $O(\log n)$ — aber nur, wenn die Daten sortiert sind und man wahlfrei zugreifen kann.
06	Hashing tauscht Speicher gegen Zeit : $O(1)$ ist erreichbar, aber Kollisionen sind unvermeidbar und die Kosten explodieren gegen $\alpha \rightarrow 1$.
07	Euler (Kanten) ist einfach, Hamilton (Knoten) ist NP — und für das TSP bleiben nur Heuristiken mit oder ohne Gütegarantie .
08	Die Chomsky-Hierarchie ordnet Sprachen den Automaten zu; das Pumping-Lemma beweist, was ein endlicher Automat nicht kann — nämlich zählen .

Ende Modul 17 — Theoretische Informatik, Prof. Dr. Aykan D. Inan, DHBW Ravensburg WDS24.